



SIRC-1 CPU Reference Manual

Featuring the SIRCIS Instruction Set

Technical Specifications

Architecture:	16-bit RISC Load/Store
Address Bus:	24-bit word address
Data Bus:	16-bit
Instruction Size:	32-bit (fixed)
Registers:	16 x 16-bit registers
Execution:	6-stage execution phases

Version 1.0

Copyright Silicon Integrated Research Corporation (SIRC) 1989

September 12, 2026

Preface

About This Manual

This reference manual documents the SIRC-1 CPU architecture and the SIRCIS (SIRC Instruction Set) instruction set completely enough that an assembler, compiler backend, emulator, or hardware implementation can be built from the manual alone, without consulting any other source.

It is written for:

- Assembly language programmers
- Compiler and toolchain developers
- Hardware designers and implementers
- Emulator and simulator authors
- System software developers

Assumed Knowledge

This manual assumes the reader is already familiar with general computer architecture concepts: registers, memory addressing, binary and hexadecimal number representation, and assembly-language programming in general. It teaches the SIRC-1 architecture specifically; it does not teach these prerequisites from first principles.

How to Read This Manual

Part I introduces the architecture as a whole: registers, data representation, the status register, and exception handling. Part II covers the instruction encoding formats, addressing modes, shift operations, and condition codes that every instruction is built from. Part III is the instruction reference itself, one dedicated entry per instruction; Chapter 11 explains the fields every entry uses (Opcodes, Syntax, Operands, Operation, Flags, Exceptions, Timing, Privilege, and so on) before the entries begin, and is worth reading first even for a reader who intends to use the rest of the manual only as a lookup reference. The appendices collect quick lookups (the complete opcode map, timing figures, undocumented-opcode notes, worked examples, and an alphabetical mnemonic index) that restate information from the main chapters in a faster-to-scan form; they never introduce a new architectural fact that isn't already established in Parts I–III.

Scope

This manual covers the SIRC-1 hardware itself: the instruction set architecture, register model, bus interface, exception handling, and coprocessor interface. It does not cover software conventions built on top of that hardware — register usage conventions, calling conventions, stack frame layout, or object/binary file format — which are the concern of the (forthcoming) SIRC-1 ABI and Toolchain documentation, not the hardware architecture. See Chapter 1's Scope section for the full statement.

Revision Status

This is Version 1.0 of the SIRC-1 CPU Reference Manual. Except where an entry explicitly notes otherwise, every instruction, register, and behavior described here applies to *SIRC-1*, *all revisions*.

Contents

Preface	iii
Notation and Glossary	xix
I Architecture Overview	1
1 Introduction	3
1.1 Overview	3
1.2 Design Philosophy	3
1.2.1 RISC Principles	3
1.2.2 Design Goals	3
1.2.3 Applications	3
1.3 Key Features	4
1.3.1 Architecture Characteristics	4
1.3.2 Execution Model	4
1.3.3 Coprocessors	4
1.3.4 Privilege Levels	5
1.4 Memory Organization	5
1.4.1 Address Space	5
1.4.2 Memory Map	5
1.4.3 Alignment	6
1.5 Document Organization	6
1.5.1 Scope	6
2 CPU Architecture	7
2.1 Overview	7
2.2 Six-Stage Execution Phases	7
2.3 Architectural Components	8
2.3.1 Control Unit (CU)	8
2.3.2 Arithmetic Logic Unit (ALU)	8
2.3.3 Addressing Unit (AU)	9
2.3.4 Register File	9
2.3.5 Barrel Shifters	9
2.4 Design Characteristics	9
2.4.1 Combinatorial Logic Implementation	9
2.4.2 Fixed Execution Sequence	10
2.4.3 Parallel Functional Units	10
2.5 External Interface	10
3 Register Model	11
3.1 Overview	11
3.2 General-Purpose Registers	11
3.2.1 Register Set	11
3.2.2 Usage Conventions	11
3.3 Address Register Pairs	11
3.3.1 Overview	11

3.3.2	Link Register (l)	12
3.3.3	Address Register (a)	12
3.3.4	Stack Pointer (s)	12
3.3.5	Program Counter (p)	13
3.4	Status Register	13
3.5	Privilege Restrictions	13
3.5.1	High Address Register Restrictions	13
3.6	Register Addressing	14
3.7	Usage Examples	14
3.7.1	Basic Register Operations	14
3.7.2	Address Register Usage	15
3.7.3	Subroutine Call	15
4	Data Representation and Memory Model	17
4.1	Overview	17
4.2	Word and Byte Ordering	17
4.3	Address Values	17
4.4	Instruction Storage	18
4.5	Exception Vector Storage	18
4.6	Immediate Values	18
4.7	Software Layout Conventions	19
4.8	Arithmetic and Address Wraparound	19
4.9	Memory Ordering	20
5	Status Register	21
5.1	Overview	21
5.2	Status Register Layout	21
5.3	Condition Flags (Non-Privileged)	21
5.3.1	Zero Flag (Z) – Bit 0	21
5.3.2	Negative Flag (N) – Bit 1	21
5.3.3	Carry Flag (C) – Bit 2	21
5.3.4	Overflow Flag (V) – Bit 3	22
5.3.5	Reserved Flags – Bits 4–7	22
5.4	Control Flags (Privileged)	22
5.4.1	Protected Mode (P) – Bit 8	22
5.4.2	Hardware Interrupt Enable Bits (E1-E4) – Bits 9–12	22
5.4.3	Exception Active (EA) – Bit 13	23
5.4.4	Trap on Address Overflow (A) – Bit 14	23
5.4.5	Trace Mode (T) – Bit 15	23
5.5	Status Register Updates	24
5.5.1	Automatic Updates	24
5.5.2	Manual Updates	24
5.6	Reading the Status Register	24
5.7	Exception Handling Effects	25
5.8	Usage Examples	25
5.8.1	Example 5-1: Conditional Execution	25
5.8.2	Example 5-2: Manual Status Flag Control	25
5.8.3	Example 5-3: Testing Bits	26
5.8.4	Example 5-4: Enabling Protected Mode	26
5.8.5	Example 5-5: Saving and Restoring Status	26

6	Exception and Interrupt Handling	27
6.1	Overview	27
6.1.1	Instructions Used in This Chapter	27
6.2	Exception Types	27
6.2.1	Faults	27
6.2.2	Hardware Exceptions (Interrupts)	30
6.2.3	User Exceptions (Traps)	30
6.3	Exception Registers	30
6.3.1	Link Registers	30
6.3.2	Cause Register	33
6.3.3	Hardware Interrupt Enable Bits	34
6.3.4	Pending Hardware Interrupts	34
6.4	Exception Processing	36
6.4.1	Exception Priority	36
6.4.2	Exception Handling Flow	37
6.4.3	Exception Entry Side Effects	37
6.4.4	Vector Table	38
6.4.5	Exception Quick Reference	39
6.5	Reset and Startup	39
6.5.1	Reset State	40
6.5.2	Reset Output Hold	40
6.5.3	Reset Vector Fetch	40
6.6	Return from Exception	41
6.6.1	Accessing Link Registers	41
6.7	Exception Coprocessor Instructions	42
6.7.1	Internal Instructions	42
II	Instruction Set Architecture	43
7	Instruction Formats	45
7.1	Overview	45
7.2	Format Overview	45
7.3	Common Fields	45
7.4	Immediate Format	45
7.4.1	Structure	45
7.4.2	Encoding Examples	47
7.5	Short Immediate Format (with Shift)	47
7.5.1	Structure	47
7.5.2	Encoding Examples	48
7.6	Register Format	49
7.6.1	Structure	49
7.6.2	Register Operand Semantics	49
7.6.3	Encoding Examples	50
7.7	Operand Encoding Details	50
7.7.1	Condition Codes	51
7.7.2	Shift Types	51
7.8	Opcode Encoding Patterns	51
7.8.1	Opcode Groups	51
7.8.2	Test vs. Save	52
7.9	Instruction Fetch and Alignment	52
7.9.1	Fetch Process	52

7.9.2 Alignment Requirements	52
7.10 Format Selection Guidelines	52
8 Addressing Modes	55
8.1 Overview	55
8.2 Addressing Mode Matrix	56
8.3 Legal Modes by Instruction Family	56
8.4 Zero-Displacement Shorthand	57
8.5 Operand Categories	57
8.6 Common Address Calculation Rules	58
8.7 Immediate Addressing	58
8.7.1 Description	58
8.7.2 Syntax	58
8.7.3 Effective Value	58
8.7.4 Usage	58
8.8 Register Direct Addressing	58
8.8.1 Description	58
8.8.2 Syntax	59
8.8.3 Effective Value	59
8.8.4 Usage	59
8.9 Address Register Direct Addressing	59
8.9.1 Description	59
8.9.2 Syntax	59
8.9.3 Effective Value	59
8.9.4 Usage	59
8.10 Indirect Immediate Addressing	59
8.10.1 Description	59
8.10.2 Syntax	59
8.10.3 Effective Address	60
8.10.4 Usage	60
8.11 Indirect Register Addressing	60
8.11.1 Description	60
8.11.2 Syntax	60
8.11.3 Effective Address	60
8.11.4 Usage	60
8.12 Post-Increment Addressing	60
8.12.1 Description	60
8.12.2 Syntax	60
8.12.3 Operation Sequence	61
8.12.4 Usage	61
8.13 Pre-Decrement Addressing	61
8.13.1 Description	61
8.13.2 Syntax	61
8.13.3 Operation Sequence	61
8.13.4 Usage	61
8.14 Short Immediate Addressing	61
8.14.1 Description	61
8.14.2 Syntax	61
8.14.3 Effective Value	62
8.14.4 Usage	62
8.15 Address Register Selection	62
8.16 Addressing Mode Examples	62

8.16.1 Example 8-1: Structure Access	62
8.16.2 Example 8-2: Array Iteration	62
8.16.3 Example 8-3: Stack Frame	62
8.17 Addressing Mode Restrictions	63
8.17.1 Privilege Mode Restrictions	63
8.17.2 Alignment	63
9 Shift Operations	65
9.1 Overview	65
9.2 Shift Types	65
9.2.1 Shift Type Encoding	65
9.3 Shift Syntax	65
9.4 Shift Type Details	66
9.4.1 Logical Shift Left (LSL)	66
9.4.2 Logical Shift Right (LSR)	66
9.4.3 Arithmetic Shift Left (ASL)	67
9.4.4 Arithmetic Shift Right (ASR)	67
9.4.5 Rotate Left (RTL)	67
9.4.6 Rotate Right (RTR)	68
9.5 Shift Operands	68
9.5.1 Mode 0: Shift by Immediate	69
9.5.2 Mode 1: Shift by Register Count	69
9.6 Shift Count	69
9.7 Status Flag Effects	69
9.7.1 Default Behavior	69
9.7.2 Carry Flag	70
9.7.3 Zero Flag	70
9.7.4 Negative Flag	70
9.7.5 Overflow Flag	70
9.8 Common Use Cases	70
9.8.1 Multiplication by Constants	70
9.8.2 Division by Powers of 2	70
9.8.3 Bit Field Extraction	70
9.8.4 Bit Manipulation	71
9.9 Multi-Word Shifts	71
9.9.1 32-bit Left Shift	71
9.9.2 32-bit Right Shift (Unsigned)	71
9.9.3 32-bit Right Shift (Signed)	71
9.10 Performance Considerations	72
9.11 Limitations	72
9.12 Status Flag Update Control	72
9.12.1 Status Override Syntax	72
9.12.2 Examples	72
9.12.3 Use Cases	73
9.12.4 Assembler Syntax Notes	73
10 Condition Codes	75
10.1 Overview	75
10.2 Condition Code Encoding	75
10.3 Condition Code Categories	75
10.3.1 Simple Conditions	75
10.3.2 Unsigned Comparisons	76

10.3.3 Signed Comparisons	76
10.3.4 Special Conditions	76
10.4 Assembly Syntax	77
10.5 Usage with Compare Instructions	77
10.5.1 Example 10-1: Unsigned Comparison	77
10.5.2 Example 10-2: Signed Comparison	77
10.5.3 Example 10-3: Equality Testing	77
10.6 Condition Code Truth Tables	78
10.6.1 Unsigned Comparison (CMPR rA, rB)	78
10.6.2 Signed Comparison (CMPR rA, rB)	78
10.7 Advanced Usage	78
10.7.1 Example 10-4: Conditional Assignment	78
10.7.2 Example 10-5: Conditional Increment	79
10.7.3 Example 10-6: Conditional Function Call	79
10.7.4 Example 10-7: Loop Construction	79
10.8 Condition Code Selection Guide	79
10.9 Performance Considerations	79
10.10 Common Pitfalls	80
 III SIRCIS Instruction Reference	 81
 11 Reading Instruction Descriptions	 83
11.1 Instruction Entry Fields	83
11.2 Notation	84
11.3 Status Flag Effect Symbols	84
11.4 Conditional Execution	85
11.5 Status Update Overrides	85
 12 Instruction Summary	 87
12.1 Overview	87
12.2 Complete Instruction List	88
12.3 Instruction Grouping Patterns	89
12.3.1 Opcode Organization	89
12.3.2 Save vs. Test Variants	89
12.4 Meta-Instructions	89
12.5 Instruction Timing	90
12.6 Next Chapters	90
 13 ALU Instructions	 91
13.1 Overview	91
13.2 ALU Instruction Families	91
13.2.1 Arithmetic Instructions	91
13.2.2 Logical Instructions	91
13.2.3 Data Movement	91
13.2.4 Test and Compare	91
13.3 Legal Forms	91
13.4 Common ALU Semantics	93
13.5 Status Flag Updates	93
13.5.1 Manual Flag Update Control	94
SHFT – Shift with Status Update Meta-Instruction	96
ADDI / ADDR – Add Immediate / Add Register	97
ADCI / ADCR – Add with Carry	99

SUBI / SUBR – Subtract Immediate / Subtract Register	101
SBCI / SBCR – Subtract with Carry	103
ANDI / ANDR – AND Immediate / AND Register	105
ORRI / ORRR – OR Immediate / OR Register	107
XORI / XORR – XOR Immediate / XOR Register	109
LOAD – Load/Move	111
CMPI / CMPR – Compare Immediate / Compare Register	113
TSAI / TSAR – Test AND Immediate / Test AND Register	115
TSXI / TSXR – Test XOR Immediate / Test XOR Register	117
13.6 Summary	118
14 Memory Access Instructions	119
14.1 Overview	119
14.2 Effective Address Calculation	119
14.3 Legal Forms	120
14.4 Common Memory Semantics	121
14.5 Status Flag Effects	122
LOAD – Load from Memory	123
STOR – Store to Memory	125
14.6 Common Memory Access Patterns	126
14.6.1 Structure Field Access	126
14.6.2 Array Iteration	126
14.6.3 Stack Operations	127
14.7 Shift Operations with Memory Instructions	127
14.7.1 STOR with Shift	127
14.7.2 Supported Shift Types	127
14.7.3 Shift Limitations	127
14.7.4 Use Cases	128
14.7.5 Status Flags	128
15 Control Flow Instructions	129
15.1 Overview	129
15.2 Legal Forms	129
15.3 Common Semantics	131
15.4 Status Flag Effects	131
15.5 Branch Meta-Instructions	132
BRAN – Branch Meta-Instruction	133
BRSR – Branch to Subroutine Meta-Instruction	134
RETS – Return from Subroutine Meta-Instruction	135
15.6 Load Effective Address	135
LDEA – Load Effective Address	136
LDEL – Load Effective Address and Link	138
15.7 Long Jump Meta-Instructions	139
LJMP – Long Jump Meta-Instruction	140
LJSR – Long Jump to Subroutine Meta-Instruction	141
15.8 Control Flow Patterns	142
15.8.1 If-Then-Else	142
15.8.2 While Loop	142
15.8.3 Switch/Jump Table	142

16 Coprocessor Instructions	145
16.1 Overview	145
16.2 Legal Forms	145
16.3 Common Semantics	146
COPI/COPR – Coprocessor Call	148
16.4 Coprocessor Compatibility	149
16.4.1 Model and Revision Policy	149
16.5 Exception Unit (Coprocessor 1)	150
16.5.1 Common Operations	150
16.6 Exception Unit Meta-Instructions	150
EXCP – User Exception Meta-Instruction	151
WAIT – Wait for Interrupt Meta-Instruction	152
RETE – Return from Exception Meta-Instruction	153
RSET – System Reset Meta-Instruction	154
ETFR/ETTR – Exception Link Transfer Meta-Instructions	155
16.7 DMA Unit (Coprocessor 2)	156
16.7.1 DMA Operations	156
DMAR – DMA Read To Registers Meta-Instruction	158
DMAW – DMA Write From Registers Meta-Instruction	160
DMAT – DMA Memory Transfer Meta-Instruction	162
16.8 Integer Maths Unit (Coprocessor 3)	163
16.8.1 Maths Operations	163
MULU/MULS – Integer Multiply Meta-Instructions	164
DIVU/DIVS – Integer Divide Meta-Instructions	165
17 Meta-Instructions	167
17.1 Overview	167
NOOP – No Operation	168
A Opcode Map	169
A.1 Complete Opcode Reference	169
A.2 Opcode Patterns	170
A.2.1 Format Identification	170
A.2.2 Save vs. Test	170
A.2.3 Memory Operations Decoding	171
A.3 Undocumented Instructions	171
A.4 Quick Lookup Table	171
B Instruction Timing	173
B.1 Overview	173
B.2 Six-Stage Execution Phases	173
B.3 Timing Characteristics	173
B.3.1 Exception and Fault Timing	174
B.4 Conditional Execution	174
B.5 Program Timing Examples	174
B.5.1 Simple Loop	174
B.5.2 Function Call Overhead	174
B.5.3 Conditional Block	175
B.6 Performance Considerations	175
B.6.1 No Pipeline Stalls	175
B.6.2 Memory Access	175
B.6.3 Optimization Guidelines	175
B.7 Theoretical Performance	176

C	Undocumented Instructions	177
C.1	Overview	177
C.2	Undefined Operand Combinations	177
C.2.1	Aliased Address-Register Writes	177
C.3	Undocumented Opcode List	178
C.4	Observed Behavior	178
C.4.1	0x08, 0x28, 0x38 (ADD Test)	178
C.4.2	0x09, 0x29, 0x39 (ADC Test)	178
C.4.3	0x0B, 0x2B, 0x3B (SBC Test)	178
C.4.4	0x0D, 0x2D, 0x3D (OR Test)	179
C.5	Implementation Notes	179
C.5.1	Current Simulator	179
C.5.2	Hardware Implementation	179
C.6	Why Undocumented?	179
C.7	Recommendations	179
C.7.1	For Application Developers	179
C.7.2	For Compiler Writers	179
C.7.3	For Hardware Implementers	179
C.7.4	For Researchers/Hackers	180
C.8	Future Standardization	180
C.9	Opcode Space for Expansion	180
D	Example Programs	181
D.1	Byte Sieve	181
D.2	Faults	182
D.3	Hardware Exception	187
D.4	Software Exception	192
D.5	Store Load	193
E	Quick Reference	197
E.1	Instruction Mnemonic Index	197
E.2	Status Register Bit Reference	199
E.3	Bus Access Type (BAT) Reference	200
E.4	Where to Find Other Tables	200
F	External Interface	201
F.0.1	Chip Layout	201
F.0.2	Pin Descriptions	202
F.0.3	Timing Notes	206

List of Tables

1	Manual-Wide Typographic Conventions	xix
1.1	SIRC-1 CPU Specifications	4
1.2	List of coprocessors	5
1.3	Exception Vector Table Regions	6
3.1	Register Encoding	14
4.1	External Byte Order for 16-bit Words	17
4.2	Address Register Pair Interpretation	17
4.3	Stored 24-bit Address Format	18
4.4	Immediate Field Interpretation	19
6.1	Exception Link Register Assignment	31
6.2	Fault-Inside-L5-Handler Timeline	36
6.3	Exception Quick Reference	39
6.4	Reset State	40
6.5	Exception Coprocessor Instruction Summary	42
7.1	Instruction Format Summary	45
7.2	Immediate Format Encoding Examples	47
7.3	Short Immediate Format Encoding Examples	48
7.4	Register Format Encoding Examples	50
7.5	Condition Code Encoding	51
7.6	Shift Type Encoding	51
7.7	Format Selection Examples	52
8.1	Addressing Modes Summary	55
8.2	Assembler Operand Forms Mapped to Addressing Modes	55
8.3	Addressing Mode Side Effects and Legal Instruction Families	56
8.4	Addressing Mode Legality by Instruction Family	57
8.5	Zero-Displacement Shorthand	57
8.6	Address Register Encoding	62
9.1	Shift Type Encoding	65
9.2	Examples of first source operands	66
9.3	Shift Operand Modes	68
9.4	Status Flag Override Modifiers	72
10.1	Condition Code Encoding	75
10.2	Unsigned Comparison Results	78
10.3	Signed Comparison Results	78
10.4	Condition Code Selection Guide	79
11.1	Instruction Description Notation	84
11.2	Status Flag Effect Symbols	85
11.3	Status Update Source Encoding	85
12.1	Complete SIRCIS Instruction Set	88
12.2	Meta-Instructions	89

13.1 Legal ALU Instruction Forms	92
13.2 Common ALU Semantics	93
13.3 ALU Status Flag Effects	94
14.1 Legal Memory Instruction Forms	120
14.2 Common Memory Instruction Semantics	121
14.3 Memory Instruction Status Flag Effects	122
15.1 Legal Control-Flow and Effective-Address Forms	129
15.2 Common Control-Flow Instruction Semantics	131
15.3 Control-Flow Instruction Status Flag Effects	132
16.1 Legal Coprocessor Instruction Forms	145
16.2 Common Coprocessor Instruction Semantics	146
16.3 Standard Coprocessor IDs	149
16.4 Exception Unit Operations	150
16.5 DMA Unit Operations	156
16.6 Integer Maths Unit Operations	163
16.7 Maths Status Word in <code>r3</code>	163
17.1 Meta-Instruction Cross-Reference	167
A.1 Complete SIRCIS Opcode Map	169
A.2 Instruction Variant Quick Reference	171
B.1 Instruction Timing	173
C.1 Undocumented Instruction Opcodes	178
E.1 Alphabetical Mnemonic Index	197
E.2 Status Register Bit Summary	199
E.3 Bus Access Type (BAT) Encoding	200
E.4 Reference Table Locations	200
F.1 SIRC CPU 64-pin DIP pinout	202
F.2 Bus Access Type Bits Meaning	204
F.3 CPU bus output timing rules	206
F.4 Bus response timing rules	207
F.5 External bus cycle ownership	207
F.6 External input sampling rules	212

List of Figures

2.1	SIRC-1 CPU Execution Phase Architecture	8
3.1	Address Register Pair Formation	12
5.1	Status Register Bit Layout	21
6.1	Exception Link Register Saved-State Layout	31
6.2	Fault Metadata Register Status Field Layout	32
7.1	Immediate Instruction Format	46
7.2	Short Immediate with Shift Format	48
7.3	Register Instruction Format	49
9.1	Logical Shift Left	66
9.2	Logical Shift Right	66
9.3	Arithmetic Shift Right	67
9.4	Rotate Left	67
9.5	Rotate Right	68
F.1	SIRC CPU 64-pin DIP pinout, top view	201
F.2	Instruction-fetch high-word and low-word reads with no wait states	208
F.3	Data read cycle with no wait state	208
F.4	Data write cycle with no wait state	208
F.5	Exception-vector fetch with high-word then low-word reads	209
F.6	Wait-state insertion when no bus response is asserted	209
F.7	Bus-error and bus-protection abort timing	209
F.8	Complete register-only ALU instruction with no wait states	210
F.9	Complete LOAD instruction with no wait states	210
F.10	Hardware reset with one-cycle RSTI assertion and reset-vector fetch	211
F.11	Enabled IRQ sampled at an instruction boundary and dispatched with no wait states	211

Notation and Glossary

This section collects the typographic conventions, normative vocabulary, and architectural terminology used throughout the manual, in one place, so a reader does not have to hunt through individual chapters to look one up. Instruction-entry-specific notation (`rD`, `rS1`, `shift`, and similar placeholder names used only inside instruction descriptions) is defined separately in Chapter 11, Table 11.1, alongside the status-flag-effect symbols (Table 11.2); this section does not repeat those, only the conventions that apply manual-wide.

Typographic Conventions

Convention	Meaning
<code>r1, sr, ph</code>	Register names appear in typewriter font via <code>\reg{}</code> , whether general-purpose, address-pair, or status.
<code>0x1A</code>	A literal instruction-opcode byte value, via <code>\opcode{}</code> . Renders as <code>0x1A</code> ; the macro takes just the two hex digits.
<code>#123</code>	A concrete immediate value written as an operand, via <code>\imm{}</code> . Renders as <code>#123</code> .
<code>0x0002</code>	A concrete memory address or vector target, via <code>\addr{}</code> .
ADDI	An instruction mnemonic, always bold typewriter and always uppercase, via <code>\mnemonic{}</code> .
<code>SR.X</code>	A field or bit of the status register, named by its bit label (for example <code>SR.A</code> , <code>SR.T</code>); see Chapter 5 for the full bit layout.
<code>(#4, a)</code>	Parentheses denote indirection through an address-register pair; a leading <code>-</code> denotes pre-decrement and a trailing <code>+</code> denotes post-increment, for example <code>-(0, a)</code> and <code>(0, a)+</code> .

Table 1: Manual-Wide Typographic Conventions

Other conventions that do not have a dedicated macro:

- **Hex literals** that aren't a register, opcode, immediate, or address value in one of the forms above (status-word values, encoded field values, bit patterns, coprocessor command words) are written `0x` followed by uppercase hex digits, for example `0x1A` rather than `0x1a`.
- **Binary literals** in tables are written as a bare digit group with no prefix, for example `00`, `10`, `111` rather than `0b10`.
- **Bit numbers** are written as plain text: "bit 15", "bits 4–7". Bit 15, or the highest-numbered bit of whatever field is under discussion, is always the most significant bit, and is always drawn on the left in every bit-field diagram.
- **Numeric ranges** use an en dash, for example `0x00–0x0F` or "bits 5–4", both in prose and in tables.

- **Word size:** "16-bit word" is stated once per chapter or major section on first use, then shortened to plain "word" for the rest of that section.

Normative Vocabulary

The following words carry a specific technical meaning whenever they appear in this manual, distinct from their everyday English sense:

must / must not A mandatory architectural requirement. Hardware or software that does not comply is not a conformant SIRC-1 implementation.

may A permitted option. The CPU, the assembler, or the programmer is free to choose either way.

should / should not A recommendation, not a conformance requirement.

reserved A field, opcode, or coprocessor ID that is not currently assigned a meaning. Software must not assume any particular value when reading a reserved field, and must write zero unless a chapter states an explicit encoding. A future CPU revision may assign a reserved field a meaning.

architecturally undefined The architecture does not specify the result. It may vary by CPU implementation, by revision, or even between two executions of the identical instruction on the identical CPU; software must never depend on any particular outcome.

implementation-defined Behavior that is fixed and consistent within one particular CPU implementation or model, but that may differ between models – for example, the extra cycles an optional maths coprocessor takes to complete a multiply. This is a stronger guarantee than *architecturally undefined*: a given implementation's behavior is at least self-consistent, even though the manual does not pin down what that behavior is.

Register Name Conventions

SIRC-1 has sixteen 16-bit registers: seven general-purpose registers `r1–r7`, four address register pairs (`l`, `a`, `s`, `p`), and the status register `sr`. Each address register pair splits into a high and low half, named by appending `h/l` to the pair name – `lh/ll`, `ah/al`, `sh/sl`, `ph/pl` – so that either half can be addressed as an ordinary 16-bit register on its own, or the pair addressed together as a 24-bit address. See Chapter 3 for the full register model, encoding, and privilege rules.

Glossary of Architectural Terms

Coprocessor One of the specialized execution units a `COP1/COPR` call can dispatch to – the processing unit itself (coprocessor `0x0`), the exception unit (`0x1`), the DMA unit (`0x2`), and the optional integer maths unit (`0x3`). See Chapter 16.

Displacement A signed value added to an address register's low word to compute an effective address, as used by memory and effective-address instructions. Distinct from *offset*, which this manual reserves for memory/structure offsets and the `system_ram_offset` register.

Exception The general term for any event that diverts program flow to a vector: a *fault*, a *hardware exception*, or a *software exception*. See Chapter 6.

Exception level The priority level of the exception currently being serviced (0 when not handling any exception). A new exception can only preempt the current one if its priority is strictly higher.

Fault A priority-7 exception raised by the CPU itself in response to an error condition (for example, a bus error or a privilege violation) or a debug event (instruction trace). Faults cannot be masked or deferred.

General-purpose register Any of `r1–r7`. The CPU does not enforce a usage convention among them.

Hardware exception An exception signaled by an external interrupt pin. Also informally called an *interrupt* at the pin level (interrupt pins, interrupt priority, interrupt enable) – the two terms are synonyms at different registers of formality, not competing definitions.

Meta-instruction An assembler-level mnemonic (for example **BRAN**, **NOOP**, **LJSR**) that assembles to a real CPU instruction's encoding rather than having an opcode of its own. See the relevant instruction-family chapter for each meta-instruction's lowering.

Protected mode The non-privileged CPU mode (`SR.P` set). Most privileged operations fault if attempted in protected mode; see Chapter 5.

Retryable fault A fault whose triggering operation does not take effect, so that returning with **RETE** safely retries it. See Chapter 6 for the full retryable set.

Software exception An exception raised deliberately by executing **EXCP**. Also called a *trap*.

Supervisor mode The privileged CPU mode (`SR.P` clear). All operations are available.

Word address SIRC-1 addresses memory in units of 16-bit words, not 8-bit bytes; incrementing an address register by one advances by one word. See Chapter 4 for the boundary between this architectural word addressing and any external byte-level representation.

Part I

Architecture Overview

CHAPTER 1

Introduction

1.1 Overview

The SIRC-1 is a 16-bit RISC processor ideal for general-purpose computing. It implements the SIRCIS (SIRC Instruction Set) instruction set architecture, which provides a powerful set of operations while still ensuring simplicity of CPU implementation.

1.2 Design Philosophy

1.2.1 RISC Principles

The SIRC-1 adheres to Reduced Instruction Set Computer (RISC) design principles:

- **Load/Store Architecture:** All ALU operations work exclusively on registers. Memory access is performed only through dedicated load and store instructions.
- **Fixed Instruction Length:** All instructions are exactly 32 bits (2 words) in length, simplifying instruction fetch and decode logic.
- **Simple Instruction Formats:** Only three instruction formats are supported: Immediate, Short Immediate with Shift, and Register.
- **Register-Rich:** Sixteen addressable registers provide ample workspace for computations without frequent memory access.
- **Consistent Execution Time:** Most instructions use exactly 6 execution phases and complete in 6 clock cycles when no bus wait states are required, eliminating the need for microcode and simplifying hardware implementation. DMA commands add transfer cycles after dispatch.

1.2.2 Design Goals

The SIRC-1 was designed with several key goals in order of importance:

Simplicity The CPU should be simple enough to be implemented in hardware without requiring complex microcode engines or multi-level instruction decoders.

Developer Ergonomics Powerful instruction encodings, including conditional execution and bit shifting, should reduce the number of instructions and branches a program must track.

Performance While simple, the design should allow for reasonable performance through efficient instruction encoding and the potential for future enhancements like pipelining.

Memory Protection Basic privilege levels and memory segmentation provide rudimentary protection for system software.

1.2.3 Applications

The SIRC-1 is designed for general-purpose computing, such as the core of a microcomputer. It should not be used in memory-constrained environments, as fixed-length instructions and the lack of byte addressing can result in larger program sizes.

1.3 Key Features

1.3.1 Architecture Characteristics

Feature	Specification
Data Width	16-bit
Addressing	Word Addressing
Address Bus	24-bit word address
Instruction Size	32-bit fixed
General-Purpose Registers	7 (r1–r7)
Address Register Pairs	4 (l, a, s, p – link, address, stack pointer, program counter)
Total Addressable Registers	16
Instruction Formats	3
Addressing Modes	7 (see Table 8.1)
Condition Codes	16

Table 1.1: SIRC-1 CPU Specifications

1.3.2 Execution Model

The SIRC-1 executes every instruction through six sequential execution phases (normally one phase per clock cycle). For a full description of each phase, see Chapter 2.

This fixed six-stage model means that ordinary processing-unit instructions take 6 clock cycles to complete when bus operations do not require wait states, regardless of their complexity. Every coprocessor call (**COP1**, **COPR**, and every meta-instruction that assembles to one) takes a second complete 6-cycle dispatch slot, for 12 cycles in total, before the next instruction is fetched. DMA commands add further cycles on top of that: the DMA unit holds instruction fetch after dispatch until its transfer has completed, so their total cost scales with the transfer size. A bus device may hold the current phase by delaying bus acknowledgement, adding wait cycles without adding execution phases. While having a fixed 6-cycle instruction execution model may seem wasteful for simple register operations that do not require memory access, it dramatically simplifies the hardware design by eliminating the need for variable-length execution sequences or microcode.

1.3.3 Coprocessors

The SIRC-1 is made up of multiple coprocessors (also known as modules) that all share the same address/data bus and register file. Only one coprocessor can be active at a time. Coprocessors are usually activated using the **COP1** or **COPR** coprocessor-call instructions (see Chapter 16), except for the "exception unit," which can also be activated when physical pins are asserted on the chip. All implementations of the SIRC-1 will have the "processing unit" as coprocessor `0x0`, the "exception unit" as coprocessor `0x1`, and the DMA unit as coprocessor `0x2`. Other coprocessors are optional and vary by SIRC-1 model. The instruction set supports up to 16 coprocessors. The coprocessor IDs are stable between models, so that programs are cross-compatible. For example, coprocessor `0x3` will always be the Integer Maths Unit. This means that if a CPU model is missing an optional coprocessor and a program requires it, the CPU raises an invalid-opcode fault so that the behavior can be emulated in software where practical.

ID	Name	Optional	Purpose
0x0	Processing Unit	No	Executes instructions
0x1	Exception Unit	No	Handles interrupts and exceptions
0x2	DMA Unit	No	Transfers blocks of data via the bus
0x3	Integer Maths Unit	Yes	Multiplies and divides integer values
0x4–0xF	Reserved/model use	Yes	Defined by future or model-specific manuals

Table 1.2: List of coprocessors

Each coprocessor can execute 16 opcodes: the first 8 (0x0–0x7) can be called in any mode, and the last 8 (0x8–0xF) can be called only in supervisor mode.

1.3.4 Privilege Levels

The SIRC-1 supports two privilege levels:

Supervisor Mode Full access to all registers and instructions. Can set the high address-register components, access privileged status register bits, and execute privileged coprocessor operations.

Protected Mode Restricted access for user programs. Cannot write to the high address-register components (providing memory segmentation/protection), cannot access privileged status register bits, and cannot execute privileged coprocessor instructions.

The privilege level is controlled by the protected mode bit in the status register.

1.4 Memory Organization

1.4.1 Address Space

The SIRC-1 uses word addressing, where each address refers to a 16-bit word. With a 24-bit address space, this allows access to $2^{24} = 16,777,216$ words, or 32 megabytes of external byte storage. Addresses are formed by combining the low 8 bits of the high address-register component with the full 16-bit low address-register component:

$$\text{Address} = (\text{RegHigh}[7 : 0] \ll 16) \mid \text{RegLow}[15 : 0] \quad (1.1)$$

The high 8 bits of the high register (bits 15–8) are not used for addressing but are available for storage, enabling techniques such as tagged pointers.

For normative byte order, multi-word storage, exception vector layout, immediate interpretation, and wraparound rules, see Chapter 4.

1.4.2 Memory Map

The CPU expects the exception vector table to begin at word address 0x000000. Each vector-table entry occupies two consecutive 16-bit words: the target address high word followed by the target address low word.

Region	Vector IDs	Word addresses	Purpose
Reset	0x00	0x000000–0x000001	Initial program counter after reset
Faults	0x01–0x0F	0x000002–0x00001F	Fault and reserved privileged exception vectors
Hardware	0x10, 0x20, 0x30, 0x40, 0x50	0x000020–0x0000A1	Hardware exception vectors, spaced by vector ID
User traps	0x60–0xFF	0x0000C0–0x0001FF	User software exception vectors

Table 1.3: Exception Vector Table Regions

Beyond these reserved regions, the memory layout is flexible and can be defined by system software.

1.4.3 Alignment

- All memory addresses refer to 16-bit words.
- There are no alignment restrictions for general memory I/O.
- All instructions span 2 words and must begin at an even word address.
- Unaligned instruction fetches (odd word address) trigger an alignment fault.

1.5 Document Organization

This manual is organized into three main parts:

Part I: Architecture Overview Covers the register model, status register, and overall CPU architecture.

Part II: Instruction Set Architecture Details instruction formats, addressing modes, shift operations, and condition codes.

Part III: SIRCIS Instruction Reference Provides detailed documentation for each instruction, including encoding, operation, and examples.

Appendices provide quick reference materials including complete opcode maps, timing diagrams, notes on undocumented instructions, and an alphabetical instruction index.

1.5.1 Scope

This manual covers the SIRC-1 hardware: the instruction set architecture, register model, bus interface, exception handling, and coprocessor interface. Software conventions — including register usage, calling conventions, stack frame layout, object file format, and linker output format — are outside the scope of this document. These topics are addressed in the SIRC-1 ABI and Toolchain documentation.

CHAPTER 2

CPU Architecture

2.1 Overview

The SIRC-1 CPU implements a combinatorial logic design with no microcode, utilizing six sequential execution phases to execute ordinary instructions in 6 clock cycles when no bus wait states are required. The architecture is composed of several key functional units that work together to fetch, decode, execute, and complete instructions.

2.2 Six-Stage Execution Phases

Most SIRC-1 instructions follow a six-phase execution model and complete in 6 clock cycles when no bus wait states are required. A phase that performs a bus operation may be held until the bus acknowledges it. DMA commands dispatch through the normal six-phase coprocessor-call path and then hold instruction fetch while the DMA unit completes its transfer. This fixed execution sequence greatly simplifies software development and hardware design.

Each address-register pair, including p , splits into a high half and a low half, written x_h and x_l (for example p_h and p_l); Chapter 3 describes this split in detail.

Phase 0: Instruction Fetch (High Word) – 1 cycle

- Fetch bits 31–16 from memory address in p
- p is unchanged

Phase 1: Instruction Fetch (Low Word) – 1 cycle

- Fetch bits 15–0 from memory address in $p + 1$
- Instruction is now fully available for decode

Phase 2: Decode and Register Fetch – 1 cycle

- Decode instruction opcode and operands
- Read source registers (optionally shifted)
- Evaluate condition codes
- Advance p_l by 2 to the address of the next instruction (p_h is unchanged)

Phase 3: Execute and Address Calculation – 1 cycle

- Perform ALU operation, and/or
- Calculate effective memory address

Phase 4: Memory Access – 1 cycle

- Read from memory (LOAD), or
- Write to memory (STOR), or
- No operation (register-only instructions)

Phase 5: Write Back – 1 cycle

- Write result to destination register
- Update status register flags if applicable
- Update address registers for post-increment/pre-decrement

2.3 Architectural Components

Figure 2.1 illustrates the major functional units within the SIRC-1 CPU and their interconnections.

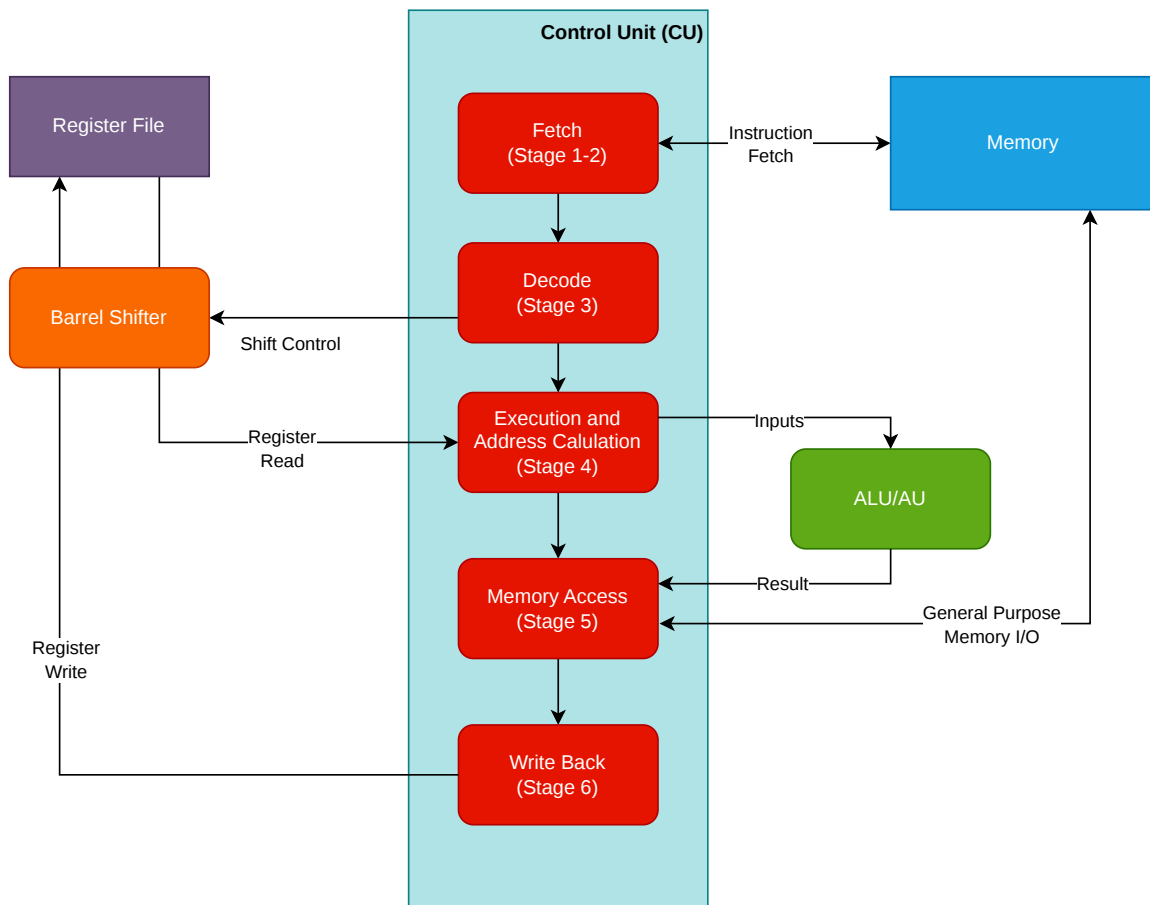


Figure 2.1: SIRC-1 CPU Execution Phase Architecture

2.3.1 Control Unit (CU)

The Control Unit sits at the heart of the SIRC-1 CPU, coordinating all other functional units throughout the instruction execution phases. It decodes instructions and generates control signals that orchestrate data movement and operations across the CPU.

Key responsibilities include:

- Instruction fetch and decode
- Execution phase coordination
- Control signal generation for all functional units
- Condition code evaluation for conditional execution

2.3.2 Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic and logical operations on register values. It supports the following operations:

- **Arithmetic:** Addition, Subtraction (with and without carry)

- **Logical:** AND, OR, XOR
- **Comparison:** Test and compare operations
- **Status Flag Generation:** Carry, Overflow, Negative, Zero

The ALU operates exclusively on data from the Register File (after optional shifting) and outputs results either back to the Register File or to memory via the data bus.

2.3.3 Addressing Unit (AU)

The Addressing Unit calculates memory addresses and manages address-related operations. It is responsible for:

- Computing effective addresses from base + displacement
- Handling pre-decrement and post-increment addressing modes
- Updating address register pairs (l, a, s, p) for LDEA, jump, and branch instructions
- Presenting calculated addresses to the address bus

The AU works in parallel with the ALU during the execution stage, allowing address calculation to occur simultaneously with ALU operations.

2.3.4 Register File

The Register File contains all CPU registers organized into three categories:

- **General Purpose Registers:** `r1-r7`
- **Address Register Pairs:** `l` (link), `a` (address), `s` (stack), `p` (program counter)
- **Status Register:** `sr`

The Register File provides multi-port access, allowing the ALU, AU, and Control Unit to read and write registers as needed during each execution phase. All registers are 16 bits wide, though address register pairs can be combined to form 24-bit addresses.

2.3.5 Barrel Shifters

A barrel shifter sits between the Register File and the CU, enabling efficient bit manipulation operations.

The barrel shifters support logical shift left (**LSL**), logical shift right (**LSR**), arithmetic shift left (**ASL**), arithmetic shift right (**ASR**), rotate left (**RTL**), and rotate right (**RTR**). Shift amounts can be specified as immediate values or dynamically from register contents.

The shift is applied to the first source operand when fetching the source registers for an instruction. Shifts cannot be applied to the result or any other source operand.

2.4 Design Characteristics

2.4.1 Combinatorial Logic Implementation

The SIRC-1 uses pure combinatorial logic with no microcode engine. Each execution phase is implemented as combinatorial circuits that process data in a single clock cycle. This approach:

- Simplifies the hardware design
- Provides a deterministic, fixed execution sequence
- Eliminates the complexity of microcode sequencing
- Makes the CPU behavior fully predictable and easy to reason about

2.4.2 Fixed Execution Sequence

Ordinary instructions execute in exactly 6 phases, regardless of instruction type or operands, and complete in 6 clock cycles when no bus wait states are required. Delayed bus acknowledgement can hold a phase and add wait cycles. Coprocessor calls are the exception: the processing-unit instruction takes six cycles and the coprocessor dispatch that follows takes a further six; DMA commands additionally hold instruction fetch until the transfer completes. This design choice simplifies software timing and eliminates the need for branch prediction or complex multi-stage control logic found in variable-time architectures.

2.4.3 Parallel Functional Units

The ALU and AU operate in parallel during phase 3 (Execute and Address Calculation), allowing address calculation to proceed simultaneously with arithmetic operations. This parallelism enables efficient execution of memory reference instructions without additional clock cycles.

2.5 External Interface

The SIRC-1 CPU exposes its bus, interrupt, and reset signals as physical pins; the chip package, the pin-by-pin descriptions, and the bus timing diagrams are given in Appendix F. One architectural fact the rest of this chapter relies on: on `RSTI` assertion the CPU aborts the current instruction or bus wait and restarts at phase 0, as described in the six-stage execution phases above.

CHAPTER 3

Register Model

3.1 Overview

The SIRC-1 CPU provides sixteen 16-bit registers, classified into three categories:

- General-purpose registers ($r1-r7$)
- Address register pairs (l, a, s, p)
- Status register (sr)

All programmer-visible registers are 16 bits wide. Address registers can be used in pairs to form 24-bit addresses for memory operations.

Only sr (cleared to zero) and the p pair (loaded from the reset vector) have defined values after reset; all other registers are architecturally undefined. See Chapter 6.

3.2 General-Purpose Registers

3.2.1 Register Set

The SIRC-1 provides seven general-purpose registers:

Register	Description
$r1$	General-purpose register 1
$r2$	General-purpose register 2
$r3$	General-purpose register 3
$r4$	General-purpose register 4
$r5$	General-purpose register 5
$r6$	General-purpose register 6
$r7$	General-purpose register 7

The programmer may use these registers for any purpose. They hold 16-bit values and may be used as source or destination operands for all ALU instructions.

3.2.2 Usage Conventions

The CPU does not enforce any register-usage convention among $r1-r7$; all seven registers are equally general purpose. Calling conventions and register-preservation rules are outside the scope of this document and are addressed in the SIRC-1 ABI and Toolchain documentation.

3.3 Address Register Pairs

3.3.1 Overview

The SIRC-1 provides four address register pairs, each consisting of a high and low 16-bit register. When used together, these pairs form 24-bit addresses by combining the low 8 bits of the high register with the full 16 bits of the low register.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Unused								High Register								Low Register															
bits 31–24								bits 23–16								bits 15–0															

Figure 3.1: Address Register Pair Formation

The high 8 bits (bits 31–24) of the high register are not used for addressing but remain available for storage, enabling techniques such as tagged pointers.

The exact address composition and stored-address layout are specified in Chapter 4.

Although each address register pair has a specific name and a suggested usage by convention, **STOR** and **LOAD** instructions treat all four pairs identically, and any pair may be used for indirect addressing.

Loading a value directly into the **p** pair performs a jump.

The link register pair updates automatically when **LDEL**, **LJSR**, or **BRSR** executes. The program counter pair increments after every instruction.

3.3.2 Link Register (l)

Component	Register
High Register	lh (privileged)
Low Register	ll

The link register pair stores return addresses for subroutine calls. When a subroutine is called using **LDEL**, **LJSR**, or **BRSR**, the return address is automatically saved to this register pair.

Typical Use:

- Storing return addresses from subroutine calls
- Alternate storage for general memory addressing

3.3.3 Address Register (a)

Component	Register
High Register	ah (privileged)
Low Register	al

This general-purpose address register pair is used for pointer operations and memory access.

Typical Use:

- Base pointer for data structures
- Array indexing
- General memory addressing

3.3.4 Stack Pointer (s)

Component	Register
High Register	sh (privileged)
Low Register	sl

The stack pointer register pair points to the current top of the stack. Pre-decrement and post-increment addressing modes are particularly useful with this register.

Typical Use:

- Stack operations (push/pop)
- Local variable allocation
- Function parameter passing

3.3.5 Program Counter (p)

Component	Register
High Register	<code>ph</code> (privileged)
Low Register	<code>pl</code>

The program counter register pair points to the instruction currently being fetched. It is not modified during either fetch phase; the low word `pl` advances by 2 during the Decode and Register Fetch phase, and `ph` is never incremented, so a wrap stays inside the segment.

Typical Use:

- Automatically updated during normal execution
- Modified by branch and jump instructions
- Used for position-independent code

3.4 Status Register

The status register (`sr`) is a special 16-bit register that contains CPU status flags and control bits. It is divided into two bytes:

- **Low Byte (bits 7–0):** Condition flags, readable in both modes and updated as a side effect of ALU and shift instructions in both modes; direct writes to `sr` are privileged
- **High Byte (bits 15–8):** Control and privileged state

In protected mode, reads of `sr` are redacted to the low byte (high byte reads as zero), and direct writes to `sr` raise a privilege violation fault. In supervisor mode, a direct write to `sr` is permitted, except that it cannot change the Exception Active (EA) bit.

The status register is discussed in detail in Chapter 5.

3.5 Privilege Restrictions

3.5.1 High Address Register Restrictions

The high component of each address register pair (`lh`, `ah`, `sh`, `ph`) is privileged:

- In **supervisor mode**: Reading and writing are both allowed.
- In **protected mode**:
 - Reading is allowed.
 - Direct writes trigger a privilege exception.
 - Address-register-pair write-back is allowed only when the high word would remain unchanged; otherwise it triggers a privilege exception.

This restriction provides a form of memory segmentation and protection. User programs running in protected mode cannot modify the segment portion of addresses, preventing them from accessing memory outside their designated segment. Instructions such as **LDEA**, **LDEL**, **BRAN**, **BRSR**, **LJMP**, **LJSR**, and **RETS** remain usable in protected mode when they stay within the current segment. If their address-register-pair write-back would alter a high word, the CPU raises a privilege violation fault.

The status register differs slightly from the high address registers:

- High address registers (**lh**, **ah**, **sh**, **ph**) remain readable in protected mode, but direct writes trigger a privilege violation fault.
- The status register's privileged byte is hidden on reads in protected mode, and direct writes in protected mode cannot modify it.

3.6 Register Addressing

Registers are addressed using 4-bit register identifiers in instruction encoding:

ID	Register	Privileged
0x0	r1	No
0x1	r2	No
0x2	r3	No
0x3	r4	No
0x4	r5	No
0x5	r6	No
0x6	r7	No
0x7	sr	Reads: high byte redacted. Writes: fully privileged
0x8	lh	Yes
0x9	ll	No
0xA	ah	Yes
0xB	al	No
0xC	sh	Yes
0xD	sl	No
0xE	ph	Yes
0xF	pl	No

Table 3.1: Register Encoding

Because writes to **sr** are fully privileged, any instruction whose destination field encodes **0x7** is privileged in protected mode, even when the operation would only alter the unprivileged low byte of **sr**. This register-index assignment is deliberate: **sr** does not occupy field **0x0** (unlike an earlier revision of this encoding), so an all-zero instruction word decodes as **ADDI [N] r1, #0** – a genuine, always-unprivileged no-op. This is what lets erased or uninitialized memory (which reads back as all-zero words) be executed safely instead of faulting; see the **NOOP** entry in Chapter 17.

3.7 Usage Examples

3.7.1 Basic Register Operations

Example 3-1:

```
1 ; Load immediate value into r1
2 LOAD r1, #100
3
4 ; Add 2 to the value stored in r1 and store the result into r2
5 LOAD r2, r1
6 ADDI r2, #2
7
8 ; Add r1 and r2, store in r3
9 ADDR r3, r1, r2
```

3.7.2 Address Register Usage

Example 3-2:

```
1 ; Load value from memory at address in 'a' register pair
2 LOAD r1, (a)
3
4 ; Store r1 to memory at address (a + 8)
5 STOR (#8, a), r1
6
7 ; Push r1 onto stack (pre-decrement)
8 STOR -(s), r1
9
10 ; Pop stack into r2 (post-increment)
11 LOAD r2, (s)+
```

3.7.3 Subroutine Call

Example 3-3:

```
1 ; Call subroutine at address in 'a' register
2 ; Return address saved to 'l' register automatically
3 LJSR a
4
5 ; ...subroutine code...
6
7 ; Return from subroutine
8 ; Copies 'l' register to 'p' register
9 RETS
```


CHAPTER 4

Data Representation and Memory Model

4.1 Overview

This chapter defines the representation rules used by registers, memory, program images, exception vectors, and multi-word values. These rules are architectural: assemblers, loaders, emulators, hardware implementations, and debuggers must agree on them.

4.2 Word and Byte Ordering

The SIRC-1 is a word-addressed processor. Each architectural memory address names one 16-bit word. Byte addressing is not provided by the CPU instruction set.

When a word is represented outside the CPU as bytes, such as in a ROM image or debugger dump, the word is stored in big-endian byte order: the most significant byte appears first.

Value	First byte	Second byte
0x1234	0x12	0x34
0xFF80	0xFF	0x80

Table 4.1: External Byte Order for 16-bit Words

Multi-word quantities are stored with the most significant word at the lowest word address. This applies to instructions, exception vectors, saved addresses, and any software-defined multi-word integer convention that chooses to use the architectural order.

4.3 Address Values

Architectural addresses are 24-bit word addresses. The four address-register pairs (l, a, s, and p) represent an address by combining the low byte of the high register with the full low register:

$$\text{Address} = ((\text{HighRegister} \& 0x00FF) \ll 16) | \text{LowRegister} \quad (4.1)$$

Bits 15–8 of the high register are not driven on the address bus and do not participate in address comparisons, effective-address calculation, instruction fetch, or exception vector dispatch. They are preserved as ordinary register storage unless an instruction explicitly writes the high register.

Pair value	High register	Low register	Address bus value
ah:a1	0x0012	0x3456	0x123456
ah:a1	0xAB12	0x3456	0x123456
ph:p1	0x0000	0x0100	0x000100

Table 4.2: Address Register Pair Interpretation

When a 24-bit address is stored in memory as a 32-bit quantity, the high word is stored first and the low word second. Only bits 7–0 of the high word are significant when the value is loaded as an address.

Stored address	Word at address N	Word at address N+1
0x123456	0x0012	0x3456
0xFECAFE	0x00FE	0xCAFE

Table 4.3: Stored 24-bit Address Format

4.4 Instruction Storage

Every instruction is 32 bits, stored as two consecutive 16-bit words:

- Word at p : instruction bits 31–16
- Word at $p+1$: instruction bits 15–0

Instruction fetch reads the high word from the address in p and the low word from that address plus one. p itself is not modified during the fetch; it advances by two words during instruction decode, so an instruction that reads or uses p as an address register sees the address of the instruction itself. Instruction addresses must be even word addresses. Fetching from an odd word address raises an alignment fault.

4.5 Exception Vector Storage

Each exception vector table entry is a stored 24-bit address in the format shown in Table 4.3. The vector table begins at word address 0x000000, so vector entry v starts at word address $v * 2$:

- Word at $v * 2$: target address high word
- Word at $v * 2 + 1$: target address low word

For example, vector 0x06 occupies word addresses 0x00000C and 0x00000D. To jump to 0x123456, those two words contain 0x0012 and 0x3456.

4.6 Immediate Values

Immediate fields are encoded as raw two's-complement bit patterns. The consuming instruction defines whether the field is interpreted as signed, unsigned, or bitwise data.

Use	Width	Interpretation
ALU arithmetic immediate	16 bits	Two's-complement operand; result is truncated to 16 bits
ALU arithmetic short immediate	8 bits	Zero-extended operand in the range 0x00–0xFF
ALU logical immediates	16 or 8 bits	Bit mask; no sign interpretation is applied by the logical operation
Memory displacements	16 bits	Two's-complement displacement added to the low address word; see Section 4.8 for the overflow trap
Coprocessor operands	16 bits	Coprocessor-defined command or parameter bits

Table 4.4: Immediate Field Interpretation

Short immediates occupy an 8-bit field and are zero-extended before use by ALU arithmetic. They are therefore suitable for compact non-negative constants and bit masks, not for encoding negative arithmetic operands. A value that cannot be represented in the selected immediate field width is truncated to that width; the reference assembler does not diagnose this truncation except for the short-immediate-with-shift form, which is range-checked.

Short immediates are not used for memory, effective-address, branch, jump, or subroutine-call displacements. Those forms use 16-bit immediate displacements, register displacements, or assembler-resolved full-width branch offsets.

4.7 Software Layout Conventions

The architectural storage order for multi-word quantities is high word first. The SIRC-1 reference manual does not otherwise define a complete ABI, stack-frame layout, argument-passing convention, or software integer library format. Where examples show structures, arrays, saved registers, or stack use, offsets are word offsets and each push or pop moves the stack pointer by one 16-bit word.

Software standards may build on the architectural high-word-first rule for wider integers and stored addresses, but those conventions are non-normative unless a separate ABI or toolchain document says otherwise.

4.8 Arithmetic and Address Wraparound

General-purpose register arithmetic is performed modulo 2^{16} . Arithmetic condition flags describe the truncated 16-bit result: C records unsigned carry or borrow, and V records signed overflow.

Address calculation for memory, effective-address, branch, jump, and auto-update operations adds displacements to the low word of the selected address register pair. The high address-register word is not changed by ordinary displacement, pre-decrement, or post-increment calculation. If the low word overflows or underflows and `SR.A` (Trap on Address Overflow)¹ is set, the instruction raises a segment-overflow fault. If the bit is clear, the low word wraps modulo 2^{16} . Displacements are signed two's-complement, and the trap fires only when the low-word calculation wraps, whether the displacement is positive or negative.

The program counter follows the same low-word wrap rule during normal instruction fetch. Since instructions are two words long and must begin at even word addresses, a fetch that would begin

¹`SR.X` denotes bit or field *X* of the status register; see Chapter 5 for the bit layout and Chapter 11 for the full notation convention.

from an odd `p1` value raises an alignment fault. If the program counter wraps the low word while `SR.A` is set, a segment-overflow fault is raised at the next instruction fetch, not on the instruction that caused the wrap.

4.9 Memory Ordering

The SIRC-1 has no architectural cache, store buffer, memory reordering, or speculative memory access. Bus reads and writes issued by the CPU are externally visible in program order. A bus device may delay completion by withholding bus acknowledgement, but this stalls the current execution phase rather than allowing later bus accesses to pass it.

CHAPTER 5

Status Register

5.1 Overview

The Status Register (`sr`) is a 16-bit special-purpose register that contains condition flags and control bits for the CPU. It is divided into two bytes with different privilege levels:

- **Low Byte (bits 7–0):** Condition flags – readable in both modes, and updated as a side effect of ALU and shift instructions in both modes; a direct write to `sr` is privileged and raises a privilege violation fault in protected mode
- **High Byte (bits 15–8):** Control and mode flags – accessible only in supervisor mode

5.2 Status Register Layout

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
T	A	EA	E4	E3	E2	E1	P					V	C	N	Z
Privileged								Non-Privileged							

Figure 5.1: Status Register Bit Layout

5.3 Condition Flags (Non-Privileged)

These flags reflect the results of ALU operations and comparisons. They can be read at any privilege level and are updated by ordinary instruction execution at any privilege level, but a direct write to `sr` is a privileged operation (see Section 5.5.2).

5.3.1 Zero Flag (Z) – Bit 0

Set when: The result of an operation is zero

Cleared when: The result of an operation is non-zero

Tested by: Equal (`==`), Not Equal (`!=`) condition codes

5.3.2 Negative Flag (N) – Bit 1

Set when: The result of an operation is negative (bit 15 = 1) when interpreted as a signed two's complement value

Cleared when: The result is positive or zero (bit 15 = 0)

Tested by: Negative Set (`NS`), Negative Clear (`NC`), and signed comparison condition codes

5.3.3 Carry Flag (C) – Bit 2

Set when: An addition produces a carry out of bit 15, or a subtraction requires a borrow

Cleared when: No carry/borrow occurs

Tested by: Carry Set (`CS`), Carry Clear (`CC`), Unsigned Higher (`HI`), Unsigned Lower or Same (`LO`) condition codes

Note: For unsigned comparisons, the carry flag acts as an unsigned overflow/borrow indicator

5.3.4 Overflow Flag (V) – Bit 3

Set when: A signed arithmetic operation produces a result that cannot be represented in 16 bits (signed overflow)

Cleared when: No signed overflow occurs

Tested by: Overflow Set (OS), Overflow Clear (OC), and signed comparison condition codes

Note: Overflow occurs when adding two positive numbers yields a negative result, or adding two negative numbers yields a positive result

5.3.5 Reserved Flags – Bits 4–7

Bits 4 through 7 of the lower byte are reserved for future use; software must not rely on them. These bits can be written by a supervisor-mode direct write to `sr`, but any subsequent ALU or shift flag update clears them to zero. They read as zero after any flag-updating instruction.

5.4 Control Flags (Privileged)

These flags control CPU behavior and can only be modified directly in supervisor mode. Attempts to write `sr` in protected mode raise a privilege violation fault.

5.4.1 Protected Mode (P) – Bit 8

When Clear (0): CPU operates in supervisor mode with full privileges

When Set (1): CPU operates in protected mode with restricted access to:

- High address-register components (`lh`, `ah`, `sh`, `ph`)
- High byte of status register
- Privileged coprocessor operations (command-word operation nibble `0x8–0xF`, bits 11–8)

Note: This bit provides basic memory protection by preventing user code from changing memory segments. Performing a restricted operation raises a privilege violation fault.

5.4.2 Hardware Interrupt Enable Bits (E1-E4) – Bits 9–12

These four bits individually control whether each of the four maskable hardware interrupt lines is enabled:

- **E1 (Bit 9):** Enable Hardware Interrupt Line 1 (Level 1, lowest maskable priority)
- **E2 (Bit 10):** Enable Hardware Interrupt Line 2 (Level 2)
- **E3 (Bit 11):** Enable Hardware Interrupt Line 3 (Level 3)
- **E4 (Bit 12):** Enable Hardware Interrupt Line 4 (Level 4, highest maskable priority)

Note: Hardware Interrupt Line 5 (Level 5) is a Non-Maskable Interrupt (NMI) and is always enabled. It cannot be disabled via the status register.

When a bit is set to 1, the corresponding hardware interrupt is enabled and can trigger an exception when asserted. When cleared to 0, interrupts on that line are disabled and are not serviced.

Important behavior:

- Disabled interrupts are not queued while the enable bit is 0.
- Hardware interrupt lines 1-4 are level-sensitive, not edge-triggered. If an interrupt pin remains asserted, it will be serviced again as soon as the current handler is finished.

- Hardware interrupts can only interrupt execution if their priority level is higher than the currently executing exception level
- This prevents the same interrupt from re-triggering while its handler is executing
- Lower priority interrupts that occur while handling a higher priority interrupt are queued and serviced after the current handler completes (after **RETE**)

These bits are typically managed by the operating system's interrupt handling code. At reset, all interrupt enable bits are cleared (interrupts disabled).

Interrupt handling is discussed in detail in Chapter 6.

5.4.3 Exception Active (EA) – Bit 13

When Set (1): The CPU is currently executing an exception handler (`current_exception_level != 0`)

When Clear (0): The CPU is executing normal code (`current_exception_level == 0`)

Automatically Set: When any exception is entered (software, hardware, or fault)

Automatically Cleared: When returning to exception level 0 via **RETE**

Purpose: Allows software to detect whether it is running within an exception handler context

Typical Use: OS code can check this bit to determine appropriate behavior, for example avoiding re-enabling interrupts during nested exception handling

Note: This bit reflects the internal `current_exception_level` state. Programs can read this bit but cannot directly modify it – it is managed automatically by the exception handling hardware.

5.4.4 Trap on Address Overflow (A) – Bit 14

When Set (1): An effective-address calculation or a program-counter advance that wraps the low word raises a segment overflow fault (vector `0x03`). For a program-counter wrap the fault is raised at the following instruction fetch

When Clear (0): Address overflows wrap around without any side effects

Purpose: Helps detect pointer errors during development

Typical Use: Enabled during debugging, disabled in production for performance

5.4.5 Trace Mode (T) – Bit 15

When Set (1): An exception is generated after every instruction

When Clear (0): Normal execution

Purpose: Allows debuggers to single-step through code

Note: Used by SIRCIT (SIRC Instruction Tracer) for debugging

5.5 Status Register Updates

5.5.1 Automatic Updates

The condition flags (Z, N, C, V) are automatically updated by most ALU instructions, unless explicitly disabled:

- **Arithmetic Instructions** (**ADDI**, **SUBI**, **ADCI**, **SBCI**): Update all four condition flags
- **Logical Instructions** (**ANDI**, **ORRI**, **XORI**): Update Z and N flags; C and V flags are cleared
- **Compare Instructions** (**CMPI**, **CMPR**): Same as Arithmetic Instructions but does not update destination
- **Test Instructions** (**TSAI**, **TSAR**, **TSXI**, **TSXR**): Same as Logical Instructions but does not update destination
- **Load Instructions** (**LOAD**, **LDEA**): **LOAD** never updates the condition flags – the assembler always assembles it with status source [N], giving the flag row – – – –. An explicit status update override written on **LOAD** leaves all four flags architecturally undefined (U U U U). **LDEA** never updates the condition flags
- **Shift Operations**: Can optionally update flags based on the instruction's status register update source field

5.5.2 Manual Updates

In supervisor mode, the status register can be directly manipulated like any other register:

```

1 ; Set protected mode bit (only in supervisor mode)
2 ORRI sr, #0x0100
3
4 ; Enable all maskable hardware interrupts (set bits 9-12)
5 ; Note: Hardware interrupt line 5 is an NMI and is always enabled
6 ORRI sr, #0x1E00
7
8 ; Disable all maskable hardware interrupts (clear bits 9-12)
9 ANDI sr, #0xE1FF
10
11 ; Restore status register saved to stack
12 LOAD sr, (s)+

```

An ALU instruction that targets `sr` writes its result to `sr` and then overwrites bits 7–0 with the flags produced by that same instruction, so only bits 15–8 can be set this way. To write the low byte, suppress the flag update with the [N] status source (for example **ORRI** [N] `sr`, #0x0003) or use **LOAD** `sr`, `rN`. A direct write to `sr` also preserves the current value of bit 13 (EA); only **RETE** restores it, so the **LOAD** `sr`, `(s)+` restore above does not reproduce the exception-active state saved at the time of the original write.

Important: When running in protected mode, any direct write to the status register raises a privilege violation fault. Protected-mode programs can still affect condition flags through ordinary ALU and shift instruction execution; they cannot directly load or store `sr`.

5.6 Reading the Status Register

In supervisor mode, reading `sr` returns the full 16-bit value. In protected mode, reading `sr` returns the lower byte in bits 7–0, with bits 15–8 masked to zero (for security).

```

1 ; In supervisor mode - gets full 16 bits
2 LOAD r1, sr ; r1 = 0xFFFF (all bits)
3
4 ; In protected mode - high byte masked
5 LOAD r1, sr ; r1 = 0x00XX (high byte forced to 0)

```

5.7 Exception Handling Effects

When an exception occurs:

1. The current status register is saved
2. Protected mode bit (P) is cleared (entering supervisor mode)
3. Trace mode bit (T) is cleared (disabling single-stepping in exception handler)
4. The hardware interrupt enable bits are not changed. A pending interrupt is serviced during a handler only if its priority level is higher than the current exception level

When returning from an exception with **RETE**, the saved status register is restored.

5.8 Usage Examples

5.8.1 Example 5-1: Conditional Execution

```

1 ; Compare r1 with r2
2 CMPR r1, r2
3
4 ; Branch if equal
5 BRAN |== @label_equal
6
7 ; Branch if r1 > r2 (unsigned)
8 BRAN |HI @label_greater
9
10 ; Add 0x16 to r1 if r1 >= r2 (signed)
11 ADDI |>= r1, #0x16

```

5.8.2 Example 5-2: Manual Status Flag Control

The SIRC-1 provides syntax to manually control how status flags are updated during ALU and shift operations. This is particularly useful for advanced control flow and flag preservation. See Section 9.12 in Chapter 9 for complete details.

```

1 ; Update flags from shift instead of ALU
2 ADDI[S] r2, #1, LSL #2 ; Flags from shift, not addition
3
4 ; Preserve flags across operations
5 CMPR r1, r2 ; Set comparison flags
6 ADDI[N] r3, #1 ; Increment without changing flags
7 BRAN |== @still_uses_cmpr_flags ; Branch uses CMPR flags

```

5.8.3 Example 5-3: Testing Bits

```
1 ; Test if bit 5 is set in r1
2 TSAI r1, #0x0020 ; AND with 0x0020, update flags only
3
4 ; Branch if bit was set (result non-zero)
5 BRAN != @bit_was_set
```

5.8.4 Example 5-4: Enabling Protected Mode

```
1 ; Entering protected mode (from supervisor mode)
2 ORRI sr, #0x0100 ; Set protected mode bit
3 ; Now running in protected mode
4
5 ; Attempting to write privileged register will fault
6 ; ADDI ah, #0x05 ; This would cause exception!
```

5.8.5 Example 5-5: Saving and Restoring Status

```
1 ; Save status register
2 LOAD r7, sr
3
4 ; Do some operations that change flags
5 ADDR r1, r2, r3
6 CMPI r1, #100
7
8 ; Restore original status
9 LOAD sr, r7
```

The restore above does not reproduce bit 13 (EA) as it stood at the time of the save: a direct write to `sr` always preserves the register's current EA value rather than adopting the written one, so only **RETE** can restore EA.

CHAPTER 6

Exception and Interrupt Handling

6.1 Overview

An exception is an event that causes the program flow to be diverted to an address stored in a predefined vector.

Some are directly caused by an instruction, some are a side effect of an instruction, and some are asynchronously caused by external hardware.

They are managed by the exception coprocessor, which takes control of the processor when an exception is pending.

6.1.1 Instructions Used in This Chapter

This chapter refers to several exception-coprocessor instructions before their full reference entries appear. Briefly:

EXCP Triggers a software exception (trap).

RETE Returns from an exception handler, restoring the saved program counter, status register, and exception level.

WAIT Halts the CPU until an enabled interrupt or reset occurs.

RSET Performs the reset sequence and jumps to the reset vector.

ETFR Copies the saved return address and/or status register from a link register into *a* and/or *r7*.

ETTR Copies *a* and/or *r7* into a link register's saved return address and/or status register.

EXCP, **RETE**, **WAIT**, **RSET**, **ETFR**, and **ETTR** are meta-instructions that assemble to **COPI** (coprocessor call) forms addressed to the exception coprocessor (coprocessor *0x1*); Section 6.7 in this chapter lists their opcodes, and Chapter 16 gives the full syntax, operation, flags, and legal forms for each.

6.2 Exception Types

6.2.1 Faults

Faults are priority 7 exceptions raised by the CPU itself in response to error conditions or debug events. They use vectors in the *0x01–0x0F* range. Unlike hardware and software exceptions, faults cannot be masked or deferred: once a fault condition is raised, the fault is dispatched at the next instruction boundary regardless of the current exception level. Two faults are conditional on whether they are raised at all: Segment Overflow requires *SR.A*, and Instruction Trace requires *SR.T* or the *TRCE* input.

The *T* bit (TraceMode) in the status register is always cleared when entering any exception handler, including fault handlers. This ensures that trace faults do not fire inside handlers. **RETE** restores the saved status register, so if *T* was set before the exception, tracing resumes after **RETE** returns.

Fault Categories

Not all faults behave the same way with respect to the faulting instruction and the RETE return address. There are three categories:

Retryable faults The operation that triggered the fault does not take effect. For most faults this means the instruction is cancelled before it commits; Alignment Fault is the exception, since the fetch itself is what fails, so there is no decoded instruction to cancel in the first place – but the effect is the same either way. The link register stores the address of the faulting instruction. RETE returns to that instruction so it can be retried, typically after the fault handler has corrected the condition that caused the fault: remapping or paging in a device for a bus fault, adjusting a permission for a bus protection fault, bringing an out-of-range address back into range for a segment overflow, or emulating the missing operation for an invalid opcode fault. This is also why Privilege Violation is retryable: the CPU guarantees that the instruction that triggered it never took effect, so a supervisor-mode handler can safely inspect it, emulate the privileged effect if appropriate, and either retry it or advance past it under its own control. Every fault in this category follows the same pattern: a handler is always free to advance past the faulting instruction with **ETFR/ETTR** instead of retrying it. For Invalid Opcode Fault this is usually the right choice – simply retrying an unemulated invalid opcode will raise the same fault again.

Alignment Fault; Bus Fault; Bus Protection Fault; Invalid Opcode Fault; Privilege Violation Fault; Segment Overflow Fault.

Post-instruction faults The instruction completes normally and its effects are committed before the fault fires. The link register stores the address of the *next* instruction. RETE advances to that instruction; the traced instruction is not re-executed.

Instruction Trace Fault.

Exception dispatch faults Raised during the exception handling machinery itself, not during normal instruction execution. The link register stores the program counter at the point the fault was detected within the exception handler. Recovery may require careful state management; these faults often indicate a hardware misconfiguration or a programming error in an exception handler.

Level Five Hardware Exception Conflict, Double Fault.

Fault Descriptions

- **Bus Fault (Retryable)** — Raised when an external device asserts the bus error CPU pin.

This could happen, for example, if an unmapped address is presented by the CPU and another chip detects this and raises an error. The instruction is cancelled before it takes effect, and the link register stores the address of the *faulting* instruction, so a handler implementing virtual memory can map in the missing page and return with **RETE** to retry the same instruction.

Because ALU results and status-register updates are only committed during the write-back phase (Chapter 2), which always runs after the memory-access phase where a bus fault is detected, a bus fault can never leave a partial status-register or register update behind.

- **Bus Protection Fault (Retryable)** — Raised when an external device asserts the bus protection error CPU pin.

This indicates that the requested address and device are valid, but the operation is disallowed. Examples include writing to ROM or accessing a protected memory region while the CPU is in protected mode. The instruction is cancelled before it takes effect, and the link register stores the address of the faulting instruction, so a handler can adjust the permission (if that is the correct response) and retry with **RETE**.

- **Alignment Fault (*Retryable*)** — Raised when fetching instructions if the program counter contains an odd word address.

This is to simplify fetching: the second word of an instruction is always at $p1 \mid 0 \times 1$, which ensures that instructions can never overflow a segment boundary. The saved return address is itself odd; a fault handler must correct p (for example, by clearing bit 0) before returning with **RETE**, or the CPU raises the same fault again immediately.

- **Segment Overflow Fault (*Retryable*)** — Raised when a computed address would fall outside the current segment.

Only raised if **SR.A** (Trap on Address Overflow) is set, since some programs intentionally rely on address wrap-around. Also raised if the program counter overflows; the captured bus address and bus access type in the fault metadata register help identify where the overflow occurred. The link register stores the address of the faulting instruction, whether the overflow occurred in the program counter or in any other address computation, so a handler can bring the address back into range and retry with **RETE**.

- **Invalid Opcode Fault (*Retryable*)** — Raised when a coprocessor call targets a non-existent coprocessor or requests an operation that the selected coprocessor does not implement.

Can be used for forward compatibility: programs written for a future coprocessor (for example, a floating-point unit) will trap on older hardware, and the handler can emulate the operation in software. Because this fault is retryable, the link register stores the address of the faulting instruction; a handler that has emulated the operation must use **ETFR/ETTR** to redirect the return address to the instruction following the trapped call before returning with **RETE** – retrying the instruction unmodified will simply raise the same fault again.

Note that this fault is specific to coprocessor instructions. Other instruction encodings outside the documented instruction set – i.e. normal processing-unit encodings – are reserved or architecturally undefined; they are not required to raise an invalid-opcode fault.

- **Privilege Violation Fault (*Retryable*)** — Raised when a privileged operation is attempted while not in supervisor mode:

1. Directly writing to the high word of any address register
2. Directly writing to the status register
3. Address-register-pair write-back that would change a high address-register word
4. Executing a supervisor-only coprocessor operation
5. Triggering a software exception with a vector below 0×60

The illegal instruction never takes effect: it is cancelled before it writes anything, and the link register stores its own address so **RETE** returns to it directly. This matters because a supervisor-mode handler needs to be able to inspect the exact instruction that faulted, decide how to respond (deny it, emulate its effect, or promote the caller), and then either retry it or use **ETFR/ETTR** to redirect the return address elsewhere. None of that is possible if the illegal instruction had already been allowed to run.

- **Instruction Trace Fault (*Post-instruction*)** — Raised after every instruction completes when the **TraceMode** SR bit is set, or when the **TRCE** input is asserted at the instruction boundary (see Appendix F, Force Trace Mode Input).

Used for single-step debugging. Because the fault fires after the instruction commits, **RETE** returns to the instruction *following* the traced one. The **T** bit is cleared on entry to the fault handler (as with all exceptions), so the handler itself is not traced. **RETE** restores the saved **SR**, which re-enables tracing unless the handler explicitly clears **T** from the saved status register via **ETTR** before returning. **SR.T** is sampled once at the start of each instruction,

before that instruction can modify it, so a write to `SR.T` takes effect starting with the next instruction boundary; exception-unit dispatches are never traced.

- **Level Five Hardware Exception Conflict** (*Exception dispatch*) — Raised when a level five hardware exception arrives while one is already being serviced.

The exception unit has no spare link register above level 6, so a second L5 cannot be handled normally. Rather than silently dropping it (which could mask a hardware misconfiguration), the CPU raises this fault. Level 5 interrupts should be treated as NMIs.

- **Double Fault** (*Exception dispatch*) — Raised when a fault occurs while another fault is already being handled.

When the CPU is already at fault level (priority 7) and another fault occurs, there is no additional banked link register available. The fault link register (register 6) is overwritten with the new return address, losing the original fault's return address. The fault metadata register (register 7) is also overwritten; it encodes a double-fault flag along with both the current and original fault types.

The CPU jumps to vector `0x08`. This is typically a last-chance handler that dumps registers and resets. If another fault occurs while the CPU is already at fault level, the CPU attempts to dispatch vector `0x08` again. There is no limit to the number of times vector `0x08` is dispatched. A double fault handler that causes a fault may cause an infinite loop without special handling from the programmer. If the double-fault vector fetch itself fails, the CPU will reset. If two fault conditions are detected within the same instruction before either has been dispatched, the resulting behavior is architecturally undefined.

6.2.2 Hardware Exceptions (Interrupts)

A hardware exception occurs when interrupt pins are signalled by external hardware. These are also referred to as "interrupts". When a hardware exception occurs, the current instruction is completed, and control is transferred to different vectors based on the interrupt "priority". The vectors for hardware exceptions are in the `0x10–0x50` range.

The return address stored in the exception link register is the address of the instruction that would have executed next, not the interrupted one – so once the handler returns via **RETE**, the interrupted program resumes normally as if nothing happened.

6.2.3 User Exceptions (Traps)

A user exception occurs when the program executes an **EXCP** instruction that signals the exception coprocessor that it should trigger an exception. They are usually used by programs running in protected mode to switch to supervisor mode in a controlled way. They are also referred to as "traps". The vectors for user exceptions are in the `0x60–0xFF` range.

As with hardware exceptions, the return address stored in the link register is the instruction following the **EXCP** call, so **RETE** resumes the calling program immediately after the trap.

Software exceptions are only accepted from normal execution. If an **EXCP** instruction is executed while any exception or fault handler is already active, the software exception is ignored, is not queued, and does not fetch its vector.

6.3 Exception Registers

The SIRC-1 exception coprocessor maintains several special-purpose registers for exception handling.

6.3.1 Link Registers

The exception unit uses banked link registers to store the processor state when an exception occurs. There are eight link registers in total, indexed 0–7:

- Register 0: Software exceptions (traps)
- Registers 1–5: Hardware exceptions (interrupts), one for each priority level
- Register 6: Faults
- Register 7: Fault metadata (stores bus address and bus access type information, see section 6.3.1 for the format)

Each link register stores three pieces of information:

- **Return Address:** The 24-bit return address, held in a 32-bit field whose top 8 bits are always zero. This is enforced by the same address masking applied whenever a full address is derived from an address register pair (Figure 3.1), so even if the source address register's high word carries data in its own top 8 bits, that data is discarded before the address ever reaches the link register.
- **Return Status Register:** The 16-bit status register value at the time of the exception
- **Saved Exception Level:** The exception level that was active when the exception was taken; **RETE** restores it into `current_exception_level`

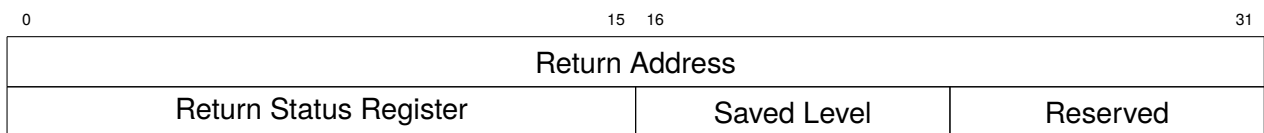


Figure 6.1: Exception Link Register Saved-State Layout

The Saved Level and Reserved fields are shown for illustration only: **ETFR** and **ETTR** transfer only the Return Address and the Return Status Register between a link register and `a/r7`. The saved exception level is not accessible to software through these instructions.

When an exception occurs, the appropriate link register (determined by the exception priority level) is populated with the current program counter and status register values. For faults in the retryable category (Alignment Fault, Bus Fault, Bus Protection Fault, Invalid Opcode Fault, Privilege Violation Fault, and Segment Overflow Fault), the return address is the address of the faulting instruction, so it can be retried. For regular exceptions (hardware and software exceptions) and Instruction Trace Fault, the return address is the address after the current instruction; see the "Fault Categories" section above for the exception-dispatch faults (Level Five Hardware Exception Conflict, Double Fault), which instead store the program counter at the point the fault was detected.

Register	Exception Source	Priority Level
0	Software exceptions	1
1	Level 1 hardware	2
2	Level 2 hardware	3
3	Level 3 hardware	4
4	Level 4 hardware	5
5	Level 5 hardware	6
6	Faults	7
7	Fault metadata	n/a

Table 6.1: Exception Link Register Assignment

Link Register Preservation

Important: Link registers can be overwritten when a fault occurs during the handling of another fault (a double fault). In this case, the fault link register (register 6) and the fault metadata register (register 7) are overwritten. This means the original return address is lost.

There is no way to fully recover from a double fault: the valid response is to declare the CPU state invalid, perform some last-chance error handling (for example, dumping the registers), and reset the CPU.

A fault handler that wants a better chance of capturing useful diagnostics before that reset should still save the link register contents early – before doing anything else that might fault again – using the **ETFR** instruction, so the original fault’s return address and status register can be included in whatever gets dumped.

ETFR only transfers the return address and status register of link registers 6 and 7; the saved exception level is not accessible to software through it, nor can **ETTR** write it back. This is not just a missing diagnostic field: **RETE** always restores `current_exception_level` directly from whatever value is currently latched in the link register, and after a double fault that value is the fault level (7), not the level that was active before the original fault. Even a handler that perfectly restores the return address and status register and attempts **RETE** anyway would leave the CPU stuck at exception level 7 – unable to service any hardware interrupt or software exception, including Level 5, since nothing outranks the highest priority level. New faults still dispatch normally regardless of the stuck level (faults bypass the priority check), so the CPU would keep running rather than visibly failing; it would simply be deaf to every interrupt and trap for the rest of the program’s life. This is the underlying reason reset, not **RETE**, is the only valid response to a double fault.

The values can be stored on the stack but since the stack pointer may be corrupted, a more robust implementation stores and loads from hardcoded memory addresses that are known to always be valid, instead of relying on the stack pointer.

Fault Metadata Register Format

The fault metadata register (register 7) stores additional information about faults beyond just the return address. The `return_status_register` field is used to encode fault-specific metadata in the following format:

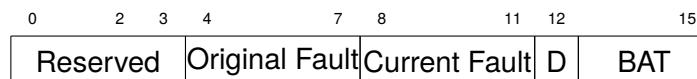


Figure 6.2: Fault Metadata Register Status Field Layout

- **Bits 0–2:** Bus access type (`BAT0–BAT2`) captured when the fault was raised (3 bits)
 - `0x0`: None
 - `0x1`: Instruction fetch
 - `0x2`: Data read
 - `0x3`: Data write
 - `0x4`: Exception vector fetch
 - `0x5`: DMA read burst
 - `0x6`: DMA write burst
 - `0x7`: Reserved
- **Bit 3:** Double fault flag (1 bit) — set to 1 if this is a double fault
- **Bits 4–7:** Current fault type (4 bits) — the type of fault that occurred

- **Bits 8–11:** Original fault type (4 bits) — only meaningful for double faults, indicates the fault type that was being handled when the double fault occurred
- **Bits 12–15:** Unused (reserved for future use)

Fault types use the following values:

- **0x01:** Bus fault
- **0x02:** Alignment fault
- **0x03:** Segment overflow fault
- **0x04:** Invalid opcode fault
- **0x05:** Privilege violation fault
- **0x06:** Instruction trace fault
- **0x07:** Level five hardware exception conflict
- **0x08:** Double fault
- **0x09:** Bus protection fault

The `return_address` field of register 7 stores the bus address that was being accessed when the fault occurred (for faults that involve memory access). For coprocessor invalid-opcode faults that occur during coprocessor dispatch before bus access, the low word of this field stores the 16-bit coprocessor command that could not be dispatched, and the high word is zero. This lets an invalid-opcode handler emulate missing optional coprocessor operations.

Exception handlers can use the **ETFR** instruction to read the fault metadata register and decode this information to determine how to handle the fault appropriately.

6.3.2 Cause Register

The cause register is a 16-bit internal register used to communicate between different CPU units and coordinate exception handling. It has the following structure:

- **Bits 15–12 (First Nibble):** Coprocessor ID (0x1 for exception unit, 0x0 for processing unit)
- **Bits 11–8 (Second Nibble):** Coprocessor opcode (the operation to execute)
- **Bits 7–0 (Third and Fourth Nibbles):** Vector or parameter value

The cause register is automatically populated by the exception unit when a pending exception needs to be serviced. The priority of exceptions is determined by examining the first nibble of the vector ID, calculated as 7 minus the value of the first nibble (for example, 0x00 is priority 7, 0x40 is priority 3, and 0x60 and above are all priority 1).

A cause value of 0x0000 — the value left in place at reset and whenever no exception is pending — selects coprocessor ID 0x0, the processing unit, so the CPU simply continues executing the program normally in the absence of any exception activity.

6.3.3 Hardware Interrupt Enable Bits

The status register contains 4 individual enable bits (bits 9–12 in the privileged byte) that control whether each maskable hardware interrupt line can trigger an exception:

- Bit 9 (E1): Hardware Interrupt Line 1 (Level 1, lowest maskable priority)
- Bit 10 (E2): Hardware Interrupt Line 2 (Level 2)
- Bit 11 (E3): Hardware Interrupt Line 3 (Level 3)
- Bit 12 (E4): Hardware Interrupt Line 4 (Level 4, highest maskable priority)

Note: Hardware Interrupt Line 5 (Level 5) is a Non-Maskable Interrupt (NMI) and cannot be disabled. It will always trigger an exception when asserted, regardless of the enable bit settings. Bit 13 is used for the Exception Active flag instead.

When an interrupt line is disabled (bit = 0), interrupts on that line are completely ignored and not queued. When enabled (bit = 1), the interrupt can trigger an exception if its priority is higher than the currently executing exception level.

6.3.4 Pending Hardware Interrupts

Hardware interrupt lines 1–4 are level-sensitive inputs; Level 5 is edge-triggered and is described separately under "Level 5 NMI-like Behavior" below. When an enabled level-sensitive interrupt line is observed asserted, the corresponding bit is set in the pending hardware-exceptions register. Disabled maskable interrupt lines are ignored at this sampling point and are not latched for later service.

For the level-sensitive lines, the pending register stores one bit per hardware interrupt line, so repeated assertions of the same line coalesce into a single pending interrupt. If the external pin remains asserted after the interrupt is serviced, it may be sampled again and become pending again.

At an instruction boundary, the exception unit selects the highest-priority pending hardware interrupt whose priority is higher than the current exception level. The selected bit is cleared as the handler is dispatched. Pending lower-priority interrupts remain latched and are considered again after the current handler returns with **RETE**.

Faults have priority over all hardware interrupts. If a fault is pending at the same instruction boundary as one or more hardware interrupts, the fault is dispatched first and the hardware interrupts remain pending. This includes instruction trace faults: when trace mode raises a post-instruction fault, that trace fault is serviced before pending hardware interrupts at the next boundary.

Hierarchical Exception Handling:

Exceptions in SIRC-1 are strictly hierarchical. A hardware interrupt can only interrupt the current execution if its priority level (exception level) is *higher* than the currently active exception level. This prevents:

- An interrupt handler from being interrupted by its own interrupt (same level)
- Lower priority interrupts from interrupting higher priority handlers
- Software exceptions from running during any hardware interrupt (would corrupt link register 0)

The exception unit maintains a `current_exception_level` field that tracks the active exception level (0–7). When not handling any exception, this is 0. When handling an exception at level *N*, only exceptions at level *> N* can interrupt.

Level 5 NMI-like Behavior:

Level 5 hardware exceptions behave similarly to a Non-Maskable Interrupt (NMI). Unlike interrupt levels 1–4, which are level-sensitive (sampled for as long as the pin is asserted), Level 5 is edge-triggered: it fires once on assertion regardless of how long the pin is held, and must be released

and reasserted to fire again. This avoids spurious Level Five Hardware Exception Conflict faults from a line that simply stays asserted. Level 5 cannot be disabled via the interrupt enable bits in the status register, and will preempt any lower-priority exception handler. However, Level 5 is *not* fully non-maskable. Because faults carry priority 7 (higher than Level 5's priority 6), a L5 interrupt that arrives while a fault is being handled is *queued* in the pending hardware exceptions register rather than serviced immediately.

Once the fault handler returns via `RETE` and the exception level drops back below 6, the pending L5 interrupt fires normally.

This means Level 5 response latency is not deterministic when a fault is in progress. For timing-critical use of the NMI line, fault handlers should be kept as short as possible.

If a second L5 interrupt arrives *while the CPU is already executing an L5 handler* (exception level 6), it is not queued. It will raise a Level Five Hardware Exception Conflict fault instead. See the fault descriptions above.

These two rules compose in a non-obvious way when a fault is raised *inside* an L5 handler while a second L5 interrupt is pending. Table 6.2 walks through the sequence step by step.

Step	Event	Exception Level	Link Register Used
1	CPU is executing inside the L5 handler.	6	Link register 5 (holds the L5 handler's return state)
2	A fault condition is raised inside the L5 handler. Priority 7 exceeds the current level (6), so the fault preempts it.	6 → 7	Link register 6 receives the L5 handler's return address, status register, and saved level (6)
3	A second L5 interrupt is asserted while the fault handler runs. Priority 6 does not exceed the current level (7), so it is not serviced.	7	Not dispatched; latched as pending in the pending hardware-exceptions register
4	The fault handler executes RETE , restoring the return address, status register, and saved level (6) from link register 6.	7 → 6	Link register 6 (restored, not overwritten)
5	Back at level 6, the exception unit reconsiders the pending L5 interrupt. Its priority (6) is not <i>higher</i> than the current level (6), so it is not dispatched normally.	6	—
6	The CPU raises a Level Five Hardware Exception Conflict fault instead, because the pending L5 collides with the still-active L5 handler.	6 → 7	Link register 6 receives the dispatch point as the return address

Table 6.2: Fault-Inside-L5-Handler Timeline

In short: the fault (priority 7) blocks the second L5 (priority 6) from firing while the fault handler runs. Once **RETE** returns to level 6, the queued L5 collides with the still-active outer L5 handler, and a Level Five Hardware Exception Conflict fault is raised then, not during the inner fault handler.

6.4 Exception Processing

6.4.1 Exception Priority

Exceptions are prioritized as follows (highest to lowest):

1. Priority 7: Faults

2. Priority 6: Level 5 hardware exceptions (NMI-like)
3. Priority 5: Level 4 hardware exceptions
4. Priority 4: Level 3 hardware exceptions
5. Priority 3: Level 2 hardware exceptions
6. Priority 2: Level 1 hardware exceptions
7. Priority 1: Software exceptions (traps)

6.4.2 Exception Handling Flow

When an exception occurs, the following steps are executed:

1. For hardware exceptions, the exception unit checks if the interrupt line is enabled in the status register. If disabled, the interrupt is ignored and not queued.
2. The exception unit checks if the exception priority level is higher than the current active exception level. If not, the exception is deferred until the current handler completes.
3. For level 5 hardware exceptions being raised when one is already being handled (both have exception level 6), a Level Five Hardware Exception Conflict fault is raised instead.
4. The vector-table address is computed by multiplying the vector ID by 2, since each vector is a 24-bit address stored as two 16-bit words.
5. The exception unit reads the high word and then the low word of the vector target address from the vector table.
6. The current program counter, status register, and current exception level are stored in the appropriate link register based on the exception priority level.
7. The Protected Mode bit in the status register is cleared, switching the CPU to supervisor mode.
8. The current exception level is updated to the level of the exception being serviced.
9. The program counter is set to the fetched vector target address, transferring control to the exception handler.

6.4.3 Exception Entry Side Effects

For pending hardware interrupts, priority and mask checks occur before the exception-vector fetch. A hardware interrupt that is disabled or not high enough priority to preempt the current handler does not fetch its vector; if it is already latched as pending, it remains pending for later service according to the pending-interrupt rules above.

Software exceptions are also screened before the exception-vector fetch. A software exception is accepted only when no exception handler is active.

After an exception source is selected for service, the exception unit fetches the high word and low word of the handler target address from the vector table. Architectural entry side effects commit only after the vector target has been fetched and the exception is accepted for service.

When entry commits, the exception unit performs the following state updates as one architectural action:

- Writes the selected link register with the current program counter, current status register, and current exception level.

- Clears `SR.P`, entering supervisor mode.
- Clears `SR.T`, preventing trace faults from firing inside the handler.
- Sets `SR.EA`, indicating that an exception handler is active.
- Preserves the condition flags, reserved lower status bits, interrupt-enable bits, and `SR.A` in the live status register.
- Updates the current exception level to the level of the exception being serviced.
- Loads the program counter from the fetched vector target address.

The saved status register in the link register is the pre-entry value, before `SR.P`, `SR.T`, or `SR.EA` are modified. **RETE** restores that saved status register, so a handler can inspect or edit the saved value with **ETFR** and **ETTR** before returning.

If an exception is not accepted because its priority is not higher than the current exception level, none of these entry side effects occur. Pending hardware interrupts remain subject to the priority and pending-interrupt rules described above.

If a bus or protection fault occurs during an exception-vector fetch before entry commits, the original exception does not enter. The CPU raises the corresponding fault with `BAT0–BAT2` set to `ExceptionVectorFetch`. If a fault-vector fetch itself fails, the CPU escalates to double fault. If the double-fault vector fetch fails, the CPU will reset.

Returning from Exceptions: see Section 6.6 for how **RETE** restores this state.

6.4.4 Vector Table

The exception vectors are stored starting at address `0x0`. Each vector is a 24-bit target address stored as two consecutive 16-bit words in the address format described in Section 4.5. The high word is stored first at `vector * 2`, followed by the low word at `vector * 2 + 1`. The vector table has 256 entries. Vectors `0x00–0x5F` (96 entries) are reserved for the CPU; vectors `0x60–0xFF` (160 entries) are user trap vectors. The vector table layout is:

- **0x00:** Reset vector
- **0x01:** Bus fault
- **0x02:** Alignment fault
- **0x03:** Segment overflow fault
- **0x04:** Invalid opcode fault
- **0x05:** Privilege violation fault
- **0x06:** Instruction trace fault
- **0x07:** Level five hardware exception conflict
- **0x08:** Double fault
- **0x09:** Bus protection fault
- **0x0A–0x0F:** Reserved for future privileged exceptions
- **0x10:** Level 5 hardware exception vector
- **0x20:** Level 4 hardware exception vector
- **0x30:** Level 3 hardware exception vector

- **0x40:** Level 2 hardware exception vector
- **0x50:** Level 1 hardware exception vector
- **0x11–0x5F (excluding 0x20, 0x30, 0x40, 0x50):** Reserved
- **0x60–0xFF:** User exception vectors (160 user-accessible trap vectors)

6.4.5 Exception Quick Reference

Vector	Address	Priority	Source	Maskability	Return Address	Metadata	P Mode
0x00	0x0000–0x0001	n/a	Reset	Not maskable	n/a	None	Yes
0x01	0x0002–0x0003	7	Bus fault	Not maskable	Next instruction	Fault metadata	Yes
0x02	0x0004–0x0005	7	Alignment fault	Not maskable	Faulting instruction	Fault metadata	Yes
0x03	0x0006–0x0007	7	Segment overflow	SR.A gated	Faulting instruction	Fault metadata	Yes
0x04	0x0008–0x0009	7	Invalid opcode	Not maskable	Faulting instruction	Fault metadata	Yes
0x05	0x000A–0x000B	7	Privilege fault	Not maskable	Next instruction	Fault metadata	Yes
0x06	0x000C–0x000D	7	Trace fault	SR.T or TRCE gated	Next instruction	Fault metadata	Yes
0x07	0x000E–0x000F	7	L5 conflict	Not maskable	Dispatch point	Fault metadata	Yes
0x08	0x0010–0x0011	7	Double fault	Not maskable	Dispatch point	Fault metadata	Yes
0x09	0x0012–0x0013	7	Bus protection	Not maskable	Next instruction	Fault metadata	Yes
0x0A–0x0F	0x0014–0x001F	7	Reserved faults	Reserved	Reserved	Reserved	Reserved
0x10	0x0020–0x0021	6	Level 5 hardware	Not maskable	Next instruction	Link register 5	Yes
0x20	0x0040–0x0041	5	Level 4 hardware	SR.E4	Next instruction	Link register 4	Yes
0x30	0x0060–0x0061	4	Level 3 hardware	SR.E3	Next instruction	Link register 3	Yes
0x40	0x0080–0x0081	3	Level 2 hardware	SR.E2	Next instruction	Link register 2	Yes
0x50	0x00A0–0x00A1	2	Level 1 hardware	SR.E1	Next instruction	Link register 1	Yes
0x11–0x5F	0x0022–0x00BF	n/a	Reserved	Reserved	Reserved	Reserved	Reserved
0x60–0xFF	0x00C0–0x01FF	1	Software trap	Priority gated	Next instruction	Link register 0	Yes

Table 6.3: Exception Quick Reference

6.5 Reset and Startup

Reset can be initiated externally by asserting the `RSTI` pin, or in software by executing the privileged `RSET` meta-instruction. Both paths converge on the same reset-vector sequence: the CPU holds reset output active, then the exception unit fetches vector `0x00` and loads the fetched target address into the program counter.

6.5.1 Reset State

State	Reset Behavior
Instruction phase	Set to phase 0.
Pending bus request	Cleared; an in-progress stalled bus request is abandoned.
Halt state	Cleared.
Wait-for-exception state	Cleared.
Current exception level	Cleared to 0. The CPU leaves reset outside any exception handler.
Status register	Cleared to <code>0x0000</code> when the reset operation is executed by the exception unit.
Program counter	Loaded from reset vector <code>0x00</code> .
General registers	Not architecturally cleared by reset. Software should treat their contents as undefined after reset.
Address register pairs	Not architecturally cleared by reset.
Exception link registers	Not architecturally cleared by reset.
Pending hardware exceptions and faults	Cleared. External interrupt pins that remain asserted after reset may be sampled again according to the normal interrupt rules.
Pending coprocessor command	Set internally to the reset operation until the reset vector sequence completes.

Table 6.4: Reset State

Because the status register is cleared to `0x0000` as part of the reset sequence, hardware interrupt lines 1–4 are masked immediately after reset – their enable bits, `SR.E1`–`SR.E4` (Section 6.3.3), are all cleared to 0. The Level 5 (NMI) line is architecturally non-maskable and remains enabled regardless of `sr`'s contents. Note also that the status register is not cleared the instant reset is requested: it is cleared during the reset-vector fetch sequence (Section 6.5.3), once that sequence reaches its execute phase.

6.5.2 Reset Output Hold

While reset is active, the CPU asserts `RSTO` and performs no bus cycles. This gives external devices sharing the reset line time to complete their own reset sequence before the CPU resumes.

For hardware reset, asserting `RSTI` immediately aborts any current instruction, exception handler, exception dispatch, or pending bus wait, then asserts `RSTO`. If `RSTI` remains asserted, `RSTO` remains asserted and the CPU does not advance. After `RSTI` is released, the reset hold completes before the CPU resumes. The reset assertion cycle counts as the first reset-output cycle; after release, five additional countdown cycles occur before the exception unit starts the reset-vector fetch.

For software reset, `RSET` completes as a normal coprocessor meta-instruction. When its write-back phase commits the reset command, the CPU requests reset and `RSTO` is asserted for six cycles before the reset-vector fetch begins.

6.5.3 Reset Vector Fetch

The reset vector is vector `0x00` in the exception vector table. After the reset-output hold, the exception unit performs an exception-vector fetch with bus access type `ExceptionVectorFetch`:

1. Read the high word of the reset target address from `0x0000`.
2. Read the low word of the reset target address from `0x0001`.
3. Clear the status register to `0x0000`.
4. Load the fetched target address into `p`.

These four steps are not one clock cycle each. Steps 1 and 2 each occupy their own fetch cycle, an instruction-decode cycle follows between step 2 and step 3 (not itemized above), and steps 3 and 4 then happen together in a single execute cycle. The reset-vector fetch therefore takes four clock cycles in total, separate from and following the reset-output hold described in Section 6.5.2.

Step 3's status-register clear is listed here, rather than only in the Reset State table (Table 6.4), because it marks the specific point in the pipeline where the clear actually happens – useful for anyone modeling cycle-accurate timing. Table 6.4 instead documents the eventual, settled reset state; no other register is touched during the reset-vector fetch itself, so it is the only state change covered both here and there.

If a bus or protection fault occurs during the reset-vector fetch, the reset target is not loaded. The CPU raises the corresponding fault with `BAT0–BAT2` set to `ExceptionVectorFetch`. From that point the normal exception-vector fetch fault rules apply: a fault-vector fetch failure escalates to double fault, and a double-fault vector fetch failure will reset the CPU.

6.6 Return from Exception

To return from an exception handler, the **RETE** (Return from Exception) instruction restores the saved state from the current link register:

1. The return address is loaded from the link register into the program counter
2. The saved status register value is restored (including interrupt enable bits)
3. The saved exception level is restored to `current_exception_level`
4. If the restored exception level is 0, `SR.EA` is cleared; otherwise the restored value of `SR.EA` stands
5. Execution resumes at the return address

After **RETE**, any pending interrupts that are now eligible to run (higher priority than the restored exception level and enabled) will be serviced before the next instruction executes.

Executing **RETE** while the current exception level is 0 is architecturally undefined.

For faults in the retryable category (Alignment Fault, Bus Fault, Bus Protection Fault, Invalid Opcode Fault, Privilege Violation Fault, and Segment Overflow Fault), **RETE** returns to the faulting instruction, allowing it to be retried. For all other faults and for regular exceptions, **RETE** returns to the instruction following the one that was executing when the exception occurred.

6.6.1 Accessing Link Registers

The exception coprocessor provides instructions to transfer data to and from the link registers:

- **ETFR (Exception Transfer From Register)**: Copies the return address and/or status register from a link register to the address register and/or `r7`
- **ETTR (Exception Transfer To Register)**: Copies the address register and/or `r7` to a link register's return address and/or status register

These instructions allow exception handlers to examine or modify the saved processor state before returning, enabling features like system calls, context switching, and debuggers.

6.7 Exception Coprocessor Instructions

The exception coprocessor (coprocessor ID `0x1`) provides several specialized instructions for exception handling and system control.

Instruction	Opcode	Privilege	Purpose
EXCP	<code>0x1</code>	User	Trigger a software exception (trap)
WAIT	<code>0x9</code>	Supervisor	Enter wait state until an enabled interrupt or reset occurs
RETE	<code>0xA</code>	Supervisor	Return from an exception handler
RSET	<code>0xB</code>	Supervisor	Perform reset sequence and jump to reset vector
ETFR	<code>0xC</code>	Supervisor	Copy saved exception state from a link register
ETTR	<code>0xD</code>	Supervisor	Write CPU register state back to a link register

Table 6.5: Exception Coprocessor Instruction Summary

Exception coprocessor opcodes `0x2–0x8` are reserved; issuing one is architecturally undefined. For full syntax, operation details, and worked examples, see Chapter 16.

6.7.1 Internal Instructions

The following opcodes are used internally by the exception unit and are not directly accessible via user instructions:

- **Fault (`0xE`)**: Automatically invoked when a fault condition is detected
- **HardwareException (`0xF`)**: Automatically invoked when a hardware interrupt is signaled
- **None (`0x0`)**: No operation, used when no exception is pending

These internal operations follow the standard exception handling flow described in the Exception Processing section. Issuing opcode `0xE` or `0xF` directly, for example via a supervisor-mode **COP1** instruction, is architecturally undefined.

Part II

Instruction Set Architecture

CHAPTER 7

Instruction Formats

7.1 Overview

All SIRCIS instructions are exactly 32 bits (4 bytes, or 2 words) in length. This fixed instruction size simplifies instruction fetch and decode logic, though it may result in larger code size compared to variable-length instruction sets.

The SIRC-1 supports three distinct instruction formats:

1. Immediate
2. Short Immediate (with Shift)
3. Register

Note: Operandless assembler meta-instructions such as **NOOP** still assemble to one of these three CPU instruction formats.

Each format is optimized for different types of operations and addressing modes.

7.2 Format Overview

Format	Typical Use
Immediate	Operations with a 16-bit constant value
Short Immediate	Operations with an 8-bit constant with optional shift
Register	Operations on multiple registers with optional shift

Table 7.1: Instruction Format Summary

7.3 Common Fields

All instruction formats share some common fields:

Opcode (6 bits) Identifies the operation to perform. With 6 bits, up to 64 distinct operations can be encoded.

Condition Flags (4 bits) Specifies the conditions under which the instruction executes. See **Chapter 10** for details.

Additional Flags (2 bits) Used in memory operations to specify address register pairs (l, a, s, p) and in ALU operations to specify how the status register is updated.

7.4 Immediate Format

7.4.1 Structure

The immediate format encodes a full 16-bit immediate value within the instruction.

Note: Due to the large 16-bit operand, shift is not supported in this format.

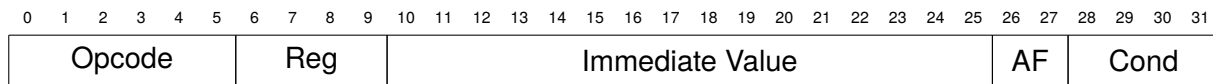


Figure 7.1: Immediate Instruction Format

Opcode (bits 31–26) 6-bit operation code

Register (bits 25–22) 4-bit destination/source register identifier. For **LDEA** and **LDEL** (opcodes 0x18–0x1F), this field instead holds a 2-bit address-register-pair index in bits 23–22 (00 = l, 01 = a, 10 = s, 11 = p) naming the destination pair, using the same encoding as the AF field; bits 25–24 are unused for these opcodes.

Immediate (bits 21–6) 16-bit signed or unsigned immediate value

Additional Flags (bits 5–4) 2-bit address register pair selector (for memory operations)

- 00 = l (link register)
- 01 = a (address register)
- 10 = s (stack pointer)
- 11 = p (program counter)

or 2-bit status register update source selector (for ALU operations)

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

Condition (bits 3–0) 4-bit condition code

7.4.2 Encoding Examples

Instruction	Opcode	Reg	Immediate	AF	Cond	Hex
ADDI r1, #100	0x00 (000000)	0x1 (0001)	0x0064 (0000 0000 0110 0100)	0b01 (01)	0x0 (0000)	0x00401910
CMPI r2, #0x8000	0x0A (001010)	0x2 (0010)	0x8000 (1000 0000 0000 0000)	0b01 (01)	0x0 (0000)	0x28A00010
LOAD r3, (#16, a)	0x14 (010100)	0x3 (0011)	0x0010 (0000 0000 0001 0000)	0b01 (01)	0x0 (0000)	0x50C00410
STOR (#-2, s), r4	0x10 (010000)	0x4 (0100)	0xFFFE (1111 1111 1111 1110)	0b10 (10)	0x0 (0000)	0x413FFFA0
LDEA a, (#4, s)	0x18 (011000)	0x1 (0001)	0x0004 (0000 0000 0000 0100)	0b10 (10)	0x0 (0000)	0x60400120
LDEA a, -(#0, s)	0x1A (011010)	0x1 (0001)	0x0000 (0000 0000 0000 0000)	0b10 (10)	0x0 (0000)	0x68400020
LDEL p, (#0, a)	0x1C (011100)	0x3 (0011)	0x0000 (0000 0000 0000 0000)	0b01 (01)	0x0 (0000)	0x70C00010
LDEL p, (#0, a)+	0x1E (011110)	0x3 (0011)	0x0000 (0000 0000 0000 0000)	0b01 (01)	0x0 (0000)	0x78C00010
BRAN #4	0x18 (011000)	0x3 (0011)	0x0004 (0000 0000 0000 0100)	0b11 (11)	0x0 (0000)	0x60C00130
BRSR #4	0x1C (011100)	0x3 (0011)	0x0004 (0000 0000 0000 0100)	0b11 (11)	0x0 (0000)	0x70C00130
ADDI == r5, #42	0x00 (000000)	0x5 (0101)	0x002A (0000 0000 0010 1010)	0b01 (01)	0x1 (0001)	0x01400A91
SUBI >= r6, #10	0x02 (000010)	0x6 (0110)	0x000A (0000 0000 0000 1010)	0b01 (01)	0xB (1011)	0x0980029B

Table 7.2: Immediate Format Encoding Examples

Notes:

- Condition code examples: 0x0 (0000) = Always, 0x1 (0001) = Equal (==), 0xB (1011) = Signed >=
- AF field for memory ops: 00=l, 01=a, 10=s, 11=p; for ALU ops: 00=no update, 01=update from ALU, 10=update from shift
- Negative immediates use two's complement (#-2 = 0xFFFE)

7.5 Short Immediate Format (with Shift)

7.5.1 Structure

The short immediate format sacrifices immediate value width to include shift information, allowing for more powerful single-instruction operations.

This format is supported only for ALU instructions; it cannot be used with branching or load/store instructions.

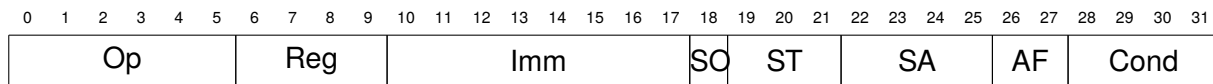


Figure 7.2: Short Immediate with Shift Format

Opcode (bits 31–26) 6-bit operation code

Register (bits 25–22) 4-bit destination/source register identifier

Immediate (bits 21–14) 8-bit unsigned immediate value, zero-extended to 16 bits before use

Shift Operand (bit 13) Determines how the shift amount field is interpreted

- 0 = Shift amount field contains the literal shift amount
- 1 = Shift amount field refers to register that contains shift amount

Shift Type (bits 12–10) Type of shift operation (see Table 7.6 for encoding details)

Shift Amount (bits 9–6) Shift amount (or register containing shift amount)

Additional Flags (bits 5–4) 2-bit status register update source selector

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

Condition (bits 3–0) 4-bit condition code

7.5.2 Encoding Examples

Instruction	Op	Reg	Imm	SO	ST	SA	AF	Cond	Hex
ADDI r1, #2, LSL #3	0x20 (100000)	0x1 (0001)	0x02 (00000010)	0 (0)	001 (001)	0x3 (0011)	0b01 (01)	0x0 (0000)	0x804084D0
ORRI r2, #1, LSL #10	0x25 (100101)	0x2 (0010)	0x01 (00000001)	0 (0)	001 (001)	0xA (1010)	0b01 (01)	0x0 (0000)	0x94804690
ANDI r3, #7, ASR #2	0x24 (100100)	0x3 (0011)	0x07 (00000111)	0 (0)	100 (100)	0x2 (0010)	0b01 (01)	0x0 (0000)	0x90C1D090
SUBI r4, #5, LSL r2	0x22 (100010)	0x4 (0100)	0x05 (00000101)	1 (1)	001 (001)	0x2 (0010)	0b01 (01)	0x0 (0000)	0x89016490
XORI != r7, #15, LSR #4	0x26 (100110)	0x7 (0111)	0x0F (00001111)	0 (0)	010 (010)	0x4 (0100)	0b01 (01)	0x2 (0010)	0x99C3C912
CMPI NS r1, #8, RTL r3	0x2A (101010)	0x1 (0001)	0x08 (00001000)	1 (1)	101 (101)	0x3 (0011)	0b01 (01)	0x5 (0101)	0xA84234D5
SHFT r3, RTR #5	0x25 (100101)	0x3 (0011)	0x00 (00000000)	0 (0)	110 (110)	0x5 (0101)	0b10 (10)	0x0 (0000)	0x94C01960

Table 7.3: Short Immediate Format Encoding Examples

Notes:

- SO (Shift Operand): 0 = literal shift amount, 1 = register contains shift amount (see rows 4 and 6)
- ST (Shift Type): 000=none, 001=LSL, 010=LSR, 011=ASL, 100=ASR, 101=RTL, 110=RTR
- AF field for ALU: 00=no update, 01=update from ALU, 10=update from shift (see SHFT row 7)
- Condition code examples: 0x0 = Always, 0x2 = Not Equal (!=), 0x5 = Negative (<0)
- SHFT is a meta-instruction using ORRI opcode 0x25 with AF=10

7.6 Register Format

7.6.1 Structure

The register format allows operations on up to three registers with optional shift.

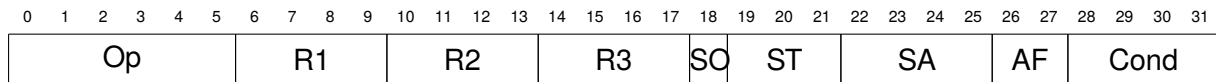


Figure 7.3: Register Instruction Format

Opcode (bits 31–26) 6-bit operation code

Register 1 (bits 25–22) 4-bit destination register. For **LDEA** and **LDEL** (opcodes 0×18 – $0\times 1F$), this field instead holds a 2-bit address-register-pair index in bits 23–22 (00 = l, 01 = a, 10 = s, 11 = p) naming the destination pair, using the same encoding as the AF field; bits 25–24 are unused for these opcodes.

Register 2 (bits 21–18) 4-bit source A register

Register 3 (bits 17–14) 4-bit source B register

Shift Operand (bit 13) Determines how the shift amount field is interpreted

- 0 = Shift amount field contains the literal shift amount
- 1 = Shift amount field refers to register that contains shift amount

Shift Type (bits 12–10) Type of shift operation (see Table 7.6 for encoding details)

Shift Amount (bits 9–6) Shift amount (or register containing shift amount)

Additional Flags (bits 5–4) 2-bit address register pair selector (for memory operations)

- 00 = l (link register)
- 01 = a (address register)
- 10 = s (stack pointer)
- 11 = p (program counter)

or 2-bit status register update source selector (for ALU operations)

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

Condition (bits 3–0) 4-bit condition code

7.6.2 Register Operand Semantics

For most three-operand instructions:

- R1 = Destination
- R2 = First source operand
- R3 = Second source operand (optional)

Note: If only two register operands are given, the assembler will use R1 as both destination and first source operand when encoding the instruction:

```
1 ADDR r1, r2, r3 ; r1 = r2 + r3
2 ADDR r1, r2 ; r1 = r1 + r2 (R2 implicitly = R1)
```

7.6.3 Encoding Examples

Instruction	Op	R1	R2	R3	SO	ST	SA	AF	Cond	Hex
ADDR r1, r2, r3	0x30 (110000)	0x1 (0001)	0x2 (0010)	0x3 (0011)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0xC048C010
ADDR r1, r2, r3, LSL #2	0x30 (110000)	0x1 (0001)	0x2 (0010)	0x3 (0011)	0 (0)	001 (001)	0x2 (0010)	0b01 (01)	0x0 (0000)	0xC048C490
LOAD r1, (r2, a)	0x15 (010101)	0x1 (0001)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x54408010
STOR -(r2, s), r1	0x13 (010011)	0x0 (0000)	0x1 (0001)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b10 (10)	0x0 (0000)	0x4C048020
LDEA a, (r2, s)	0x19 (011001)	0x1 (0001)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b10 (10)	0x0 (0000)	0x64408020
LDEA a, -(r2, s)	0x1B (011011)	0x1 (0001)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b10 (10)	0x0 (0000)	0x6C408020
LDEL p, (r2, a)	0x1D (011101)	0x3 (0011)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x74C08010
LDEL p, (r2, a)+	0x1F (011111)	0x3 (0011)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x7CC08010
LJMP a, r2	0x19 (011001)	0x3 (0011)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x64C08010
LJSR a, r2	0x1D (011101)	0x3 (0011)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x74C08010
CMPR r4, r5	0x3A (111010)	0x4 (0100)	0x4 (0100)	0x5 (0101)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0xE9114010
XORR r2, r3, r4, RTR #1	0x36 (110110)	0x2 (0010)	0x3 (0011)	0x4 (0100)	0 (0)	110 (110)	0x1 (0001)	0b01 (01)	0x0 (0000)	0xD88D1850
SUBR » r1, r2, r3, ASR r4	0x32 (110010)	0x1 (0001)	0x2 (0010)	0x3 (0011)	1 (1)	100 (100)	0x4 (0100)	0b01 (01)	0xD (1101)	0xC848F11D
ANDR CS r5, r6, r7, LSL r1	0x34 (110100)	0x5 (0101)	0x6 (0110)	0x7 (0111)	1 (1)	001 (001)	0x1 (0001)	0b01 (01)	0x3 (0011)	0xD159E453
SHFT r1, ASL #3	0x25 (100101)	0x1 (0001)	0x0 (0000)	0x0 (0000)	0 (0)	011 (011)	0x3 (0011)	0b10 (10)	0x0 (0000)	0x94400CE0

Table 7.4: Register Format Encoding Examples

Notes:

- When only two register operands are given, R2 is encoded as a copy of R1 and the second operand is encoded in R3, for example ADDR r4, r5 uses R1=0x4, R2=0x4, R3=0x5
- SO (Shift Operand): 0 = literal shift amount, 1 = register contains shift amount (see the SUBR|» and ANDR|CS rows)
- ST (Shift Type): 000=none, 001=LSL, 010=LSR, 011=ASL, 100=ASR, 101=RTL, 110=RTR
- AF for memory ops: 00=l, 01=a, 10=s, 11=p; for ALU ops: 00=no update, 01=update from ALU, 10=update from shift (see the SHFT row)
- Condition code examples: 0x0 = Always, 0xD = Signed > (>), 0x3 = Carry Set (CS)
- SHFT is a meta-instruction using ORRI opcode 0x25 with AF=10

7.7 Operand Encoding Details

7.7.1 Condition Codes

Code	Mnemonic	Condition
0000	AL (or none)	Always (unconditional)
0001	==	Equal ($Z = 1$)
0010	!=	Not Equal ($Z = 0$)
0011	CS	Carry Set ($C = 1$)
0100	CC	Carry Clear ($C = 0$)
0101	NS	Negative Set ($N = 1$)
0110	NC	Negative Clear ($N = 0$)
0111	OS	Overflow Set ($V = 1$)
1000	OC	Overflow Clear ($V = 0$)
1001	HI	Unsigned Higher ($C = 0$ AND $Z = 0$)
1010	LO	Unsigned Lower or Same ($C = 1$ OR $Z = 1$)
1011	>=	Signed Greater or Equal ($N = V$)
1100	«	Signed Less Than ($N \neq V$)
1101	»	Signed Greater Than ($Z = 0$ AND $N = V$)
1110	<=	Signed Less or Equal ($Z = 1$ OR $N \neq V$)
1111	NV	Never (never executes)

Table 7.5: Condition Code Encoding

For more details on condition codes see **Chapter 10**

7.7.2 Shift Types

Code	Type	Mnemonic
000	None (no shift)	NUL
001	Logical Shift Left	LSL
010	Logical Shift Right	LSR
011	Arithmetic Shift Left	ASL
100	Arithmetic Shift Right	ASR
101	Rotate Left	RTL
110	Rotate Right	RTR
111	Reserved	—

Table 7.6: Shift Type Encoding

For more details on shift operations see **Chapter 9**

7.8 Opcode Encoding Patterns

The SIRCIS instruction set uses systematic opcode patterns to simplify decoding:

7.8.1 Opcode Groups

- **0x0_**: Immediate operand

- **0x1_**: Memory operations
- **0x2_**: Short immediate with shift
- **0x3_**: Register operand

7.8.2 Test vs. Save

This pattern applies to the ALU groups **0x0_**, **0x2_**, and **0x3_** only. Each ALU operation has both a "save result" and a "test only" variant:

- **0x_0–0x_6**: Save result to destination
- **0x_8–0x_E**: Test only (update flags, do not save the result)
- **0x_F**: Coprocessor call (**COP1/COPR**), not a test-only form

Adding **0x8** to a "save result" opcode in the range **0x_0–0x_6** gives its "test only" form. Opcodes **0x27** and **0x2F** are exceptions to this pattern: they are undocumented opcodes (see Appendix C), not a "save result" operation or a coprocessor call.

For example:

- **0x04**: **ANDI** – AND immediate, save result
- **0x0C**: **TSAI** – Test AND immediate (does not save the result)

7.9 Instruction Fetch and Alignment

7.9.1 Fetch Process

Since instructions are 32 bits and the data bus is 16 bits wide, each instruction requires two fetch cycles:

1. Fetch the high word from the address in `p1`.
2. Fetch the low word from `p1 + 1`.
3. After decode, `p1` advances by 2.

`p1` still holds the address of the instruction being executed while it is decoded, because address calculations that select `p` use that value.

7.9.2 Alignment Requirements

Instructions span two consecutive words and must begin at an even word address. The program counter (`p1`) must always be even.

Attempting to fetch an instruction from an odd word address will trigger an alignment fault exception.

7.10 Format Selection Guidelines

When writing assembly code, the assembler automatically selects the appropriate format based on operands:

Assembly	Format Used	Notes
<code>ADDI r1, #1000</code>	Immediate	Full 16-bit immediate
<code>ADDI r1, #10, LSL #2</code>	Short Immediate	8-bit immediate + shift
<code>ADDR r1, r2, r3</code>	Register	Three register operands

Table 7.7: Format Selection Examples

The choice of format affects instruction encoding but is generally transparent to the programmer, as the assembler handles the details.

CHAPTER 8

Addressing Modes

8.1 Overview

The SIRC-1 CPU supports seven addressing modes that specify how operands are accessed. These modes determine whether an operand is an immediate value, register contents, an address-register pair named directly, or data in memory at a computed address. The 8-bit short-immediate encoding used by ALU shift-and-add forms is an encoding-width variant of Immediate addressing, not a separate mode; Table 8.2 maps every operand form the assembler accepts onto one of the seven modes.

Mode	Description
Immediate	Operand is a constant value encoded in the instruction
Register Direct	Operand is the contents of a specified register
Address Register Direct	Operand names an address-register pair directly, with no indirection
Indirect Immediate	Memory address = address register + immediate offset
Indirect Register	Memory address = address register + register offset
Post-Increment	Indirect register, then increment address register
Pre-Decrement	Decrement address register, then use as indirect

Table 8.1: Addressing Modes Summary

Assembler operand form	Example	Addressing mode
Immediate	#100	Immediate
Immediate, 8-bit shift-qualified	#10, LSL #2	Immediate (encoding-width variant; see Short Immediate Addressing)
Register Direct	r1	Register Direct
Address Register Direct	a	Address Register Direct
Indirect Register Displacement	(r2, a)	Indirect Register
Indirect Immediate Displacement	(#8, a), (a)	Indirect Immediate
Indirect Register Displacement, Post-Increment	(r2, s) +	Post-Increment
Indirect Immediate Displacement, Post-Increment	(#2, s) +, (s) +	Post-Increment
Indirect Register Displacement, Pre-Decrement	-(r2, s)	Pre-Decrement
Indirect Immediate Displacement, Pre-Decrement	-(#2, s), -(s)	Pre-Decrement
Shift Definition	LSL #2	Not an addressing mode; modifies the register operand it follows (Chapter 9)

Table 8.2: Assembler Operand Forms Mapped to Addressing Modes

8.2 Addressing Mode Matrix

Mode	Syntax	Effective value or address	Memory access	Legal families
Immediate	#imm16, or #imm8, shift (short-immediate width variant)	16-bit encoded value; the 8-bit short-immediate form is zero-extended, and its shift applies to the register operand	None	ALU, LOAD-immediate, COPI, branch meta-instructions
Register Direct	rN	16-bit register value	None	ALU, LOAD register-copy, COPR, register-offset forms
Address Register Direct	addr	addr.high : addr.low, used directly	None	LJMP/LJSR src, LDEA/LDEL dest
Indirect Immediate	(#offset, addr), (addr)	addr.high : (addr.low + offset)	LOAD reads; STOR writes; LDEA/LDEL none	Memory, LDEA, LDEL
Indirect Register	(r0, addr)	addr.high : (addr.low + r0)	LOAD reads; STOR writes; LDEA/LDEL none	Memory, LDEA, LDEL
Post-Increment	(#offset, addr)+, (addr)+, (r0, addr)+	Address is calculated from original addr; addr.low += 1 after access	LOAD reads; LDEL none	LOAD, LDEL, LJSR (assembles to LDEL)
Pre-Decrement	-(#offset, addr), -(addr), -(r0, addr)	addr.low -= 1; address then uses decremented addr.low + offset	STOR writes; LDEA none	STOR, LDEA (and LDEA-derived meta-instructions)

Table 8.3: Addressing Mode Side Effects and Legal Instruction Families

8.3 Legal Modes by Instruction Family

Table 8.3 answers "what does this addressing-mode syntax mean, and which instruction families can use it in general?" Table 8.4 answers the reverse question, "for this specific instruction, exactly which operand forms are legal?" The two tables must agree; consult the one that matches the question you are asking.

Instruction family	Legal operand forms	Explicit exclusions and notes
ALU result instructions	OPI rD, #imm16; OPI rD, #imm8, shift; OPR rD, rS1, rS2[, shift]; register shorthand OPR rD, rS2.	No memory operands. Full 16-bit immediate forms do not encode shifts.
ALU test instructions	CMPI/TSAl/TSXI rS, #imm16; CMPI/TSAl/TSXI rS, #imm8, shift; CMPR/TSAR/TSXR rS1, rS2[, shift].	No destination write-back. No memory operands.
LOAD value	LOAD rD, #imm16; LOAD rD, rS.	No public short-immediate LOAD syntax. Direct register-copy forms do not accept shift suffixes.
LOAD memory	LOAD rD, (#offset, addr); LOAD rD, (rO, addr); LOAD rD, (#offset, addr)+; LOAD rD, (rO, addr)+.	No pre-decrement LOAD form. No shift suffix on any LOAD form.
STOR memory	STOR (#offset, addr), rS; STOR (rO, addr), rS[, shift]; STOR -(#offset, addr), rS; STOR -(rO, addr), rS[, shift].	No post-increment STOR form. Shift suffix is only available on register-displacement STOR forms.
LDEA	LDEA dest, (#offset, src); LDEA dest, (rO, src); LDEA dest, -(#offset, src); LDEA dest, -(rO, src).	No post-increment LDEA form. No shift suffixes.
LDEL	LDEL dest, (#offset, src); LDEL dest, (rO, src); LDEL dest, (#offset, src)+; LDEL dest, (rO, src)+.	No pre-decrement LDEL form. No shift suffixes.
Control-flow meta-instructions	BRAN/BRSR #disp; BRAN/BRSR @label; RETS; LJMP src[, #offset rO]; LJSR src[, #offset rO]; LJSR (#offset, src)+; LJSR (rO, src)+.	LJMP (src) and LJSR (src) are not public syntax; use LJMP src or LJSR src for zero displacement.
Coprocessor	COPI #imm16; COPR rS; standard coprocessor meta-instructions as documented in Chapter 16.	No hidden destination register syntax, no memory operands, no shifts, and no status-update override suffixes.

Table 8.4: Addressing Mode Legality by Instruction Family

8.4 Zero-Displacement Shorthand

Immediate-displacement indirect forms may omit the displacement when it is zero. The following forms are exact assembler-level equivalents:

Shorthand	Canonical form	Meaning
(addr)	(#0, addr)	Use the selected address register
(addr)+	(#0, addr)+	Use the address, then increment
-(addr)	-(#0, addr)	Decrement, then use the address

Table 8.5: Zero-Displacement Shorthand

The shorthand does not change which instruction families are legal. For example, `LOAD r1, (a)+` is legal because **LOAD** has a post-increment immediate-displacement form, but `STOR (a)+, r1` is not legal because **STOR** has no post-increment form. Likewise, `STOR -(s), r1` is legal because **STOR** has a pre-decrement form, but the corresponding pre-decrement **LOAD** form is not legal.

Register-displacement forms do not have a corresponding shorthand: `(rO, addr)`, `(rO, addr)+`, and `-(rO, addr)` always name the displacement register explicitly.

8.5 Operand Categories

Instruction descriptions use these operand categories:

Register value A 16-bit general-purpose, status, or address-component register operand used directly by an ALU, load, store, or coprocessor instruction.

Immediate value A constant encoded in the instruction. Full immediates are 16 bits; short immediates are 8 bits.

Memory source An effective address read by **LOAD**.

Memory destination An effective address written by **STOR**.

Branch target An effective address written to the program-counter pair p .

Address destination An address-register pair written by **LDEA** or **LDEL**.

Alterable destination Any destination register, address-register pair, or memory location that the instruction can modify when its condition is true.

8.6 Common Address Calculation Rules

For indirect, post-increment, pre-decrement, branch, jump, and effective-address forms, address arithmetic is performed on the low word of the selected address-register pair. The high word supplies the segment and is preserved unless the instruction explicitly writes an address-register pair destination.

Post-increment and pre-decrement change the selected address register by exactly one word. The displacement or index register participates only in the effective address calculation; it does not change the auto-update amount.

When p is used as the source address register, the value used for address calculation is the address of the instruction currently executing. PC-relative displacements are therefore relative to the address of the branch instruction itself, not to the next instruction; a displacement of 2 targets the following instruction and a displacement of 0 re-executes the branch. **BRSR** and **LDEL** still link to $p1 + 2$.

8.7 Immediate Addressing

8.7.1 Description

The operand is a constant 16-bit value encoded directly in the instruction.

8.7.2 Syntax

```
1 ADDI r1, #100 ; 16-bit immediate
2 ADDI r2, #-50 ; Signed 16-bit immediate
```

8.7.3 Effective Value

16-bit value encoded into the instruction. Arithmetic instructions interpret this field according to Chapter 4.

8.7.4 Usage

- Loading constant values
- Arithmetic with known values
- Bit manipulation with masks

8.8 Register Direct Addressing

8.8.1 Description

The operand is the contents of a specified register. This is the most common mode for ALU operations in a load/store architecture.

8.8.2 Syntax

```
1 ADDR r1, r2, r3 ; r1 = r2 + r3
2 ADDR r4, r5 ; r4 = r4 + r5
```

8.8.3 Effective Value

The 16-bit value currently stored in the specified register.

8.8.4 Usage

- All ALU operations between registers
- Register-to-register data movement
- Testing register values

8.9 Address Register Direct Addressing

8.9.1 Description

The operand names an address-register pair (*l*, *a*, *s*, or *p*) directly, without indirection. **LDEA** and **LDEL** use this form to select the destination pair that receives the computed address; **LJMP** and **LJSR** use it to select the source pair whose contents become the new value of *p*.

8.9.2 Syntax

```
1 LDEA a, (#4, s) ; a = s + 4 (destination named directly)
2 LJMP a ; p = a (source named directly)
```

8.9.3 Effective Value

The full 32-bit contents of the named address-register pair (high word and low word).

8.9.4 Usage

- Selecting which address-register pair an effective-address instruction writes
- Jumping or calling through an address already held in a register pair

8.10 Indirect Immediate Addressing

8.10.1 Description

The effective address is computed by adding a 16-bit signed immediate offset to an address register pair low word. Memory instructions use the computed address for a load or store; effective-address instructions write the computed address to an address-register pair.

8.10.2 Syntax

```
1 LOAD r1, (#8, a) ; Load from address (a + 8)
2 STOR (#-4, s), r2 ; Store to address (s - 4)
3 LDEA p, (#100, p) ; Jump relative (pc = pc + 100)
```

8.10.3 Effective Address

$$EA = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{Offset}) \quad (8.1)$$

The address register can be: `l` (link), `a` (address), `s` (stack), or `p` (program counter).

Offset is a signed two's-complement value. When `SR.A` (Trap on Address Overflow) is set, the Segment Overflow Fault (Chapter 6) fires only if the low-word addition wraps; a negative offset that keeps the sum within the low word's range does not fault.

8.10.4 Usage

- Accessing structure fields (fixed offset from base)
- Array element access (if index is constant)
- Stack frame access (local variables)
- Position-independent code (PC-relative addressing)

8.11 Indirect Register Addressing

8.11.1 Description

Similar to indirect immediate, but the offset is taken from a register instead of being encoded as an immediate value. This enables runtime-computed offsets.

8.11.2 Syntax

```
1 LOAD r1, (r2, a) ; Load from address (a + r2)
2 STOR (r4, s), r3 ; Store to address (s + r4)
```

8.11.3 Effective Address

$$EA = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{IndexRegister}) \quad (8.2)$$

8.11.4 Usage

- Array indexing with variable index
- Table lookups
- Dynamic offset calculations

8.12 Post-Increment Addressing

8.12.1 Description

The effective address is computed using indirect immediate or indirect register addressing, then the low word of the address register pair is incremented by one after the memory operation.

This mode is particularly useful for iterating through arrays or implementing stack operations.

8.12.2 Syntax

```
1 LOAD r1, (r2, s)+ ; Load from (s + r2), then s = s + 1
2 LOAD r1, (#2, s)+ ; Load from (s + 2), then s = s + 1
```

8.12.3 Operation Sequence

1. Compute $EA = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{Offset})$
2. Perform memory operation (load)
3. $\text{AddressRegister.low} = \text{AddressRegister.low} + 1$

8.12.4 Usage

- Stack pop operations
- Forward array traversal
- Buffer reading

8.13 Pre-Decrement Addressing

8.13.1 Description

The low word of the address register pair is decremented before computing the effective address for the memory operation.

This mode is the complement of post-increment and is useful for stack push operations.

8.13.2 Syntax

```
1 STOR -(r2, s), r1 ; s = s - 1, then store to (s + r2)
2 STOR -(#2, s), r3 ; s = s - 1, then store to (s + 2)
```

8.13.3 Operation Sequence

1. $\text{AddressRegister.low} = \text{AddressRegister.low} - 1$
2. Compute $EA = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{Offset})$
3. Perform memory operation (store)

8.13.4 Usage

- Stack push operations
- Backward array traversal
- Buffer writing

8.14 Short Immediate Addressing

8.14.1 Description

Similar to immediate addressing, but limited to 8-bit values so shift information can fit into the instruction encoding. This is a width variant of Immediate addressing, not a separate addressing mode; see Table 8.2.

8.14.2 Syntax

```
1 ADDI r1, #10, LSL #2 ; r1 = (r1 << 2) + 10
2 SUBI r2, #1, LSL #8 ; r2 = (r2 << 8) - 1
```

8.14.3 Effective Value

8-bit value encoded into the instruction and zero-extended before ALU arithmetic.

The shift applies to the register operand, not to the immediate constant.

8.14.4 Usage

- Loading constant values
- Arithmetic with known values
- Bit manipulation with masks

8.15 Address Register Selection

Many addressing modes require specifying which address register pair to use. The 2-bit address register field encodes:

Bits	Symbol	Register Pair
00	l	Link Register (lh, ll)
01	a	Address Register (ah, al)
10	s	Stack Pointer (sh, sl)
11	p	Program Counter (ph, pl)

Table 8.6: Address Register Encoding

8.16 Addressing Mode Examples

8.16.1 Example 8-1: Structure Access

```

1 ; Assume 'a' points to a struct:
2 ; struct Point {
3 ; word 0: int16_t x
4 ; word 1: int16_t y
5 ; }
6
7 LOAD r1, (#0, a) ; r1 = point.x
8 LOAD r2, (#1, a) ; r2 = point.y

```

8.16.2 Example 8-2: Array Iteration

```

1 ; Iterate array of 16-bit values
2 ; Assume 'a' points to array start
3 ; r7 = counter
4 :loop
5 LOAD r1, (#0, a)+ ; Load element, advance by 1
6 ; ... process r1 ...
7 SUBI r7, #1
8 BRAN |!= @loop

```

8.16.3 Example 8-3: Stack Frame

```

1 :begin_subroutine
2
3 ; Function prologue - save registers
4 STOR -(#0, s), r1
5 STOR -(#0, s), r2
6 STOR -(#0, s), r3
7
8 ; Access parameters (above saved regs)
9 LOAD r4, (#3, s) ; First parameter
10 LOAD r5, (#4, s) ; Second parameter
11
12 ; Function epilogue - restore registers
13 LOAD r3, (#0, s)+
14 LOAD r2, (#0, s)+
15 LOAD r1, (#0, s)+
16 RETS

```

8.17 Addressing Mode Restrictions

8.17.1 Privilege Mode Restrictions

Protected mode restricts writes to the high word of an address register:

- **Direct high-register writes:** Trigger a privilege violation fault
- **Address-register-pair write-back:** Allowed only when the high word would remain unchanged; otherwise triggers a privilege violation fault
- **Post-increment/Pre-decrement:** Always allowed in protected mode; the auto-update writes only the low word of the pair and never carries into the high word

8.17.2 Alignment

- All memory addresses refer to 16-bit words
- There are no alignment restrictions for general memory I/O
- All instructions span 2 words and must begin at an even word address
- Unaligned instruction fetches (odd word address) trigger an alignment fault exception

CHAPTER 9

Shift Operations

9.1 Overview

The SIRC-1 CPU supports powerful shift operations that can be applied to operands in many instructions. Shifts can multiply or divide values by powers of 2, extract bit fields, or perform bit rotations.

Shift operations are encoded within Short Immediate and Register format instructions, allowing a value to be shifted before being used in an ALU operation—all within a single instruction.

Shift operations are only applied to source operands. A shift cannot be applied to the result of an operation before register writeback.

Note: By default, when shifts are used with ALU instructions (apart from the **SHFT** meta-instruction), the status register flags are updated based on the ALU operation result, not the shift. This behavior can be overridden using the status override syntax (see Section 9.12).

9.2 Shift Types

9.2.1 Shift Type Encoding

Code	Mnemonic	Type	Description
000	NUL	–	No shift performed
001	LSL	Logical Shift Left	Shift left, fill with zeros
010	LSR	Logical Shift Right	Shift right, fill with zeros
011	ASL	Arithmetic Shift Left	Same data result as LSL; sets V on signed overflow
100	ASR	Arithmetic Shift Right	Shift right, preserving sign bit
101	RTL	Rotate Left	Rotate left within the operand
110	RTR	Rotate Right	Rotate right within the operand
111	–	Reserved	Reserved for future use

Table 9.1: Shift Type Encoding

NUL is the no-shift form: it leaves the operand unchanged but still routes it through the shifter, so **SHFT rD, NUL #0** is a legal way to drive the status flags from the unshifted value of **rD**.

9.3 Shift Syntax

A shift suffix may follow the ALU short-immediate and ALU register forms, the **SHFT** meta-instruction, and the register-displacement **LOAD/STOR** forms. It is rejected on **LOAD rD, rS**, on **LDEA/LDEL**, and on coprocessor instructions.

For ALU instructions, the shift is applied in the "Decode and Register Fetch" phase, before the ALU operation. In register-format instructions the shifted operand is **R2** (first source); in short-immediate instructions it is the register named by the **Reg** field (first source, since the short-immediate format has no separate source register). In register-displacement **LOAD** forms the shift is applied to the offset register used to compute the effective address, never to the value loaded from memory;

in register-displacement **STOR** forms it is applied to the source register before its value is written to memory.

If using the three-operand form of an instruction, the shifted operand is the second operand. If using the two-operand form, the second operand is inferred to be the same as the destination.

See this table for some examples of which operand is shifted:

Instruction	How the CPU Interprets It	First Source Operand
ADDR r1, r2, r3, LSL #1	ADDR r1, r2, r3, LSL #1	r2
ADDR r1, r2, LSL #1	ADDR r1, r1, r2, LSL #1	r1
ADDI r1, #2, LSL #1	ADDI r1, r1, #2, LSL #1	r1

Table 9.2: Examples of first source operands

9.4 Shift Type Details

9.4.1 Logical Shift Left (LSL)

Operation: Shift bits to the left, filling vacated bits with zeros.

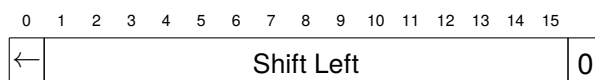


Figure 9.1: Logical Shift Left

- The last bit shifted out of bit 15 (bit 16-n for a count of n) is placed in the carry flag
- Bit 0 receives 0
- Equivalent to multiplying by 2^n where n is the shift count

Example 9-1:

```

1 ; Multiply r1 by 4
2 ADDI r1, #0, LSL #2 ; r1 = (r1 << 2) = r1 * 4
3
4 ; Set bit 10 (16-bit immediate form)
5 ORRI r3, #0x0400 ; r3 |= 0x0400

```

9.4.2 Logical Shift Right (LSR)

Operation: Shift bits to the right, filling vacated bits with zeros.

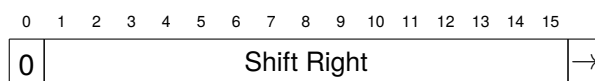


Figure 9.2: Logical Shift Right

- The last bit shifted out of bit 0 (bit n-1 for a count of n) is placed in the carry flag
- Bit 15 receives 0
- Equivalent to unsigned division by 2^n
- Use for unsigned values only

Example 9-2:

```

1 ; Divide unsigned value by 8
2 ADDI r1, #0, LSR #3 ; r1 = (r1 >> 3) = r1 / 8 (unsigned)
3
4 ; Extract high byte
5 ADDI r3, #0, LSR #8 ; r3 = (r3 >> 8)

```

9.4.3 Arithmetic Shift Left (ASL)

Operation: Produces the same data result as LSL, but sets the overflow flag if the sign bit changes.

- Equivalent to signed multiplication by 2^n
- Can cause signed overflow if sign bit changes
- Overflow flag is set if sign bit changes during shift

Example 9-3:

```

1 ; Multiply signed value by 2
2 ADDI r1, #0, ASL #1 ; r1 = (r1 << 1) = r1 * 2 (signed)

```

9.4.4 Arithmetic Shift Right (ASR)

Operation: Shift bits to the right, preserving the sign bit (bit 15).

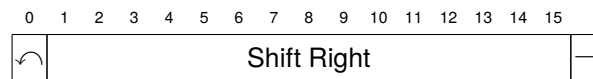


Figure 9.3: Arithmetic Shift Right

- The last bit shifted out of bit 0 (bit n-1 for a count of n) is placed in the carry flag
- Bit 15 (sign bit) is replicated into all bits vacated by the shift
- Equivalent to signed division by 2^n (rounds toward negative infinity)
- Use for signed values only

Example 9-4:

```

1 ; Divide signed value by 4
2 ADDI r1, #0, ASR #2 ; r1 = (r1 >> 2) = r1 / 4 (signed)
3
4 ; Average two signed values
5 ADDR r3, r4, r5
6 ADDI r3, #0, ASR #1 ; r3 = (r3 >> 1) = (r4 + r5) / 2

```

9.4.5 Rotate Left (RTL)

Operation: Rotate bits to the left within the 16-bit operand. The bit that wraps from bit 15 to bit 0 is also copied into the carry flag. The incoming carry flag is not used as an input.

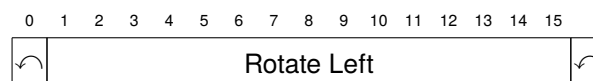


Figure 9.4: Rotate Left

- Bit 15 wraps into bit 0 and is copied into the carry flag
- The previous carry flag value is ignored
- No data is lost
- Useful for bit manipulation, circular buffers, and cyclic bit patterns

Example 9-5:

```

1 ; Rotate r1 left by 1
2 SHFT r1, RTL #1
3
4 ; Circular rotate of one word
5 SHFT r3, RTL #4

```

9.4.6 Rotate Right (RTR)

Operation: Rotate bits to the right within the 16-bit operand. The bit that wraps from bit 0 to bit 15 is also copied into the carry flag. The incoming carry flag is not used as an input.

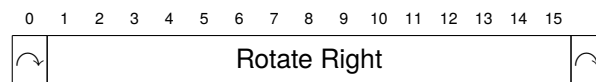


Figure 9.5: Rotate Right

- Bit 0 wraps into bit 15 and is copied into the carry flag
- The previous carry flag value is ignored
- No data is lost

Example 9-6:

```

1 ; Rotate r1 right by 1
2 SHFT r1, RTR #1
3
4 ; Check if bit 0 is set
5 SHFT r2, RTR #1 ; Bit 0 -> carry
6 ; Now branch on carry to test the old bit 0

```

9.5 Shift Operands

The Shift Operand (SO) bit determines what value is shifted:

SO Bit	Mode	Description
0	Shift by Immediate	The first source operand (Register format: R2; Short Immediate format: the Reg field) is shifted by a literal amount
1	Shift Count in Register	The first source operand (Register format: R2; Short Immediate format: the Reg field) is shifted by the amount in the shift count register

Table 9.3: Shift Operand Modes

9.5.1 Mode 0: Shift by Immediate

The first source operand (R2) is shifted by a literal amount before the ALU operation.

```
1 ; Multiply r1 by 4 and add 1
2 ADDI r1, #1, LSL #2 ; r1 = (r1 << 2) + 1
3                               ; SO=0: r1 is shifted by literal 2
```

9.5.2 Mode 1: Shift by Register Count

The shift count is taken from the shift amount field (interpreted as a register), allowing variable-length shifts. R2 (first source) is still what gets shifted.

```
1 ; Variable shift
2 ; Shift amount register contains shift count
3 ADDR r1, r2, r3, LSL r4 ; r1 = (r2 << r4) + r3
4                               ; SO=1: shift r2 by value in r4
5
6 ; Barrel shifter emulation
7 ADDR r4, r5, r6, ASR r7 ; r4 = (r5 >> r7) + r6 (arithmetic)
```

9.6 Shift Count

The shift count is 0–15 when specified as an immediate (SO = 0); the field is 4 bits wide, and both the assembler and the encoder enforce this range, so a count greater than 15 can never occur through the immediate path. When the shift count is taken from a register (SO = 1), the full 16-bit register value is available as a count, and a count greater than 15 is well defined:

- Shift count of 0 means no shift is performed
- Maximum immediate shift count is 15
- For **LSL**, **LSR**, and **ASL**, a register-supplied count of 16 or more produces the same result as a count of exactly 16: every original bit is shifted out, giving 0x0000. The carry flag reflects the last bit shifted out at count 16, as for any other count.
- For **ASR**, a register-supplied count of 16 or more sign-extends fully: 0xFFFF if the original value was negative, 0x0000 if it was positive or zero. This is the same rule as any other count, just carried to its limit – every vacated bit is the sign bit.
- **RTL** and **RTR** have no vacated bits to define, since a rotate never discards data; a register-supplied count is used modulo 16, so a count of 16 behaves as a count of 0, a count of 17 behaves as a count of 1, and so on.

9.7 Status Flag Effects

9.7.1 Default Behavior

When shift operations are used with ALU instructions, the status flags are **by default** updated based on the **ALU operation result**, not the shift operation. For example, in `CMPI r2, #1, LSL #2`, the flags reflect the comparison result, not the shift.

To update flags based on the shift operation instead, use the `[S]` status override modifier (see Section 9.12), or use the **SHIFT** meta-instruction documented in Chapter 13.

9.7.2 Carry Flag

For all shifts and rotates:

- The last bit shifted out is placed in the carry flag
- For shifts of more than 1 position, only the final bit shifted out matters
- A shift count of 0 leaves the operand unchanged and clears the carry flag. When the status register is updated from the shifter, all four condition flags are written, so no flag survives a shift-sourced update; to preserve flags across a shift, use the [N] status override

9.7.3 Zero Flag

Set if the result after shifting is zero.

9.7.4 Negative Flag

Set if bit 15 of the result is 1 (result is negative when interpreted as signed).

9.7.5 Overflow Flag

- For **ASL**: Set if the sign bit changes during the shift (signed overflow)
- For other shifts: Cleared to 0

9.8 Common Use Cases

9.8.1 Multiplication by Constants

Example 9-7:

```
1 ; Multiply by 2
2 ADDI r1, #0, LSL #1 ; r1 = (r1 << 1) = r1 * 2
3
4 ; Multiply by 3
5 ADDR r1, r2, r2, LSL #1 ; r1 = (r2 << 1) + r2 = r2 * 3
6
7 ; Multiply by 5
8 ADDR r1, r2, r2, LSL #2 ; r1 = (r2 << 2) + r2 = r2 * 5
```

9.8.2 Division by Powers of 2

Example 9-8:

```
1 ; Unsigned divide by 4
2 LOAD r1, r2 ; r1 = r2
3 ADDI r1, #0, LSR #2 ; r1 = (r1 >> 2) = r1 / 4 (unsigned)
4
5 ; Signed divide by 8
6 LOAD r1, r2 ; r1 = r2
7 ADDI r1, #0, ASR #3 ; r1 = (r1 >> 3) = r1 / 8 (signed)
```

9.8.3 Bit Field Extraction

Example 9-9:

```
1 ; Extract bits 8-11 from r1
2 LOAD r2, r1 ; r2 = r1
3 ADDI r2, #0, LSR #8 ; Shift down
4 ANDI r2, #0x0F ; Mask to 4 bits
```

```

5
6 ; Extract and sign-extend bit 7
7 LOAD r2, r1 ; r2 = r1
8 ADDI r2, #0, LSL #8 ; Shift bit 7 to bit 15
9 ADDI r2, #0, ASR #8 ; Arithmetic shift to sign-extend

```

9.8.4 Bit Manipulation

Example 9-10:

```

1 ; Set bit N (where N is in r2)
2 LOAD r3, #1 ; r3 = 1
3 ADDI r3, #0, LSL r2 ; r3 = (r3 << r2) = 1 << N
4 ORRR r1, r3 ; r1 |= (1 << N)
5
6 ; Clear bit N
7 LOAD r3, #1
8 ADDI r3, #0, LSL r2 ; r3 = (r3 << r2) = 1 << N
9 LOAD r4, #0xFFFF
10 XORR r3, r4, r3 ; r3 = 0xFFFF ^ (1 << N) = ~(1 << N)
11 ANDR r1, r1, r3 ; r1 = r1 & ~(1 << N)
12
13 ; Test bit N
14 LOAD r3, #1
15 ADDI r3, #0, LSL r2 ; r3 = (r3 << r2) = 1 << N
16 TSAR r1, r3 ; Test r1 & (1 << N)
17 BRAN != @bit_was_set ; Branch if bit set

```

9.9 Multi-Word Shifts

For operations on values larger than 16 bits, shifts and rotates can be combined:

9.9.1 32-bit Left Shift

Example 9-11:

```

1 ; Shift 32-bit value in r1:r2 left by 1
2 ; r1 = high word, r2 = low word
3 SHFT r2, LSL #1 ; Shift low, MSB -> carry
4 ADCI r1, #0, LSL #1 ; Shift high and add carry into bit 0

```

9.9.2 32-bit Right Shift (Unsigned)

Example 9-12:

```

1 ; Shift 32-bit value in r1:r2 right by 1 (unsigned)
2 SHFT r1, LSR #1 ; Shift high, LSB -> carry
3 ORRI[N] r2, #0, LSR #1 ; Shift low while preserving carry from high word
4 ORRI|CS r2, #0x8000 ; If old high bit 0 was set, insert it into low bit 15

```

9.9.3 32-bit Right Shift (Signed)

Example 9-13:

```

1 ; Shift 32-bit value in r1:r2 right by 1 (signed)
2 SHFT r1, ASR #1 ; Arithmetic shift high, LSB -> carry
3 ORRI[N] r2, #0, LSR #1 ; Shift low while preserving carry from high word
4 ORRI|CS r2, #0x8000 ; If old high bit 0 was set, insert it into low bit 15

```

9.10 Performance Considerations

- All shift operations complete in the same number of cycles regardless of shift count
- Shifts can be performed "for free" as part of another instruction (no extra cycles)
- Using shifts for multiplication/division by powers of 2 is much faster than using a multiply/divide instruction (if one existed)
- Barrel shifter implementation in hardware allows any shift count in constant time

9.11 Limitations

- The immediate shift count is limited to 0–15 (4-bit field)
- A register-supplied shift count greater than 15 is architecturally undefined
- For a shift greater than 15 positions, use multiple shift instructions with immediate counts
- Rotates do not consume the carry flag; **RTL** and **RTR** rotate only within the 16-bit operand
- The reserved shift type (111) has architecturally undefined behavior

9.12 Status Flag Update Control

By default, ALU instructions with shift operations update the status register flags based on the **ALU operation result**, not the shift operation. However, the SIRC-1 provides manual control over which operation updates the flags.

9.12.1 Status Override Syntax

A status override modifier can be added immediately after an instruction mnemonic using square brackets:

Modifier	Meaning	Description
[A]	ALU	Update flags from ALU result (default)
[S]	Shift	Update flags from shift result, not ALU
[N]	None	Do not update status flags at all

Table 9.4: Status Flag Override Modifiers

9.12.2 Examples

Example 9-14: Default Behavior (ALU flags):

```

1 ; Without override, flags reflect the ALU addition result
2 ADDI r2, #1, LSL #2 ; r2 = (r2 << 2) + 1
3                               ; Flags: based on addition result

```

Example 9-15: Shift Flag Override:

```

1 ; Update flags based on shift result instead of ALU
2 ADDI[S] r2, #1, LSL #2 ; r2 = (r2 << 2) + 1
3                               ; Flags: based on (r2 << 2), not the +1
4
5 ; Check if shifted value is zero
6 CMPI[S] r3, #0, LSR #4 ; Compare (r3 >> 4) with 0
7                               ; Flags: based on shift, not comparison

```

Example 9-16: No Flag Update:

```

1 ; Perform operation without affecting status flags
2 ADDI[N] r1, #5, LSL #1 ; r1 = (r1 << 1) + 5
3                               ; Flags: unchanged
4
5 ; Useful when preserving flags from previous operation
6 CMPR r4, r5 ; Set flags based on comparison
7 ADDI[N] r6, #1 ; Increment r6 without changing flags
8 BRAN|== @equal_branch ; Branch using flags from CMPR

```

9.12.3 Use Cases**Testing Shift Results**

To determine properties of a shifted value without performing a subsequent comparison:

```

1 ; Check if r1 << 3 would be negative
2 ADDI[S] r1, #0, LSL #3 ; r1 = r1 << 3, flags from shift
3 BRAN|NS @overflow_detected ; Branch if sign bit set

```

Preserving Flags

When performing operations that should not affect condition codes:

```

1 ; Test condition
2 CMPR r1, r2 ; Compare r1 and r2
3
4 ; Do some work without affecting flags
5 ADDI[N] r3, #1 ; Increment counter (flags unchanged)
6 LOAD r4, (#0, a)+ ; Load next value (LOAD never updates flags)
7
8 ; Branch using original comparison flags
9 BRAN|>= @r1_was_greater

```

SHFT Meta-Instruction

The **SHFT** meta-instruction is the ALU-family shorthand for this common case:

```

1 SHFT r1, LSL #3 ; ORRI[S] r1, #0, LSL #3

```

See Chapter 13 for the full **SHFT** instruction entry.

9.12.4 Assembler Syntax Notes

- The status override modifier must appear immediately after the instruction mnemonic with no spaces: **ADDI**[S]
- The modifier is optional; omitting it defaults to [A] (ALU flag updates)
- The override applies to the entire instruction, including any conditional execution
- Explicit [A] is rarely needed but can improve code readability

CHAPTER 10

Condition Codes

10.1 Overview

Every SIRC-1 instruction can be executed conditionally based on the state of the condition flags in the status register. This allows for efficient implementation of conditional logic without requiring explicit branch instructions.

The 4-bit condition code field in each instruction specifies the conditions under which the instruction executes. If the condition is not met, the instruction has no architectural effect: no register, flag, memory, address-register or coprocessor state changes, and no privilege check is performed.

10.2 Condition Code Encoding

Code	Mnemonic	Condition (when instruction executes)
0000	AL (or none)	Always (unconditional)
0001	==	Equal ($Z = 1$)
0010	!=	Not Equal ($Z = 0$)
0011	CS	Carry Set ($C = 1$)
0100	CC	Carry Clear ($C = 0$)
0101	NS	Negative Set ($N = 1$)
0110	NC	Negative Clear ($N = 0$)
0111	OS	Overflow Set ($V = 1$)
1000	OC	Overflow Clear ($V = 0$)
1001	HI	Unsigned Higher ($C = 0$ AND $Z = 0$)
1010	LO	Unsigned Lower or Same ($C = 1$ OR $Z = 1$)
1011	>=	Signed Greater or Equal ($N = V$)
1100	«	Signed Less Than ($N \neq V$)
1101	»	Signed Greater Than ($Z = 0$ AND $N = V$)
1110	<=	Signed Less or Equal ($Z = 1$ OR $N \neq V$)
1111	NV	Never (never executes)

Table 10.1: Condition Code Encoding

10.3 Condition Code Categories

10.3.1 Simple Conditions

These conditions test a single status flag:

Always (default) No condition code specified; instruction always executes.

== (Equal) Executes when $Z = 1$. Typically used after **CMP** to test equality.

!= (Not Equal) Executes when $Z = 0$. Typical use: test inequality.

CS (Carry Set) Executes when $C = 1$. SIRC-1 sets Carry on a *borrow*, not on unsigned overflow of an add; after a subtraction or **CMP**, $C = 1$ means the first operand was unsigned lower than the second. This is the opposite polarity from processors (for example ARM) whose Carry flag means *no borrow* occurred.

CC (Carry Clear) Executes when $C = 0$. After a subtraction or **CMP**, $C = 0$ means no borrow occurred: the first operand was unsigned higher than or equal to the second.

NS (Negative Set) Executes when $N = 1$. Tests if result is negative (signed).

NC (Negative Clear) Executes when $N = 0$. Tests if result is positive or zero (signed).

OS (Overflow Set) Executes when $V = 1$. Indicates signed overflow.

OC (Overflow Clear) Executes when $V = 0$. No signed overflow.

10.3.2 Unsigned Comparisons

These conditions are used after comparing unsigned integers:

HI (Unsigned Higher) Executes when $C = 0$ AND $Z = 0$. Tests if first operand $>$ second operand (unsigned).

LO (Unsigned Lower or Same) Executes when $C = 1$ OR $Z = 1$. Tests if first $\leq$ second (unsigned).

Note: For unsigned comparisons:

- Use **HI** and **LO** after **CMP**
- **HI** tests if first $>$ second (unsigned)
- **LO** tests if first $\leq$ second (unsigned)
- **CS** is equivalent to unsigned $<$
- **CC** is equivalent to unsigned $\geq$
- Because Carry means borrow on SIRC-1, **CS/CC** are the opposite way round from processors whose Carry flag means no borrow (for example ARM); prefer **HI/LO** if that is easier to remember, since their names match their meaning directly.

10.3.3 Signed Comparisons

These conditions are used after comparing signed integers:

>= (Signed Greater or Equal) Executes when $N = V$. First operand $\geq$ second operand (signed).

« (Signed Less Than) Executes when $N \neq V$. First operand $<$ second operand (signed).

» (Signed Greater Than) Executes when $Z = 0$ AND $N = V$. First operand $>$ second operand (signed).

<= (Signed Less or Equal) Executes when $Z = 1$ OR $N \neq V$. First operand $\leq$ second operand (signed).

10.3.4 Special Conditions

Never (NV) Never executes: no register, flag, memory, address-register or coprocessor state changes, and no privilege check is performed. Can be used for instruction padding or to temporarily disable instructions during debugging.

10.4 Assembly Syntax

Condition codes are specified by appending a pipe character (|) followed by the condition mnemonic to the instruction:

```

1 ; Unconditional
2 ADDI r1, #10
3
4 ; Conditional - execute only if equal
5 ADDI|== r2, #20
6
7 ; Conditional branch
8 BRAN|!= @loop_start
9
10 ; Multiple conditions
11 ADDR|HI r3, r4, r5
12 STOR|NS (#0, s), r6

```

10.5 Usage with Compare Instructions

The most common use of condition codes is with compare (**CMP**) instructions:

10.5.1 Example 10-1: Unsigned Comparison

```

1 ; if (r1 > r2) { r3 = r4; } (unsigned)
2 CMPR r1, r2 ; Compare r1 and r2
3 LOAD|HI r3, r4 ; r3 = r4 if r1 > r2 (unsigned)
4
5 ; if (r1 <= 100) { branch } (unsigned)
6 CMPI r1, #100
7 BRAN|LO @target

```

10.5.2 Example 10-2: Signed Comparison

```

1 ; if (r1 >= r2) { r3 = r4; } (signed)
2 CMPR r1, r2 ; Compare r1 and r2
3 LOAD|>= r3, r4 ; r3 = r4 if r1 >= r2 (signed)
4
5 ; if (r1 < -10) { branch } (signed)
6 CMPI r1, #-10
7 BRAN|<< @negative_case

```

10.5.3 Example 10-3: Equality Testing

```

1 ; if (r1 == r2) { branch }
2 CMPR r1, r2
3 BRAN|== @equal_case
4
5 ; if (r1 != 0) { branch }
6 CMPI r1, #0
7 BRAN|!= @non_zero_case

```

10.6 Condition Code Truth Tables

10.6.1 Unsigned Comparison (CMPR rA, rB)

Relationship	Z	C	Condition	Code	Meaning
$rA = rB$	1	X	==	0001	Equal
$rA \neq rB$	0	X	!=	0010	Not Equal
$rA > rB$ (unsigned)	0	0	HI	1001	Higher
$rA < rB$ (unsigned)	X	1	CS	0011	Lower
$rA \geq rB$ (unsigned)	X	0	CC	0100	Higher or Same
$rA \leq rB$ (unsigned)	C = 1 OR Z = 1*		LO	1010	Lower or Same

Table 10.2: Unsigned Comparison Results

X = the flag value does not affect this relationship.

*Unlike every other row, LO is not a single-corner condition: it is the OR of two flag values, so it cannot be expressed as one value per column with the other marked X. This is the only row in the table with that shape.

Note: SIRC-1's Carry flag is set on a *borrow*, so CS (C = 1) means unsigned lower and CC (C = 0) means unsigned higher or same. This is the opposite pairing from processors whose Carry flag means no borrow occurred; HI and LO are unaffected and mean exactly what their names say.

10.6.2 Signed Comparison (CMPR rA, rB)

Relationship	Z	N	V	Condition	Code	Meaning
$rA = rB$	1	X	X	==	0001	Equal
$rA \neq rB$	0	X	X	!=	0010	Not Equal
$rA > rB$ (signed)	0	0	0	»	1101	Greater
$rA > rB$ (signed)	0	1	1	»	1101	Greater
$rA < rB$ (signed)	X	0	1	«	1100	Less
$rA < rB$ (signed)	X	1	0	«	1100	Less
$rA \geq rB$ (signed)	X	0	0	>=	1011	Greater or Equal
$rA \geq rB$ (signed)	X	1	1	>=	1011	Greater or Equal
$rA \leq rB$ (signed)	1	X	X	<=	1110	Less or Equal
$rA \leq rB$ (signed)	0	0	1	<=	1110	Less or Equal
$rA \leq rB$ (signed)	0	1	0	<=	1110	Less or Equal

Table 10.3: Signed Comparison Results

X = the flag value does not affect this relationship.

10.7 Advanced Usage

10.7.1 Example 10-4: Conditional Assignment

```

1 ; max = (a > b) ? a : b (unsigned)
2 CMPR r1, r2 ; Compare a and b
3 LOAD|HI r3, r1 ; r3 = a if a > b
4 LOAD|LO r3, r2 ; r3 = b if a <= b

```

10.7.2 Example 10-5: Conditional Increment

```

1 ; if (condition) count++
2 CMPI r1, #0
3 ADDI |!= r2, #1 ; Increment r2 if r1 != 0

```

10.7.3 Example 10-6: Conditional Function Call

```

1 ; if (r1 > 0) call_function()
2 CMPI r1, #0
3 LJSR |>> a ; Call function if r1 > 0 (signed)

```

10.7.4 Example 10-7: Loop Construction

```

1 ; for (i = 10; i > 0; i--)
2 LOAD r1, #10 ; i = 10
3 loop:
4     ; ... loop body ...
5     SUBI r1, #1 ; i--
6     BRAN |HI @loop ; Continue if i > 0 (unsigned)

```

10.8 Condition Code Selection Guide

Intent	After CMP	Condition Code
If equal	CMPR a, b	==
If not equal	CMPR a, b	!=
If a > b (unsigned)	CMPR a, b	HI
If a ≥ b (unsigned)	CMPR a, b	CC
If a < b (unsigned)	CMPR a, b	CS
If a ≤ b (unsigned)	CMPR a, b	LO
If a > b (signed)	CMPR a, b	»
If a ≥ b (signed)	CMPR a, b	>=
If a < b (signed)	CMPR a, b	«
If a ≤ b (signed)	CMPR a, b	<=
If negative	CMPI a, #0	NS
If positive or zero	CMPI a, #0	NC
If overflow occurred	After arithmetic	OS
If no overflow	After arithmetic	OC

Table 10.4: Condition Code Selection Guide

10.9 Performance Considerations

- A predicated instruction costs the same six cycles whether or not it executes; the saving over a branch is the branch instruction itself, not a pipeline penalty, since the SIRC-1 has no overlapped pipeline to flush.
- When a condition is false, the instruction still takes 6 cycles but performs no operation.
- For short conditional sequences, conditional execution can be faster than branching.
- For longer conditional blocks, explicit branches may be more efficient.

10.10 Common Pitfalls

- **Signed vs. Unsigned:** Using unsigned conditions (HI, LO) for signed values or vice versa produces incorrect results.
- **Flag Clobbering:** Flags are modified by most ALU instructions. Ensure the condition flags haven't been modified between the comparison and conditional instruction:

```
1 ; WRONG - flags clobbered
2 CMPR r1, r2
3 ADDI r3, #100 ; Modifies flags!
4 BRAN|HI @target ; Tests wrong flags
5
6 ; CORRECT - flags preserved
7 CMPR r1, r2
8 BRAN|HI @target
9 ADDI r3, #100 ; After the branch
```

- **Inverted Logic:** Remember that `CMP A, B` computes `A - B`, so condition codes test A relative to B, not B relative to A.

Part III

SIRCIS Instruction Reference

CHAPTER 11

Reading Instruction Descriptions

This part of the manual describes each SIRCIS instruction in a consistent format. The goal is to make each instruction entry usable as an implementation contract, not only as programmer guidance.

11.1 Instruction Entry Fields

Each instruction entry begins with a title naming the mnemonic or mnemonic family, followed immediately by one applicability line: `SIRC-1, all revisions` for CPU instructions, or the coprocessor and revision required for coprocessor instructions, per the model and revision policy in Chapter 16. The fields below then follow, in this order, for every entry:

Mnemonic The assembly mnemonic or mnemonic family, given in the entry title. Immediate and register variants are listed together when they perform the same operation.

Opcodes The 6-bit opcode values for each supported instruction format. Meta-instructions that assemble to another instruction's encoding use **Assembles to** instead of **Opcodes**.

Instruction Fields One line per encoding field the instruction uses: register fields, the condition field, the status update source field and which of its encodings are legal for this instruction, shift type and count, and immediate width. Values come from Chapter 7 and the facts digest; an entry must not invent a value here.

Syntax The accepted assembler forms. Meta-instructions are called out separately from real CPU instruction encodings.

Operands The permitted operand kinds and addressing modes for the instruction. Addressing-mode names come from the closed set in Table 8.1 (Chapter 8): Immediate, Register Direct, Address Register Direct, Indirect Immediate, Indirect Register, Post-Increment, and Pre-Decrement. An entry must not introduce a mode name outside that set; the 8-bit short-immediate encoding is a width variant of Immediate, not a mode.

Operation Pseudocode for the architectural effect of the instruction. Arithmetic is performed on 16-bit word values unless otherwise stated.

Description Prose description of the instruction's behavior, supplementing the Operation pseudocode.

Flags The effect on the condition flags in the status register.

Write-back Whether the computed result is written to a register, discarded, or used to update control state.

Exceptions Faults or traps that can be raised by the instruction. When an exception can occur, the entry must state whether partial side effects are possible.

Condition field The condition field values the instruction accepts for predicated execution. Every normal instruction may be conditional. If the condition is false, the instruction has no architectural side effects unless a later instruction description explicitly states otherwise.

Timing The base execution time and any circumstances that add wait states.

Privilege Whether the instruction is available in protected mode, supervisor mode, or both.

Example / Examples One or more worked examples of the instruction in use. Each example is numbered `Example N-M:`, where N is the chapter number (or appendix letter) and M counts from 1 within the chapter; multiple examples in one entry take consecutive numbers.

Notes Additional informative remarks. A Notes block is informative and must not introduce a requirement that is not already stated in a normative field.

11.2 Notation

Notation	Meaning
<code>rD</code>	Destination general-purpose register
<code>rS</code>	Source general-purpose register
<code>rS1</code>	First source register, usually the left-hand operand
<code>rS2</code>	Second source register, usually the right-hand operand
<code>rO</code>	Offset or index general-purpose register supplying a register displacement in a memory or effective-address form
<code>#imm16</code>	16-bit immediate word encoded in immediate format
<code>#imm8</code>	8-bit immediate encoded in short immediate format
<code>shift</code>	Optional shift definition. It applies to the register operand being shifted (the first source register in register and short-immediate formats, or the offset register for register-displacement LOAD/STOR forms); it never applies to an immediate value, a second source register, or loaded data. See Chapter 9 for the full encoding.
<code>SR.X</code>	Status register field X
<code>addr / src</code>	Source address-register pair (l, a, s, or p) supplying the base address for a memory-class or address-calculation instruction
<code>dest</code>	Destination address-register pair for an address-calculation instruction, encoded in the register field
<code>#offset / #disp</code>	Displacement added to the low word of an address-register pair
<code>@label</code>	Symbol reference resolved by the linker; PC-relative for BRAN and BRSR , an absolute low-word address otherwise
<code> cond</code>	Condition-field suffix selecting the predicate for conditional execution

Table 11.1: Instruction Description Notation

11.3 Status Flag Effect Symbols

Flag tables use the following symbols:

Symbol	Meaning
*	Updated from the instruction result
0	Cleared to zero
1	Set to one
–	Preserved
S	Updated from the shifter result when [S] is used
U	Architecturally undefined; the implementation leaves the flag in an unspecified state

Table 11.2: Status Flag Effect Symbols

11.4 Conditional Execution

Every encoded instruction contains a condition-code field. If the condition evaluates true, the instruction executes normally. If the condition evaluates false, the instruction is skipped: destination registers, address registers, memory, the program counter, link register, coprocessor state, and status flags are not modified by that instruction.

11.5 Status Update Overrides

ALU instructions can select the source of status flag updates with the status update field:

Syntax	Encoded AF	Effect
[N]	00	Preserve status flags
[A]	01	Update flags from the ALU result
[S]	10	Update flags from the shifter result
Reserved	11	Reserved for future use

Table 11.3: Status Update Source Encoding

When no override is written, ALU instructions use [A]. **SHFT** defaults to [S] instead, and **LOAD** always encodes [N] and never updates the status flags, regardless of any override syntax. Instructions that do not support status updates must not use status update override syntax.

CHAPTER 12

Instruction Summary

12.1 Overview

The SIRCIS instruction set consists of 64 possible opcodes (6-bit opcode field), of which 50 are documented and assigned; the remaining 14 are undocumented (see Appendix C). The instructions are organized into logical groups:

- **ALU Instructions** (**0x00–0x0E**, **0x20–0x2E**, **0x30–0x3E**): Arithmetic, logical, and comparison operations
- **Memory Instructions** (**0x10–0x17**): Load and store operations
- **Address and Control Flow** (**0x18–0x1F**): Effective-address calculation, jumps, and calls
- **Coprocessor** (**0x0F**, **0x3F**): Coprocessor interface

0x2F is undocumented (see Appendix C).

12.2 Complete Instruction List

Opcode	Mnemonic	Format	Flags	Operation
0x00	ADDI	Immediate	NZCV	Add immediate
0x01	ADCI	Immediate	NZCV	Add immediate with carry
0x02	SUBI	Immediate	NZCV	Subtract immediate
0x03	SBCI	Immediate	NZCV	Subtract immediate with carry
0x04	ANDI	Immediate	NZ (C,V=0)	AND immediate
0x05	ORRI	Immediate	NZ (C,V=0)	OR immediate
0x06	XORI	Immediate	NZ (C,V=0)	XOR immediate
0x07	LOAD	Immediate	–	Load immediate (move)
0x08	–	Immediate	–	Undocumented
0x09	–	Immediate	–	Undocumented
0x0A	CMPI	Immediate	NZCV	Compare immediate
0x0B	–	Immediate	–	Undocumented
0x0C	TSAI	Immediate	NZ (C,V=0)	Test AND immediate
0x0D	–	Immediate	–	Undocumented
0x0E	TSXI	Immediate	NZ (C,V=0)	Test XOR immediate
0x0F	COPI	Immediate	–	Coprocessor call immediate
0x10	STOR	Indirect Imm	–	Store to (addr + imm)
0x11	STOR	Indirect Reg	–	Store to (addr + reg)
0x12	STOR	Pre-Dec	–	Store with pre-decrement
0x13	STOR	Pre-Dec Reg	–	Store with pre-dec (reg)
0x14	LOAD	Indirect Imm	–	Load from (addr + imm)
0x15	LOAD	Indirect Reg	–	Load from (addr + reg)
0x16	LOAD	Post-Inc	–	Load with post-increment
0x17	LOAD	Post-Inc Reg	–	Load with post-inc (reg)
0x18	LDEA	Indirect Imm	–	Load effective address
0x19	LDEA	Indirect Reg	–	Load effective address (reg)
0x1A	LDEA	Pre-Dec Imm	–	Load effective address, pre-dec
0x1B	LDEA	Pre-Dec Reg	–	Load effective address, pre-dec (reg)
0x1C	LDEL	Indirect Imm	–	Load effective address and link
0x1D	LDEL	Indirect Reg	–	Load effective address and link (reg)
0x1E	LDEL	Post-Inc Imm	–	Load effective address and link, post-inc
0x1F	LDEL	Post-Inc Reg	–	Load effective address and link, post-inc (reg)
0x20	ADDI	Short Imm+Shift	NZCV	Add short immediate
0x21	ADCI	Short Imm+Shift	NZCV	Add short imm with carry
0x22	SUBI	Short Imm+Shift	NZCV	Subtract short immediate
0x23	SBCI	Short Imm+Shift	NZCV	Subtract short imm with carry
0x24	ANDI	Short Imm+Shift	NZ (C,V=0)	AND short immediate
0x25	ORRI	Short Imm+Shift	NZ (C,V=0)	OR short immediate
0x26	XORI	Short Imm+Shift	NZ (C,V=0)	XOR short immediate
0x27	–	Short Imm+Shift	–	Undocumented
0x28	–	Short Imm+Shift	–	Undocumented
0x29	–	Short Imm+Shift	–	Undocumented
0x2A	CMPI	Short Imm+Shift	NZCV	Compare short immediate
0x2B	–	Short Imm+Shift	–	Undocumented
0x2C	TSAI	Short Imm+Shift	NZ (C,V=0)	Test AND short immediate
0x2D	–	Short Imm+Shift	–	Undocumented
0x2E	TSXI	Short Imm+Shift	NZ (C,V=0)	Test XOR short immediate
0x2F	–	Short Imm+Shift	–	Undocumented
0x30	ADDR	Register	NZCV	Add register
0x31	ADCR	Register	NZCV	Add register with carry
0x32	SUBR	Register	NZCV	Subtract register
0x33	SBCR	Register	NZCV	Subtract register with carry
0x34	ANDR	Register	NZ (C,V=0)	AND register
0x35	ORRR	Register	NZ (C,V=0)	OR register
0x36	XORR	Register	NZ (C,V=0)	XOR register
0x37	LOAD	Register	–	Load from register (move)
0x38	–	Register	–	Undocumented
0x39	–	Register	–	Undocumented
0x3A	CMPR	Register	NZCV	Compare register
0x3B	–	Register	–	Undocumented
0x3C	TSAR	Register	NZ (C,V=0)	Test AND register
0x3D	–	Register	–	Undocumented
0x3E	TSXR	Register	NZ (C,V=0)	Test XOR register
0x3F	COPR	Register	–	Coprocessor call register

Table 12.1: Complete SIRCIS Instruction Set

See Table 11.2 for the flag-effect symbol definitions. The Flags column shows – for **LOAD** (0x07, 0x37) only when the default [N] status update source is used; an explicit status update override on **LOAD** leaves all four flags architecturally undefined. See Chapter 13 for details.

12.3 Instruction Grouping Patterns

12.3.1 Opcode Organization

The opcode space is systematically organized:

- **0x0_**: Immediate operand format
- **0x1_**: Memory and control flow operations
- **0x2_**: Short immediate with shift format
- **0x3_**: Register operand format

12.3.2 Save vs. Test Variants

Many ALU instructions have two variants:

- **+0x00 to +0x07**: Save result to destination register
- **+0x08 to +0x0E**: Test only (update flags, discard result)
- **+0x0F**: Coprocessor call (neither result nor flags are written)

Examples:

- **0x04** = **ANDI** (save result), **0x0C** = **TSAI** (test only)
- **0x06** = **XORI** (save result), **0x0E** = **TSXI** (test only)

12.4 Meta-Instructions

Several commonly-used operations are implemented as assembler meta-instructions that assemble to specific opcodes:

Meta-Instruction	Assembles To	Purpose
NOOP	<code>ADDI[N] r1, #0</code>	No operation
RETS	<code>LDEA p, (#0, 1)</code>	Return from subroutine
SHFT	<code>ORRI[S] rD, #0, shift</code>	Shift with status update
BRAN	<code>LDEA p, ...</code>	PC-relative branch
BRSR	<code>LDEL p, ...</code>	PC-relative call
LJMP	<code>LDEA p, ...</code>	Long jump
LJSR	<code>LDEL p, ...</code>	Long call
EXCP	<code>COPI #0x11vv</code>	User exception
WAIT	<code>COPI #0x1900</code>	Wait for interrupt
RETE	<code>COPI #0x1A00</code>	Return from exception
RSET	<code>COPI #0x1B00</code>	System reset
ETFR	<code>COPI #0x1Cxy</code>	Read exception state
ETTR	<code>COPI #0x1Dxy</code>	Write exception state
DMAR	<code>COPI #0x28xx</code>	DMA read to registers
DMAW	<code>COPI #0x29xx</code>	DMA write from registers
DMAT	<code>COPI #0x2Ann</code>	DMA memory transfer
MULU	<code>COPI #0x3000</code>	Unsigned multiply
MULS	<code>COPI #0x3100</code>	Signed multiply
DIVU	<code>COPI #0x3200</code>	Unsigned divide
DIVS	<code>COPI #0x3300</code>	Signed divide

Table 12.2: Meta-Instructions

12.5 Instruction Timing

Ordinary processing-unit instructions use exactly 6 execution phases and complete in 6 clock cycles when no bus wait states are required:

1. Instruction Fetch (High Word)
2. Instruction Fetch (Low Word)
3. Decode and Register Fetch
4. Execute and Address Calculation
5. Memory Access
6. Write Back

Coprocessor calls (**COP1**, **COPR** and every meta-instruction that assembles to one) take a second 6-cycle slot for the coprocessor dispatch that begins at the next phase 0, for 12 cycles in total before the next instruction is fetched.

This fixed sequence simplifies hardware design. Delayed bus acknowledgement may hold a phase and add wait cycles. DMA meta-instructions dispatch through the normal 6-cycle **COP1** path and then add DMA command and transfer cycles.

12.6 Next Chapters

The following chapters provide detailed documentation for each instruction group:

- **Chapter 13:** ALU instructions (arithmetic, logical, compare)
- **Chapter 14:** Load and store operations
- **Chapter 15:** Branches, jumps, and address operations
- **Chapter 16:** Coprocessor interface
- **Chapter 17:** Meta-instruction cross-reference

CHAPTER 13

ALU Instructions

13.1 Overview

The ALU (Arithmetic Logic Unit) instructions perform arithmetic and logical operations on register operands. All ALU instructions follow the load/store architecture principle: they operate only on registers, not directly on memory.

ALU instructions are available in three format variants:

- **Immediate:** Operation with 16-bit constant
- **Short Immediate with Shift:** Operation with 8-bit constant and optional shift
- **Register:** Operation between registers with optional shift

13.2 ALU Instruction Families

13.2.1 Arithmetic Instructions

- **ADD** – Addition
- **ADC** – Add with Carry
- **SUB** – Subtraction
- **SBC** – Subtract with Carry (Borrow)

13.2.2 Logical Instructions

- **AND** – Bitwise AND
- **ORR** – Bitwise OR
- **XOR** – Bitwise XOR (Exclusive OR)

13.2.3 Data Movement

- **LOAD** – Load immediate or copy register

13.2.4 Test and Compare

- **CMP** – Compare (subtract without saving result)
- **TSA** – Test AND (AND without saving result)
- **TSX** – Test XOR (XOR without saving result)

13.3 Legal Forms

ALU instructions are register-only operations. Immediate operands are encoded in the instruction word; memory operands are not legal for this instruction family.

Instruction	Syntax	Encoding	Notes
ADD	ADDI rD, #imm16	0x00	Writes result
ADD	ADDI rD, #imm8, shift	0x20	Writes result
ADD	ADDR rD, rS1, rS2[, shift]	0x30	Writes result
ADD	ADDR rD, rS2[, shift]	0x30	Writes result (shorthand: rS1 = rD)
ADC	ADCI rD, #imm16	0x01	Writes result
ADC	ADCI rD, #imm8, shift	0x21	Writes result
ADC	ADCR rD, rS1, rS2[, shift]	0x31	Writes result
ADC	ADCR rD, rS2[, shift]	0x31	Writes result (shorthand: rS1 = rD)
SUB	SUBI rD, #imm16	0x02	Writes result
SUB	SUBI rD, #imm8, shift	0x22	Writes result
SUB	SUBR rD, rS1, rS2[, shift]	0x32	Writes result
SUB	SUBR rD, rS2[, shift]	0x32	Writes result (shorthand: rS1 = rD)
SBC	SBCI rD, #imm16	0x03	Writes result
SBC	SBCI rD, #imm8, shift	0x23	Writes result
SBC	SBCR rD, rS1, rS2[, shift]	0x33	Writes result
SBC	SBCR rD, rS2[, shift]	0x33	Writes result (shorthand: rS1 = rD)
AND	ANDI rD, #imm16	0x04	Writes result
AND	ANDI rD, #imm8, shift	0x24	Writes result
AND	ANDR rD, rS1, rS2[, shift]	0x34	Writes result
AND	ANDR rD, rS2[, shift]	0x34	Writes result (shorthand: rS1 = rD)
ORR	ORRI rD, #imm16	0x05	Writes result
ORR	ORRI rD, #imm8, shift	0x25	Writes result
ORR	ORRR rD, rS1, rS2[, shift]	0x35	Writes result
ORR	ORRR rD, rS2[, shift]	0x35	Writes result (shorthand: rS1 = rD)
XOR	XORI rD, #imm16	0x06	Writes result
XOR	XORI rD, #imm8, shift	0x26	Writes result
XOR	XORR rD, rS1, rS2[, shift]	0x36	Writes result
XOR	XORR rD, rS2[, shift]	0x36	Writes result (shorthand: rS1 = rD)
LOAD	LOAD rD, #imm16	0x07	Writes result; preserves flags
LOAD	LOAD rD, rS	0x37	Writes result; preserves flags
CMP	CMPI rS, #imm16	0x0A	Updates flags only
CMP	CMPI rS, #imm8, shift	0x2A	Updates flags only
CMP	CMPR rS1, rS2[, shift]	0x3A	Updates flags only
TSA	TSAI rS, #imm16	0x0C	Updates flags only
TSA	TSAI rS, #imm8, shift	0x2C	Updates flags only
TSA	TSAR rS1, rS2[, shift]	0x3C	Updates flags only
TSX	TSXI rS, #imm16	0x0E	Updates flags only
TSX	TSXI rS, #imm8, shift	0x2E	Updates flags only
TSX	TSXR rS1, rS2[, shift]	0x3E	Updates flags only

Table 13.1: Legal ALU Instruction Forms

Register shorthand: When a register-form ALU instruction omits `rS1`, the assembler encodes `rD` as both the destination and first source operand. For example, `ADDR r1, r2` is encoded as `ADDR r1, r1, r2`. A shift definition may be combined with the shorthand; because the shift always applies to the left operand, `ADDR r1, r2, LSL #2` computes $(r1 \ll 2) + r2$, not $r1 + (r2 \ll 2)$.

Short-immediate LOAD: Opcode `0x27` has no public assembly syntax and is undocumented.

13.4 Common ALU Semantics

Topic	Definition
Condition false	The instruction is skipped and has no side effects. Registers and status flags are preserved.
Immediate format	The left operand is the named register. The right operand is the encoded 16-bit immediate.
Short immediate format	The left operand is the named register after the encoded shift is applied. The right operand is the encoded 8-bit immediate.
Register format	The left operand is <code>rS1</code> after the encoded shift is applied. The right operand is <code>rS2</code> .
Test variants	CMP , TSA , and TSX compute a result for flag updates but do not write it back.
LOAD	LOAD writes the selected operand to <code>rD</code> and does not update status flags.
Timing	ALU instructions complete in 6 cycles, plus any instruction-fetch wait states.
Exceptions	Raises no data-memory faults. In protected mode, an ALU instruction whose register field names a privileged register raises a privilege-violation fault at decode; see the notes below.

Table 13.2: Common ALU Semantics

ALU instruction execution does not perform data-memory access and does not raise data bus, data bus-protection, data alignment, segment-overflow, or invalid-opcode faults. In protected mode, an ALU instruction whose register field names `sr`, `ah`, `lh`, `ph`, or `sh` raises a privilege-violation fault at decode and has no other effect. Instruction fetch can still raise the normal fetch faults before an ALU instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits. Every 6-bit opcode decodes; no processing-unit encoding raises an invalid-opcode fault. Encodings outside the documented ALU forms execute with architecturally undefined results (see Appendix C).

13.5 Status Flag Updates

Most ALU instructions update the status register flags:

Arithmetic (ADD, ADC, SUB, SBC, CMP) Update all four flags: N, Z, C, V

Logical (AND, ORR, XOR, TSA, TSX) Update N and Z; clear C and V

Load (LOAD) Do not update flags

Family	N	Z	C	V	Default AF	Meaning
ADD	*	*	*	*	[A]	C is carry out; V is signed over-flow.
ADC	*	*	*	*	[A]	Adds incoming C; C is carry out.
SUB	*	*	*	*	[A]	C is set when a borrow occurs.
SBC	*	*	*	*	[A]	Subtracts incoming C as borrow input; C is set on outgoing borrow.
CMP	*	*	*	*	[A]	Same flags as SUB; result is discarded.
AND	*	*	0	0	[A]	C and V are cleared.
ORR	*	*	0	0	[A]	C and V are cleared.
XOR	*	*	0	0	[A]	C and V are cleared.
TSA	*	*	0	0	[A]	Same flags as AND; result is discarded.
TSX	*	*	0	0	[A]	Same flags as XOR; result is discarded.
LOAD	-	-	-	-	[N]	Status flags are preserved.

Table 13.3: ALU Status Flag Effects

See Table 11.2 for the flag-effect symbol definitions. The table shows the effect when the instruction uses its default status update source. If [N] is encoded, all status flags are preserved. If [A] is encoded, flags are updated from the ALU result according to the instruction family. If [S] is encoded, flags are updated from the shifter result instead of the ALU result; see Chapter 9 for the shifter flag rules. The immediate format has no shift fields, so [S] on an immediate-format instruction writes a constant zero into the flag byte: N, Z, C, and V are all cleared. Use [S] only with short-immediate and register forms. Public **LOAD** forms do not support status update override syntax and always preserve the status flags; an encoding of the **LOAD** opcode with a non-None AF field, which is not reachable from public assembly syntax, leaves all four status flags architecturally undefined ($\cup \cup \cup \cup$).

13.5.1 Flag Computation Formulas

Table 13.3 states which flags each family updates; this section states exactly how, as a Boolean formula in terms of the sign bit (bit 15) of the destination operand (D_m), the source operand (S_m), and the result (R_m). $\cdot$ is Boolean AND, $+$ is Boolean OR, and $\bar{X}$ is the complement of X .

Family	C	V
ADD, ADC	Carry out of bit 15 of the full addition	$D_m \cdot S_m \cdot \bar{R_m} + \bar{D_m} \cdot \bar{S_m} \cdot R_m$
SUB, SBC, CMP	$\bar{D_m} \cdot S_m + S_m \cdot R_m + R_m \cdot \bar{D_m}$	$D_m \cdot \bar{S_m} \cdot \bar{R_m} + \bar{D_m} \cdot S_m \cdot R_m$
AND, ORR, XOR, TSA, TSX	0 (always cleared)	0 (always cleared)

Table 13.4: Carry and Overflow Flag Formulas

N and Z are the same for every family: $N = R_m$ (the result's own sign bit), and Z is set when the full 16-bit result is zero. **ADC** and **SBC** use the same formulas as **ADD** and **SUB** respectively – the incoming carry/borrow is folded into the addition or subtraction before R_m (and the carry/borrow

out) is evaluated, so no separate formula is needed for them. **CMP/TSA/TSX** compute the same flags as the corresponding **SUB/AND/XOR** family and simply discard the result.

13.5.2 Manual Flag Update Control

ALU instructions except **LOAD** support manual control over how status flags are updated using the status override syntax. See Section 9.12 in Chapter 9 for complete details.

- **INSTR[A]** – Update flags from ALU result (default behavior)
- **INSTR[S]** – Update flags from shift result instead of ALU
- **INSTR[N]** – Do not update status flags
- AF encoding 11 (binary) is reserved and has no assembly syntax; the status register is left unchanged.

Example 13-1:

```

1 ; Update flags from shift operation
2 ADDI[S] r1, #0, LSL #3 ; Flags reflect (r1 << 3), not the ALU
3
4 ; Preserve flags during operation
5 CMPR r2, r3 ; Set comparison flags
6 ADDI[N] r4, #1 ; Increment without changing flags
7 BRAN|>= @branch_if_r2_gte_r3 ; Use flags from CMPR

```

SHIFT

Shift with Status Update Meta-Instruction

Applicability: SIRC-1, all revisions

Assembles to: ORRI[S] rD, #0, shift

Instruction Fields:

- **Register (bits 25–22):** rD, the destination and shifted source register; any of the 16 registers, 0x0–0xF.
- **Immediate (bits 21–14):** fixed at 0x00 by the meta-instruction.
- **Shift type and count:** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15, a register count above 15 is architecturally undefined.
- **AF (bits 5–4):** fixed at 10 (Shift) by the meta-instruction.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 SHIFT rD, shift ; rD = shifted rD; flags from shift
```

Operands: rD is both the destination and shifted source register. *shift* may use any standard immediate or register shift count supported by short-immediate ALU instructions.

Operation:

```
shifted = shift(rD)
rD = shifted OR 0
SR = flags_from_shift_result
```

Description:

Provides concise syntax for shifting a register value while taking status flags from the shifter result. It is not a separate CPU operation. The zero OR operation preserves the shifted source value, while [S] makes the status flags reflect the shift result instead of the OR result.

Flags: Updated from the shifter result by default.

Write-back: Inherits ORRI write-back to rD.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no shift result is written and status flags are preserved.

Timing: Same as ORRI: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (sr, ah, lh, ph, sh); see Chapter 6.

Example 13-2:

```
1 ; Shift r1 left by 3 and update flags
2 SHIFT r1, LSL #3 ; ORRI[S] r1, #0, LSL #3
3
4 ; Shift right and test result
5 SHIFT r3, LSR #2 ; r3 = r3 >> 2
6 BRAN |== @zero_result ; Branch if shifted result is zero
```

Notes:

- Immediate shift counts are 0–15; the assembler rejects a larger literal. A register shift count above 15 is architecturally undefined.
- All six shift types are supported, plus **NUL**: NUL, LSL, LSR, ASL, ASR, RTL, and RTR. **SHIFT** rD, NUL #0 sets the flags from the unshifted value of rD.

ADDI / ADDR

Add Immediate / Add Register

Applicability: SIRC-1, all revisions

Opcodes: 0x00 (Imm), 0x20 (Short Imm), 0x30 (Reg)

Instruction Fields:

- **Register:** destination *rD* (immediate and short-immediate forms) or *rD*, *rS1*, *rS2* (register form); any of the 16 registers, 0x0–0xF.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for LSL/LSR/ASL/ASR, and is taken modulo 16 for RTL/RTR) – see Chapter 9.
- **AF (bits 5–4):** 00 = None, 01 = Alu (default), 10 = Shift, 11 = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 ADDI rD, #imm16 ; rD = rD + imm16
2 ADDI rD, #imm8, shift ; rD = shift(rD) + imm8
3 ADDR rD, rS1, rS2 ; rD = rS1 + rS2
4 ADDR rD, rS1, rS2, shift ; rD = shift(rS1) + rS2
```

Operands: *rD* is the destination and first source in immediate forms. Register forms use *rS1* as the first source and *rS2* as the second source; assembler shorthand may omit *rS1*. Short-immediate and register forms may apply a shift to the first source operand.

Operation:

```
result = operand1 + operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = signed_overflow
```

Description:

Adds two values and stores the result in the destination register. In short immediate format with shift, the destination register is shifted before adding the immediate. The carry flag is set if an unsigned overflow occurs (carry out of bit 15). The overflow flag is set if a signed overflow occurs (the sign of the result is incorrect for the operation).

Flags: N Z C V

Write-back: If the condition is true, writes the 16-bit result to *rD*.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (*sr*, *ah*, *lh*, *ph*, *sh*); see Chapter 6.

Example 13-3:

```
1 ; Simple addition
2 ADDI r1, #100 ; r1 = r1 + 100
3
4 ; Add two registers
5 ADDR r3, r1, r2 ; r3 = r1 + r2
6
7 ; Scaled addition (multiply by 3)
8 ADDR r4, r5, r5, LSL #1 ; r4 = r5 + (r5 * 2) = r5 * 3
9
10 ; Build large constant
11 ADDI r1, #1, LSL #12 ; r1 = (r1 << 12) + 1
```

Notes:

- For multi-precision arithmetic, use **ADC** to propagate carry
- Adding zero (**ADDI** r1, #0) can be used to move values or test flags
- Shift operations allow efficient constant multiplication

ADCI / ADCR

Add with Carry

Applicability: SIRC-1, all revisions

Opcodes: `0x01` (Imm), `0x21` (Short Imm), `0x31` (Reg)

Instruction Fields:

- **Register:** destination `rD` (immediate and short-immediate forms) or `rD`, `rS1`, `rS2` (register form); any of the 16 registers, `0x0–0xF`.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for LSL/LSR/ASL/ASR, and is taken modulo 16 for RTL/RTR) – see Chapter 9.
- **AF (bits 5–4):** 00 = None, 01 = Alu (default), 10 = Shift, 11 = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```

1 ADCI rD, #imm16 ; rD = rD + imm16 + C
2 ADCI rD, #imm8, shift ; rD = shift(rD) + imm8 + C
3 ADCR rD, rS1, rS2 ; rD = rS1 + rS2 + C
4 ADCR rD, rS1, rS2, shift ; rD = shift(rS1) + rS2 + C

```

Operands: Same operand forms as **ADD**. The incoming `C` flag is read as an additional least-significant carry input.

Operation:

```

result = operand1 + operand2 + SR.C
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = signed_overflow

```

Description:

Adds two values plus the carry flag and stores the result. In short immediate format with shift, the destination register is shifted before adding the immediate. This instruction is used for multi-precision (32-bit, 48-bit, etc.) arithmetic to propagate the carry from lower-order additions to higher-order additions.

Flags: N Z C V

Write-back: If the condition is true, writes the 16-bit result to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (`sr`, `ah`, `lh`, `ph`, `sh`); see Chapter 6.

Example 13-4:

```
1 ; 32-bit addition (r1:r2 = r3:r4 + r5:r6)
2 ; Low words
3 ADDR r2, r4, r6 ; r2 = r4 + r6, set carry
4
5 ; High words (with carry propagation)
6 ADCR r1, r3, r5 ; r1 = r3 + r5 + carry
```

Notes:

- Always use after **ADD** for multi-word arithmetic
- The carry flag must be in the correct state before **ADC**
- Can be used to add 1 conditionally based on carry flag

SUBI / SUBR

Subtract Immediate / Subtract Register

Applicability: SIRC-1, all revisions

Opcodes: 0x02 (Imm), 0x22 (Short Imm), 0x32 (Reg)

Instruction Fields:

- **Register:** destination *rD* (immediate and short-immediate forms) or *rD*, *rS1*, *rS2* (register form); any of the 16 registers, 0x0–0xF.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for LSL/LSR/ASL/ASR, and is taken modulo 16 for RTL/RTR) – see Chapter 9.
- **AF (bits 5–4):** 00 = None, 01 = Alu (default), 10 = Shift, 11 = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 SUBI rD, #imm16 ; rD = rD - imm16
2 SUBI rD, #imm8, shift ; rD = shift(rD) - imm8
3 SUBR rD, rS1, rS2 ; rD = rS1 - rS2
4 SUBR rD, rS1, rS2, shift ; rD = shift(rS1) - rS2
```

Operands: Same operand forms as **ADD**. The second operand is subtracted from the first operand.

Operation:

```
result = operand1 - operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow
```

Description:

Subtracts the second operand from the first and stores the result. In short immediate format with shift, the destination register is shifted before subtracting the immediate. The carry flag is set if a borrow occurs ($\text{operand2} > \text{operand1}$ for unsigned). The overflow flag indicates signed overflow.

Flags: N Z C V

Write-back: If the condition is true, writes the 16-bit result to *rD*.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (*sr*, *ah*, *lh*, *ph*, *sh*); see Chapter 6.

Example 13-5:

```
1 ; Decrement by 1
2 SUBI r1, #1 ; r1 = r1 - 1
3
```

```
4 ; Subtract registers
5 SUBR r3, r1, r2 ; r3 = r1 - r2
6
7 ; Loop counter
8 SUBI r7, #1 ; Decrement counter
9 BRAN != @loop ; Branch if not zero
```

Notes:

- Carry is set when the subtraction requires a borrow
- For multi-precision subtraction, use **SBC**
- Subtracting a value from itself (**SUBR** r1, r1, r1) clears the register

SBCI / SBCR

Subtract with Carry

Applicability: SIRC-1, all revisions

Opcodes: 0x03 (Imm), 0x23 (Short Imm), 0x33 (Reg)

Instruction Fields:

- **Register:** destination *rD* (immediate and short-immediate forms) or *rD*, *rS1*, *rS2* (register form); any of the 16 registers, 0x0–0xF.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for LSL/LSR/ASL/ASR, and is taken modulo 16 for RTL/RTR) – see Chapter 9.
- **AF (bits 5–4):** 00 = None, 01 = Alu (default), 10 = Shift, 11 = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```

1 SBCI rD, #imm16 ; rD = rD - imm16 - C
2 SBCI rD, #imm8, shift ; rD = shift(rD) - imm8 - C
3 SBCR rD, rS1, rS2 ; rD = rS1 - rS2 - C
4 SBCR rD, rS1, rS2, shift ; rD = shift(rS1) - rS2 - C

```

Operands: Same operand forms as SUB. The incoming C flag is read as the borrow input.

Operation:

```

result = operand1 - operand2 - SR.C
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow

```

Description:

Subtracts the second operand and borrow from the first operand. In short immediate format with shift, the destination register is shifted before subtracting the immediate. Used for multi-precision subtraction to propagate borrows from lower-order to higher-order subtractions.

Flags: N Z C V

Write-back: If the condition is true, writes the 16-bit result to *rD*.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (*sr*, *ah*, *lh*, *ph*, *sh*); see Chapter 6.

Example 13-6:

```

1 ; 32-bit subtraction (r1:r2 = r3:r4 - r5:r6)
2 ; Low words
3 SUBR r2, r4, r6 ; r2 = r4 - r6, set/clear carry
4
5 ; High words (with borrow propagation)
6 SBCR r1, r3, r5 ; r1 = r3 - r5 - borrow

```

Notes:

- Always use after **SUB** for multi-word subtraction
- Carry is treated as the borrow input for multi-word subtraction

ANDI / ANDR

AND Immediate / AND Register

Applicability: SIRC-1, all revisions

Opcodes: 0x04 (Imm), 0x24 (Short Imm), 0x34 (Reg)

Instruction Fields:

- **Register:** destination *rD* (immediate and short-immediate forms) or *rD*, *rS1*, *rS2* (register form); any of the 16 registers, 0x0–0xF.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for LSL/LSR/ASL/ASR, and is taken modulo 16 for RTL/RTR) – see Chapter 9.
- **AF (bits 5–4):** 00 = None, 01 = Alu (default), 10 = Shift, 11 = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 ANDI rD, #imm16 ; rD = rD & imm16
2 ANDI rD, #imm8, shift ; rD = shift(rD) & imm8
3 ANDR rD, rS1, rS2 ; rD = rS1 & rS2
4 ANDR rD, rS1, rS2, shift ; rD = shift(rS1) & rS2
```

Operands: Same operand forms as **ADD**. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.

Operation:

```
result = operand1 AND operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:

Performs bitwise AND operation. Each bit in the result is 1 only if the corresponding bits in both operands are 1. In short immediate format with shift, the destination register is shifted before ANDing with the immediate. Commonly used for bit masking and clearing specific bits.

Flags: N Z (C and V cleared)

Write-back: If the condition is true, writes the 16-bit result to *rD*.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (*sr*, *ah*, *lh*, *ph*, *sh*); see Chapter 6.

Example 13-7:

```
1 ; Mask lower byte
2 ANDI r1, #0x00FF ; r1 = r1 & 0xFF
3
```

```
4 ; Clear bit 5
5 ANDI r2, #0xFFDF ; r2 = r2 & 0xFFDF (clear bit 5)
6
7 ; Check if two registers share any set bits
8 ANDR r3, r1, r2 ; r3 = r1 & r2
9 CMPI r3, #0
10 BRAN != @common_bits ; Branch if any bits in common
```

Notes:

- ANDing with 0xFFFF leaves value unchanged
- ANDing with 0x0000 clears the register
- Use **TSA** to test bits without modifying the register

ORRI / ORRR

OR Immediate / OR Register

Applicability: SIRC-1, all revisions

Opcodes: 0x05 (Imm), 0x25 (Short Imm), 0x35 (Reg)

Instruction Fields:

- **Register:** destination *rD* (immediate and short-immediate forms) or *rD*, *rS1*, *rS2* (register form); any of the 16 registers, 0x0–0xF.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for LSL/LSR/ASL/ASR, and is taken modulo 16 for RTL/RTR) – see Chapter 9.
- **AF (bits 5–4):** 00 = None, 01 = Alu (default), 10 = Shift, 11 = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 ORRI rD, #imm16 ; rD = rD | imm16
2 ORRI rD, #imm8, shift ; rD = shift(rD) | imm8
3 ORRR rD, rS1, rS2 ; rD = rS1 | rS2
4 ORRR rD, rS1, rS2, shift ; rD = shift(rS1) | rS2
```

Operands: Same operand forms as **AND**. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.

Operation:

```
result = operand1 OR operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:

Performs bitwise OR operation. Each bit in the result is 1 if either corresponding bit in the operands is 1. In short immediate format with shift, the destination register is shifted before ORing with the immediate. Commonly used for setting specific bits.

Flags: N Z (C and V cleared)

Write-back: If the condition is true, writes the 16-bit result to *rD*.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (*sr*, *ah*, *lh*, *ph*, *sh*); see Chapter 6.

Example 13-8:

```
1 ; Set bit 7
2 ORRI r1, #0x0080 ; r1 = r1 | 0x80
3
```

```
4 ; Set multiple bits
5 ORRI r2, #0xF000 ; Set high nibble
6
7 ; Combine two registers
8 ORRR r3, r1, r2 ; r3 = r1 | r2
9
10 ; Set bit N (variable)
11 LOAD r4, #1 ; r4 = 1
12 SHFT r4, LSL r5 ; r4 = 1 << r5
13 ORRR r1, r4 ; r1 |= (1 << r5)
```

Notes:

- ORing with 0x0000 leaves value unchanged
- ORing with 0xFFFF sets all bits
- Use for enabling flag bits or combining bit fields

XORI / XORR

XOR Immediate / XOR Register

Applicability: SIRC-1, all revisions

Opcodes: `0x06` (Imm), `0x26` (Short Imm), `0x36` (Reg)

Instruction Fields:

- **Register:** destination `rD` (immediate and short-immediate forms) or `rD`, `rS1`, `rS2` (register form); any of the 16 registers, `0x0–0xF`.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for LSL/LSR/ASL/ASR, and is taken modulo 16 for RTL/RTR) – see Chapter 9.
- **AF (bits 5–4):** 00 = None, 01 = Alu (default), 10 = Shift, 11 = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 XORI rD, #imm16 ; rD = rD ^ imm16
2 XORI rD, #imm8, shift ; rD = shift(rD) ^ imm8
3 XORR rD, rS1, rS2 ; rD = rS1 ^ rS2
4 XORR rD, rS1, rS2, shift ; rD = shift(rS1) ^ rS2
```

Operands: Same operand forms as **AND**. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.

Operation:

```
result = operand1 XOR operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:

Performs bitwise XOR (exclusive OR) operation. Each bit in the result is 1 if the corresponding bits in the operands are different. In short immediate format with shift, the destination register is shifted before XORing with the immediate. Used for toggling bits, bitwise comparison, and simple encryption.

Flags: N Z (C and V cleared)

Write-back: If the condition is true, writes the 16-bit result to `rD`.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (`sr`, `ah`, `lh`, `ph`, `sh`); see Chapter 6.

Example 13-9:

```
1 ; Toggle bit 3
2 XORI r1, #0x0008 ; r1 = r1 ^ 0x08
3
4 ; Invert all bits
5 XORI r2, #0xFFFF ; r2 = ~r2
6
7 ; Check if registers are equal
8 XORR r3, r1, r2 ; r3 = r1 ^ r2
9 ; If r3 == 0, then r1 == r2
10
11 ; Swap using XOR (no temporary)
12 XORR r1, r1, r2 ; r1 = r1 ^ r2
13 XORR r2, r1, r2 ; r2 = r1 ^ r2 = (r1 ^ r2) ^ r2 = r1
14 XORR r1, r1, r2 ; r1 = (r1 ^ r2) ^ r1 = r2
```

Notes:

- XORing with 0x0000 leaves value unchanged
- XORing with 0xFFFF inverts all bits (bitwise NOT)
- XORing a value with itself always gives 0
- Use **TSX** to check for differences without modifying register

LOAD

Load/Move

Applicability: SIRC-1, all revisions

Opcodes: 0x07 (Imm), 0x37 (Reg). Opcode 0x27 is undocumented.

Instruction Fields:

- **Register:** destination *rD* (immediate form) or *rD*, *rS* (register form); any of the 16 registers, 0x0–0xF.
- **Immediate:** 16-bit value (immediate form, bits 21–6). The register form has no immediate field.
- **AF (bits 5–4):** the public assembly syntax always encodes 00 (None), which preserves the status flags. An encoding of this opcode with a different AF value, not reachable from assembly syntax, leaves all four status flags architecturally undefined (U U U U).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 LOAD rD, #imm16 ; rD = imm16
2 LOAD rD, rS ; rD = rS
```

Operands: Immediate form writes a 16-bit immediate value to *rD*. Register form copies *rS* to *rD*. No short-immediate, memory, or shift forms are public assembly syntax; the register-to-register form does not accept a shift definition (use **SHIFT**). The indirect memory forms of **LOAD** are documented in Chapter 14.

Operation:

```
destination = operand
; Flags NOT updated
```

Description:

The ALU passes the second operand straight through to the destination; the first operand is ignored. This is the primary way to load constants or move data between registers. Opcode 0x27 has no public assembly syntax and is undocumented.

Flags: Preserved.

Write-back: If the condition is true, writes the loaded or copied value to *rD*.

Exceptions: Does not access memory in these forms and raises no instruction-specific faults.

Condition field: If the condition is false, no result is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (*sr*, *ah*, *lh*, *ph*, *sh*); see Chapter 6.

Example 13-10:

```
1 ; Load constant
2 LOAD r1, #1234 ; r1 = 1234
3
4 ; Copy register
5 LOAD r2, r1 ; r2 = r1
6
7 ; Load with arithmetic
8 LOAD r3, #1
9 ADDI r3, #0, LSL #8 ; r3 = (r3 << 8) + 0 = 256
10
11 ; Zero a register
```

```
12 LOAD r4, #0 ; r4 = 0
13
14 ; Load large constant (combine)
15 LOAD r5, #0x12
16 ADDI r5, #0, LSL #8 ; r5 = (r5 << 8) + 0 = 0x1200
17 ORRI r5, #0x34 ; r5 = 0x1234
```

Notes:

- Does not update flags, unlike **ADD**
- Most efficient way to move data between registers
- For 16-bit constants, use immediate form
- To build shifted or composite constants, combine **LOAD** with ALU instructions such as **ADDI** or **ORRI**

CMPI / CMPR

Compare Immediate / Compare Register

Applicability: SIRC-1, all revisions

Opcodes: 0x0A (Imm), 0x2A (Short Imm), 0x3A (Reg)

Instruction Fields:

- **Register:** source *rS* (immediate and short-immediate forms) or *rS1*, *rS2* (register form); any of the 16 registers, 0x0–0xF. No destination register is written.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** NUL, LSL, LSR, ASL, ASR, RTL, RTR; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for LSL/LSR/ASL/ASR, and is taken modulo 16 for RTL/RTR) – see Chapter 9.
- **AF (bits 5–4):** 00 = None, 01 = Alu (default), 10 = Shift, 11 = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 CMPI rS, #imm16 ; Compare rS with imm16
2 CMPI rS, #imm8, shift ; Compare shift(rS) with imm8
3 CMPR rS1, rS2 ; Compare rS1 with rS2
4 CMPR rS1, rS2, shift ; Compare shift(rS1) with rS2
```

Operands: Same source operand forms as **SUB**. The first operand is compared with the second operand; no destination register is written.

Operation:

```
result = operand1 - operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow
```

Description:

Performs subtraction but discards the result, only updating the status flags. In short immediate format with shift, the source register is shifted before comparing with the immediate. This is the primary instruction for implementing conditional execution and branches. The flags can be tested with condition codes to determine the relationship between the operands.

Flags: N Z C V

Write-back: None. The subtraction result is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (*sr*, *ah*, *lh*, *ph*, *sh*); see Chapter 6. The privilege check is made on the register field even though this instruction writes no register, so **CMPI** *sr*, #0 in protected mode raises a privilege-violation fault.

Example 13-11:

```
1 ; Test if r1 is zero
2 CMPI r1, #0
3 BRAN|= @is_zero ; Branch if r1 == 0
4
5 ; Test if r1 > r2 (unsigned)
6 CMPR r1, r2
7 BRAN|HI @r1_greater
8
9 ; Test if r1 >= r2 (signed)
10 CMPR r1, r2
11 BRAN|>= @r1_greater_or_equal
12
13 ; Range check: if (r1 >= 10 && r1 < 20)
14 CMPI r1, #10
15 BRAN|<< @out_of_range ; Branch if r1 < 10
16 CMPI r1, #20
17 BRAN|>= @out_of_range ; Branch if r1 >= 20
18 ; r1 is in range [10, 20)
```

Notes:

- Does not modify any register, only flags
- Use with conditional instructions or branches
- Remember: **CMPI** A, B computes A - B
- For signed comparisons, use >=, <, >, <=
- For unsigned comparisons, use HI, LO, CS, CC

TSAI / TSAR

Test AND Immediate / Test AND Register

Applicability: SIRC-1, all revisions

Opcodes: `0x0C` (Imm), `0x2C` (Short Imm), `0x3C` (Reg)

Instruction Fields:

- **Register:** source `rS` (immediate and short-immediate forms) or `rS1`, `rS2` (register form); any of the 16 registers, `0x0–0xF`. No destination register is written.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for `LSL`/`LSR`/`ASL`/`ASR`, and is taken modulo 16 for `RTL`/`RTR`) – see Chapter 9.
- **AF (bits 5–4):** `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 TSAI rS, #imm16 ; Test rS & imm16
2 TSAI rS, #imm8, shift ; Test shift(rS) & imm8
3 TSAR rS1, rS2 ; Test rS1 & rS2
4 TSAR rS1, rS2, shift ; Test shift(rS1) & rS2
```

Operands: Same source operand forms as **AND**. The result is used only for status flag updates; no destination register is written.

Operation:

```
result = operand1 AND operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:

Performs bitwise AND but discards the result, only updating flags. In short immediate format with shift, the source register is shifted before ANDing with the immediate. Used to test if specific bits are set without modifying the register. The zero flag indicates whether any tested bits were set.

Flags: `N` `Z` (`C` and `V` cleared)

Write-back: None. The AND result is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (`sr`, `ah`, `lh`, `ph`, `sh`); see Chapter 6. The privilege check is made on the register field even though this instruction writes no register, so `TSAI sr, #0` in protected mode raises a privilege-violation fault.

Example 13-12:

```
1 ; Test if bit 5 is set
2 TSAI r1, #0x0020 ; Test r1 & 0x20
3 BRAN != @bit5_set ; Branch if bit 5 was set
4
5 ; Test if any high byte bits are set
6 TSAI r2, #0xFF00
7 BRAN != @has_upper_bits
8
9 ; Test if r1 and r2 have any bits in common
10 TSAR r1, r2
11 BRAN == @no_common_bits ; Branch if no bits in common
12
13 ; Test multiple bits
14 TSAI r3, #0xF000 ; Test high nibble
15 BRAN == @upper_clear ; Branch if all tested bits clear
```

Notes:

- Does not modify any register, only flags
- Z=1 means all tested bits were 0
- Z=0 means at least one tested bit was 1
- Use for bit testing without side effects
- More efficient than AND + CMP for simple tests

TSXI / TSXR

Test XOR Immediate / Test XOR Register

Applicability: SIRC-1, all revisions

Opcodes: `0x0E` (Imm), `0x2E` (Short Imm), `0x3E` (Reg)

Instruction Fields:

- **Register:** source `rS` (immediate and short-immediate forms) or `rS1`, `rS2` (register form); any of the 16 registers, `0x0–0xF`. No destination register is written.
- **Immediate:** 16-bit value (immediate form, bits 21–6) or 8-bit value zero-extended to 16 bits (short-immediate form, bits 21–14).
- **Shift type and count (short-immediate and register forms only):** `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL`, `RTR`; an immediate count is 0–15; a register count is used in full (16 or more clamps to a full shift for `LSL/LSR/ASL/ASR`, and is taken modulo 16 for `RTL/RTR`) – see Chapter 9.
- **AF (bits 5–4):** `00` = None, `01` = Alu (default), `10` = Shift, `11` = Reserved (status register unchanged).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 TSXI rS, #imm16 ; Test rS ^ imm16
2 TSXI rS, #imm8, shift ; Test shift(rS) ^ imm8
3 TSXR rS1, rS2 ; Test rS1 ^ rS2
4 TSXR rS1, rS2, shift ; Test shift(rS1) ^ rS2
```

Operands: Same source operand forms as **XOR**. The result is used only for status flag updates; no destination register is written.

Operation:

```
result = operand1 XOR operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:

Performs bitwise XOR but discards the result, only updating flags. In short immediate format with shift, the source register is shifted before XORing with the immediate. Used to test if bits match a specific pattern or to compare values for equality. `Z=1` indicates the values are identical.

Flags: `N Z` (`C` and `V` cleared)

Write-back: None. The XOR result is discarded after status flags are updated.

Exceptions: Does not access memory and raises no instruction-specific faults.

Condition field: If the condition is false, no status flags are updated.

Timing: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, except when the register field names a privileged register (`sr`, `ah`, `lh`, `ph`, `sh`); see Chapter 6. The privilege check is made on the register field even though this instruction writes no register, so `TSXI sr, #0` in protected mode raises a privilege-violation fault.

Example 13-13:

```
1 ; Test if r1 equals specific value
2 TSXI r1, #0x1234 ; Test r1 ^ 0x1234
3 BRAN== @is_0x1234 ; Branch if r1 == 0x1234
4
5 ; Test if two registers are equal
6 TSXR r1, r2 ; Test r1 ^ r2
7 BRAN== @registers_equal ; Branch if r1 == r2
8
9 ; Test if bit pattern matches
10 ANDI r3, #0x00FF ; Mask to low byte
11 TSXI r3, #0x00AA ; Test if low byte == 0xAA
12 BRAN== @pattern_match
```

Notes:

- Does not modify any register, only flags
- Z=1 means values are identical (all bits match)
- Can be used as equality test (alternative to CMP for == only)
- More efficient than XOR + CMP for simple equality tests

13.6 Summary

The ALU instructions form the computational core of the SIRCIS instruction set. Key points:

- All ALU operations work only on registers (load/store architecture)
- Three format variants provide flexibility (immediate, short+shift, register)
- Test variants (CMP, TSA, TSX) allow non-destructive testing
- Shift operations can be combined with ALU ops for powerful single-instruction operations
- Consistent flag behavior makes conditional execution predictable

CHAPTER 14

Memory Access Instructions

14.1 Overview

Memory access instructions transfer data between registers and memory. Following the load/store architecture, these are the only instructions that can access memory—all arithmetic and logical operations must work on registers.

The SIRC-1 provides two primary memory operations:

- **LOAD** – Read from memory into a register
- **STOR** – Write from a register to memory

LOAD and **STOR** each support four memory addressing forms. Both support indirect addressing with an immediate offset and indirect addressing with a register offset; **LOAD** additionally supports post-increment and **STOR** additionally supports pre-decrement:

- Indirect with immediate offset: `(#offset, addr)`
- Indirect with register offset: `(r0, addr)`
- Post-increment (**LOAD** only): `(#offset, addr)+` or `(r0, addr)+`
- Pre-decrement (**STOR** only): `-(#offset, addr)` or `-(r0, addr)`

Important: **STOR**'s register-offset forms support an optional shift suffix, applied to the source register value before it is written to memory. **LOAD** does not support a shift suffix on any form: the offset register `r0` is always used unshifted. See Section 14.7 for details. Shifts are also **not** supported when loading immediate values or copying register values directly.

14.2 Effective Address Calculation

For all memory operations, the effective address (EA) is calculated as:

$$EA = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{Displacement}) \quad (14.1)$$

Where:

- Address Register is one of: `l`, `a`, `s`, or `p`
- Displacement is either a 16-bit signed immediate value or the unshifted contents of the offset register `r0`

The high word supplies the segment and is not changed by the displacement; only the low word participates in the addition, modulo 2^{16} .

14.3 Legal Forms

Instruction	Syntax	Encoding	Notes
LOAD	LOAD rD, (#offset, addr)	0x14	Immediate offset
LOAD	LOAD rD, (rO, addr)	0x15	Register offset; no shift suffix
LOAD	LOAD rD, (#offset, addr)+	0x16	Immediate offset; post-increment
LOAD	LOAD rD, (rO, addr)+	0x17	Register offset; no shift suffix; post-increment
STOR	STOR (#offset, addr), rS	0x10	Immediate offset
STOR	STOR (rO, addr), rS[, shift]	0x11	Register offset; optional source shift
STOR	STOR -(#offset, addr), rS	0x12	Immediate offset; pre-decrement
STOR	STOR -(rO, addr), rS[, shift]	0x13	Register offset; optional shift; pre-decrement

Table 14.1: Legal Memory Instruction Forms

Zero displacement: Immediate-displacement memory forms may omit #0. Thus `LOAD rD, (addr)`, `LOAD rD, (addr)+`, `STOR (addr), rS`, and `STOR -(addr), rS` assemble as the matching #0 forms. This shorthand does not add new addressing modes: **LOAD** has no pre-decrement form and **STOR** has no post-increment form.

Aliased register writes: In post-increment **LOAD** forms the destination register must not be either half of the auto-updated address-register pair; in pre-decrement **STOR** forms the source register must not be either half of the auto-updated pair. The result of such an encoding is architecturally undefined and the assembler rejects it.

14.4 Common Memory Semantics

Topic	Definition
Condition false	The instruction is skipped and has no side effects. Registers, memory, address registers, and status flags are preserved.
Address calculation	Immediate-offset forms add the 16-bit signed displacement to the selected address register low word. Register-offset forms add the unshifted <code>r0</code> to the selected address register low word; <code>r0</code> is never shifted, even on STOR 's register-offset forms, which shift the value being stored instead. The selected address register high word supplies the segment and is unchanged.
Post-increment LOAD	The memory address is calculated from the original address register value. The address register low word is incremented by one word in write-back after the memory access.
Pre-decrement STOR	The memory address uses the selected address register low word decremented by one word, plus the offset. The address register low word is decremented by one word in write-back after the memory access.
Status flags	Memory instructions preserve all status flags. Shifts used by memory instructions cannot update the status register.
Timing	Memory instructions complete in 6 cycles when the bus access is acknowledged without wait states. Bus wait states extend the memory-access phase.
Exceptions	Memory instructions can raise data bus, data bus-protection, segment-overflow, and privilege-violation faults; see the notes below.
Fault side effects	Destination register writes and address-register auto-updates occur in write-back after the memory-access phase, so a fault that aborts execution before write-back leaves them undone. Data bus, data bus-protection, segment-overflow, and privilege-violation faults are all retryable: the saved return address is the address of the faulting instruction, so returning from the fault handler with RETE retries the same memory access (see Chapter 6).

Table 14.2: Common Memory Instruction Semantics

Memory instructions can raise data bus and data bus-protection faults during the memory-access phase. The 16-bit displacement is signed two's-complement; effective-address calculation can raise a segment-overflow fault when `SR.A` (Trap on Address Overflow) is set and the low-word address calculation wraps. This segment-overflow fault is retryable: the saved return address is the address of the faulting instruction (see Chapter 6). A memory instruction raises a privilege-violation fault in protected mode when its register field names a privileged register (`sr`, `ah`, `lh`, `ph`, or `sh`); this applies to the **LOAD** destination register and to the **STOR** source register, even though **STOR**

only reads it. Post-increment and pre-decrement address-register auto-update writes only the low word of the pair, so it never changes the high word and never raises a privilege-violation fault. Documented memory forms do not raise invalid-opcode faults. Instruction fetch can still raise the normal fetch faults before a memory instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits. Processing-unit encodings outside the documented memory forms are reserved or architecturally undefined; they are not required to raise an invalid-opcode fault.

14.5 Status Flag Effects

Memory instructions do not update status flags. All four condition flags are preserved regardless of addressing form, shift result, or auto-update side effect. This applies only to the memory-access **LOAD** forms (**0x14**–**0x17**) and **STOR** (**0x10**–**0x13**); see Chapter 13 for the flag behavior of the **LOAD-immediate** (**0x07**) and **LOAD-register** (**0x37**) forms. The **AF** field (bits 5–4) is not a status-update source for these opcodes; it selects the address-register pair used to form the effective address.

Instruction	N	Z	C	V	Meaning
LOAD (0x14 – 0x17)	-	-	-	-	Flags are preserved.
STOR (0x10 – 0x13)	-	-	-	-	Flags are preserved.

Table 14.3: Memory Instruction Status Flag Effects

See Table 11.2 for the flag-effect symbol definitions.

LOAD

Load from Memory

Applicability: SIRC-1, all revisions

Opcodes: **0x14** (Indirect Imm), **0x15** (Indirect Reg), **0x16** (Post-Inc Imm), **0x17** (Post-Inc Reg)

Instruction Fields:

- **Register (bits 25–22, Immediate format, **0x14/0x16**):** Destination register *rD*.
- **R1 (bits 25–22, Register format, **0x15/0x17**):** Destination register *rD*.
- **R2 (bits 21–18, Register format, **0x15/0x17**):** Unused, fixed at 0. The assembler does not accept a shift suffix on any **LOAD** form: **LOAD** has no field that shifts the offset register or the loaded value. Use **SHIFT** on *rO* before the **LOAD** if a scaled offset is needed.
- **R3 (bits 17–14, Register format, **0x15/0x17**):** Offset register *rO*, used unshifted.
- **Immediate (bits 21–6, Immediate format, **0x14/0x16**):** 16-bit signed displacement.
- **AF (bits 5–4):** Address-register pair – 00 = l, 01 = a, 10 = s, 11 = p. Not a status-update source for these opcodes.
- **Condition field (bits 3–0):** All 16 condition-field encodings are legal; see **Condition field** below.

Syntax:

```
1 LOAD rD, (#offset, addr) ; rD = memory[addr + offset]
2 LOAD rD, (addr) ; equivalent to LOAD rD, (#0, addr)
3 LOAD rD, (rO, addr) ; rD = memory[addr + rO]
4 LOAD rD, (#offset, addr) + ; rD = memory[addr + offset]; addr += 1
5 LOAD rD, (addr) + ; equivalent to LOAD rD, (#0, addr) +
6 LOAD rD, (rO, addr) + ; rD = memory[addr + rO]; addr += 1
```

Operands: *rD* is the destination general-purpose register; a privileged register (*sr*, *ah*, *lh*, *ph*, or *sh*) raises a privilege-violation fault in protected mode. *addr* is the address-register pair that supplies the segment and base low word. Immediate-offset forms use a 16-bit signed displacement. Register-offset forms use *rO* as the displacement register, always unshifted; **LOAD** does not accept a shift suffix.

Operation:

```
if post_increment then
    effective_address = addr + offset
    destination = memory[effective_address]
    addr.low = addr.low + 1
else
    effective_address = addr + offset
    destination = memory[effective_address]
endif
; Flags are preserved
```

Description:

Loads a 16-bit value from memory into the destination register. Post-increment forms update the address register after the load, useful for array traversal.

Flags: Preserved.

Write-back: If the condition is true, writes the loaded value to `rD`. Post-increment forms also write the incremented address-register low word.

Exceptions: May raise data bus or data bus-protection faults during the memory read. May raise a segment-overflow fault during effective-address calculation when `SR.A` is set; this fault is not retryable (see Chapter 6). May raise a privilege-violation fault in protected mode if `rD` names a privileged register.

Condition field: If the condition is false, no memory read occurs, no destination register is written, no address register is incremented, and status flags are preserved.

Timing: 6 cycles when the memory read is acknowledged without wait states; wait states extend the memory-access phase.

Privilege: Available in protected mode and supervisor mode. Protected-mode data-memory access is subject to bus protection. A privilege-violation fault is raised in protected mode if `rD` names a privileged register (`sr`, `ah`, `lh`, `ph`, or `sh`).

Example 14-1:

```
1 ; Load from fixed offset
2 LOAD r1, (#4, a) ; r1 = memory[a + 4]
3
4 ; Load using variable offset
5 LOAD r2, (r3, a) ; r2 = memory[a + r3]
6
7 ; Array iteration
8 LOAD r4, (#0, a)+ ; r4 = *a; a += 1
9
10 ; Stack pop
11 LOAD r5, (#0, s)+ ; r5 = *s; s += 1
12
13 ; LOAD has no shift suffix: scale the offset register first with SHFT
14 SHFT r2, LSL #2 ; r2 = r2 << 2
15 LOAD r3, (r2, a) ; r3 = memory[a + r2]
```

STOR

Store to Memory

Applicability: SIRC-1, all revisions

Opcodes: **0x10** (Indirect Imm), **0x11** (Indirect Reg), **0x12** (Pre-Dec Imm), **0x13** (Pre-Dec Reg)

Instruction Fields:

- **Register (bits 25–22, Immediate format, 0x10/0x12):** Source register *rS*.
- **R1 (bits 25–22, Register format, 0x11/0x13):** Unused, encoded as 0.
- **R2 (bits 21–18, Register format, 0x11/0x13):** Source register *rS*.
- **R3 (bits 17–14, Register format, 0x11/0x13):** Offset register *rO*.
- **Immediate (bits 21–6, Immediate format, 0x10/0x12):** 16-bit signed displacement.
- **Shift Type (bits 12–10, Register format, 0x11/0x13):** 000 none, 001 LSL, 010 LSR, 011 ASL, 100 ASR, 101 RTL, 110 RTR, 111 reserved. Applies to *rS* before the value is written to memory.
- **Shift Amount (bits 9–6, Register format, 0x11/0x13):** Immediate count 0–15, or (Shift Operand bit set) a register holding the count; a register count above 15 is architecturally undefined.
- **AF (bits 5–4):** Address-register pair – 00 = l, 01 = a, 10 = s, 11 = p. Not a status-update source for these opcodes.
- **Condition field (bits 3–0):** All 16 condition-field encodings are legal; see **Condition field** below.

Syntax:

```

1 STOR (#offset, addr), rS ; memory[addr + offset] = rS
2 STOR (addr), rS ; equivalent to STOR (#0, addr), rS
3 STOR (rO, addr), rS ; memory[addr + rO] = rS
4 STOR -(#offset, addr), rS ; addr -= 1; memory[addr + offset] = rS
5 STOR -(addr), rS ; equivalent to STOR -(#0, addr), rS
6 STOR -(rO, addr), rS ; addr -= 1; memory[addr + rO] = rS
7
8 ; With optional shift on the source value (register-offset memory forms
   only)
9 STOR (rO, addr), rS, shift ; Shift rS, then store
10 STOR -(rO, addr), rS, shift ; Decrement, shift rS, then store

```

Operands: *rS* is the source general-purpose register; a privileged register (*sr*, *ah*, *lh*, *ph*, or *sh*) raises a privilege-violation fault in protected mode, even though **STOR** only reads it. *addr* is the address-register pair that supplies the segment and base low word. Immediate-offset forms use a 16-bit signed displacement. Register-offset forms use *rO* as the displacement register and may apply a shift to the source value before storing it.

Operation:

```

if pre_decrement then
    decremented_low = addr.low - 1
    effective_address = (addr.high : decremented_low) + offset
    memory[effective_address] = shift_if_encoded(source)

```

```
    addr.low = decremented_low
else
    effective_address = addr + offset
    memory[effective_address] = shift_if_encoded(source)
endif
; Source register and flags are preserved
```

Description:

Stores a 16-bit value from a register into memory. Pre-decrement forms use the decremented address for the store and write the decremented address register value back after the memory access, useful for stack push operations.

Flags: Preserved.

Write-back: If the condition is true, writes the source value to memory. Pre-decrement forms also write the decremented address-register low word.

Exceptions: May raise data bus or data bus-protection faults during the memory write. May raise a segment-overflow fault during effective-address calculation when `SR.A` is set; this fault is not retryable (see Chapter 6). May raise a privilege-violation fault in protected mode if `rS` names a privileged register.

Condition field: If the condition is false, no memory write occurs, no address register is decremented, and status flags are preserved.

Timing: 6 cycles when the memory write is acknowledged without wait states; wait states extend the memory-access phase.

Privilege: Available in protected mode and supervisor mode. Protected-mode data-memory access is subject to bus protection. A privilege-violation fault is raised in protected mode if `rS` names a privileged register (`sr`, `ah`, `lh`, `ph`, or `sh`), even though **STOR** only reads it.

Example 14-2:

```
1 ; Store to fixed offset
2 STOR (#8, a), r1 ; memory[a + 8] = r1
3
4 ; Store using variable offset
5 STOR (r3, a), r2 ; memory[a + r3] = r2
6
7 ; Store with shift (data shifted before storing)
8 STOR (r2, a), r1, LSL #1 ; memory[a + r2] = r1 << 1
9 STOR (r3, a), r1, LSR #3 ; memory[a + r3] = r1 >> 3
10
11 ; Stack push
12 STOR -(#0, s), r4 ; s -= 1; memory[s] = r4
```

14.6 Common Memory Access Patterns

14.6.1 Structure Field Access

```
1 ; struct { int16_t x, y, z; } point;
2 ; 'a' points to point
3 LOAD r1, (#0, a) ; r1 = point.x
4 LOAD r2, (#1, a) ; r2 = point.y
5 LOAD r3, (#2, a) ; r3 = point.z
```

14.6.2 Array Iteration

```
1 ; Process array of 16-bit values
2 ; 'a' = array base, r7 = count
```

```

3 :loop
4     LOAD r1, (#0, a)+ ; Load element, advance
5     ; ... process r1 ...
6     SUBI r7, #1
7     BRAN != @loop

```

14.6.3 Stack Operations

```

1 ; Push registers
2 STOR -(#0, s), r1
3 STOR -(#0, s), r2
4 STOR -(#0, s), r3
5
6 ; Pop registers (reverse order)
7 LOAD r3, (#0, s)+
8 LOAD r2, (#0, s)+
9 LOAD r1, (#0, s)+

```

14.7 Shift Operations with Memory Instructions

Only **STOR**'s register-offset forms support an optional shift suffix, applied to the source register value before it is written to memory. **LOAD** does not support a shift suffix on any form – the offset register *r0* is always used unshifted, and there is no field that shifts the loaded value. This asymmetry is architectural, not an oversight: use **SHFT** on the offset register before a **LOAD** if a scaled index is needed.

14.7.1 STOR with Shift

When storing to memory, the source register value is shifted **before** being written to memory. The original register value is **not** modified:

```

1 STOR (r2, a), r1, LSL #1 ; memory[a + r2] = r1 << 1 (r1 unchanged)
2 STOR (r2, a), r1, ASL #2 ; memory[a + r2] = r1 << 2 (r1 unchanged)
3 STOR (r4, a), r1, ASR #4 ; memory[a + r4] = r1 >> 4 (r1 unchanged)

```

14.7.2 Supported Shift Types

All standard shift types are supported:

- **LSL** – Logical Shift Left (fill with zeros)
- **LSR** – Logical Shift Right (fill with zeros)
- **ASL** – Arithmetic Shift Left (same data result as LSL; sets V on signed overflow)
- **ASR** – Arithmetic Shift Right (sign-extend)
- **RTL** – Rotate Left within the operand
- **RTR** – Rotate Right within the operand

14.7.3 Shift Limitations

Important: The shift suffix is **only** supported on **STOR**'s register-offset forms. The following do **not** support a shift suffix:

- Any **LOAD** form, register-offset included – **LOAD** *r1*, (*r2*, *a*) – No shift suffix exists

- Loading immediate values: `LOAD r1, #0x1234` – No shift allowed
- Copying registers: `LOAD r1, r2` – No shift allowed
- Immediate-offset memory load: `LOAD r1, (#0, a)` – No shift allowed
- Immediate-offset memory store: `STOR (#0, a), r1` – No shift allowed

For any of these, load or store the unshifted value and use an ALU instruction or **SHFT** for the shift operation.

14.7.4 Use Cases

Scaled Indexing

LOAD has no shift suffix, so scaling an index for array access takes a separate **SHFT** before the **LOAD**:

```
1 ; r2 holds an element index; each element occupies 4 words
2 SHFT r2, LSL #2 ; r2 = r2 << 2
3 LOAD r3, (r2, a) ; r3 = memory[a + r2]
```

Bit Manipulation

Shifting the source register before a **STOR** is useful for packing a scaled or shifted value into memory without a separate ALU instruction:

```
1 ; Store a scaled value
2 STOR (r3, a), r2, LSL #2 ; memory[a + r3] = r2 * 4
```

Combined with Auto-Increment/Decrement

STOR's shift suffix works seamlessly with pre-decrement mode:

```
1 ; Decrement, shift the source value, and store
2 STOR -(r2, a), r1, LSR #2 ; a -= 1; memory[a + r2] = r1 >> 2
```

14.7.5 Status Flags

Note: Unlike shifts in ALU instructions, a shift performed during a **STOR** cannot update the status register flags. For flag updates based on a shift result, use an ALU instruction or the **SHFT** meta-instruction (see Chapter 13).

CHAPTER 15

Control Flow Instructions

15.1 Overview

Control flow in SIRC-1 is built from effective-address instructions. The real CPU opcodes compute an address into an address-register pair; jumps and calls are produced by targeting the program-counter pair `p`.

The SIRC-1 provides these control-flow and address-calculation operations:

- **LDEA** – Load Effective Address into an address-register pair
- **LDEL** – Load Effective Address and Link; also saves the return address in `l`
- **BRAN** – PC-relative assembler meta-instruction for `LDEA p, ...`
- **BRSR** – PC-relative assembler meta-instruction for `LDEL p, ...`
- **RETS** – Return meta-instruction for `LDEA p, (#0, l)`
- **LJMP** – Absolute-jump assembler meta-instruction for `LDEA p, ...`
- **LJSR** – Absolute-call assembler meta-instruction for `LDEL p, ...`

All encoded control-flow instructions can be conditional. If the condition is false, the instruction is skipped and has no side effects, as described in Chapter 11.

15.2 Legal Forms

Instruction	Syntax	Encoding	Notes
LDEA	LDEA dest, (#offset, src)	0x18	Immediate-offset effective address
LDEA	LDEA dest, (r0, src)	0x19	Register-offset effective address
LDEA	LDEA dest, -(#offset, src)	0x1A	Pre-decrement source, immediate offset
LDEA	LDEA dest, -(r0, src)	0x1B	Pre-decrement source, register offset
LDEL	LDEL dest, (#offset, src)	0x1C	Immediate-offset effective address and link; dest excludes <code>l</code>
LDEL	LDEL dest, (r0, src)	0x1D	Register-offset effective address and link; dest excludes <code>l</code>
LDEL	LDEL dest, (#offset, src)+	0x1E	Post-increment source, immediate offset; dest excludes <code>l</code>
LDEL	LDEL dest, (r0, src)+	0x1F	Post-increment source, register offset; dest excludes <code>l</code>
BRAN	BRAN #disp	LDEA meta	Assembles to LDEA p, (#disp, p)
BRAN	BRAN @label	LDEA meta	PC-relative linker-computed displacement
BRSR	BRSR #disp	LDEL meta	Assembles to LDEL p, (#disp, p)
BRSR	BRSR @label	LDEL meta	PC-relative linker-computed call
RETS	RETS	LDEA meta	Assembles to LDEA p, (#0, l)
LJMP	LJMP src	LDEA meta	Assembles to LDEA p, (#0, src)
LJMP	LJMP src, #offset	LDEA meta	Assembles to LDEA p, (#offset, src)
LJMP	LJMP src, r0	LDEA meta	Assembles to LDEA p, (r0, src)
LJSR	LJSR src	LDEL meta	Assembles to LDEL p, (#0, src)
LJSR	LJSR src, #offset	LDEL meta	Assembles to LDEL p, (#offset, src)
LJSR	LJSR src, r0	LDEL meta	Assembles to LDEL p, (r0, src)
LJSR	LJSR (#offset, src)+	LDEL meta	Assembles to LDEL p, (#offset, src)+
LJSR	LJSR (r0, src)+	LDEL meta	Assembles to LDEL p, (r0, src)+

Table 15.1: Legal Control-Flow and Effective-Address Forms

Address registers: `src` and `dest` are address-register pairs: `l`, `a`, `s`, or `p`. The branch meta-instructions always use `p` as both the source and destination address-register pair. `l` is never a legal **LDEL** destination, because **LDEL** always writes the return address to `l`; the destination and the link write would alias. **LDEL** destinations are `a`, `s`, or `p`.

Zero displacement: Immediate-displacement forms may omit `#0`. Thus `LDEA dest, (src)`, `LDEA dest, -(src)`, `LDEL dest, (src)`, and `LDEL dest, (src)+` assemble as the matching `#0` forms. For non-auto-update absolute jumps and calls, use the meta-instruction syntax `LJMP src` or `LJSR src`; `LJMP (src)` and `LJSR (src)` are not public syntax. The post-increment call shorthand `LJSR (src)+` is legal and assembles as `LDEL p, (#0, src)+`.

Aliased register writes: Instructions that write the same address-register pair through more than one write-back path have architecturally undefined behavior. The assembler rejects exactly these forms: **LDEA** pre-decrement with destination equal to source; **LDEL** with destination `l` (any form); **LDEL** post-increment with source `l` or with destination equal to source; **LJSR** post-increment from `l` or `p`; **LOAD** post-increment whose destination is a half of the auto-updated pair; and **STOR** pre-decrement whose source is a half of the auto-updated pair.

Shift suffixes: Control-flow and effective-address displacements are not shifted. Register-offset forms do not accept shift suffixes in the assembler.

15.3 Common Semantics

Topic	Definition
Condition false	The instruction is skipped and has no side effects. The program counter advances to the next instruction normally.
Address calculation	The selected source address-register high word supplies the segment. The target low word is the selected source low word plus the immediate or register displacement.
Pre-decrement LDEA	The source low word is decremented by one word before the displacement is applied. The decremented source low word is written back.
Post-increment LDEL	The target address is calculated from the original source value. The source low word is incremented by one word in write-back.
Subroutine link	LDEL , BRSR , and LJSR save the normal next-instruction address in 1 .
Status flags	Preserved. Control-flow instructions do not support status update override syntax.
Timing	6 cycles, plus any instruction-fetch wait states. These instructions do not perform a data-memory bus access.
Exceptions	Effective-address calculation can raise segment-overflow faults when SR.A is set; see the notes below.
Privilege	In protected mode, address-register-pair write-back is allowed only when every written high word would remain unchanged. If any high word would change, the instruction raises a privilege-violation fault instead of completing write-back. Direct writes to high address registers also raise a privilege-violation fault.
Fault side effects	Program-counter, destination address-register, source auto-update, and link-register updates occur after condition evaluation. If a fault aborts execution before write-back, those effects do not complete.

Table 15.2: Common Control-Flow Instruction Semantics

Control-flow and effective-address instructions do not perform data-memory access and do not raise data bus, data bus-protection, or invalid-opcode faults in documented forms. Effective-address calculation can raise a segment-overflow fault when **SR.A** (Trap on Address Overflow) is set and the 16-bit low-word address calculation wraps. Instruction fetch can still raise the normal fetch faults before the instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits.

15.4 Status Flag Effects

Control-flow and effective-address instructions do not update status flags. All four condition flags are preserved regardless of whether the instruction branches, links, faults, or is skipped. Control-flow instructions do not support status update override syntax.

Instruction	N	Z	C	V	Meaning
LDEA	-	-	-	-	Flags are preserved.
LDEL	-	-	-	-	Flags are preserved.

Table 15.3: Control-Flow Instruction Status Flag Effects

See Table 11.2 for the flag-effect symbol definitions. Meta-instructions (**BRAN**, **BRSR**, **RETS**, **LJMP**, **LJSR**) inherit the flag-preservation behavior of the **LDEA** or **LDEL** form they assemble to.

15.5 Branch Meta-Instructions

BRAN

Branch Meta-Instruction

Applicability: SIRC-1, all revisions

Assembles to: **LDEA** with *p* as the source and destination

Instruction Fields:

- **Register field (bits 25–22, low 2 bits used):** fixed at 0×3 (*p*) by the meta-instruction.
- **AF (bits 5–4):** fixed at 0×3 (*p*) by the meta-instruction.
- **Immediate (bits 21–6):** signed 16-bit displacement.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 BRAN #disp ; p = p + disp
2 BRAN @label ; p = p + label offset
3 BRAN|cond #disp ; Conditional PC-relative branch
```

Operands: *#disp* is a signed 16-bit word displacement (-32768 to $+32767$) added to the low word of *p* modulo 2^{16} ; a branch cannot change the program segment, so the displacement wraps within the current 64K segment. *@label* is resolved by the linker to a word displacement from the address of the branch instruction itself.

Operation:

```
; Assembles to LDEA p, (#disp, p)
if condition:
    p = p + displacement
```

Description:

Branches relative to the current program counter. **BRAN** is an assembler meta-instruction, not a distinct CPU opcode. The displacement base is the branching instruction's own address, so **BRAN** #0 branches to itself and **BRAN** #2 falls through to the next instruction. Use **LDEA** *p*, (*#offset*, *src*) or **LDEA** *p*, (*r0*, *src*) for non-PC-relative computed jumps.

Write-back: Inherits **LDEA** write-back to *p*.

Flags: Preserved.

Exceptions: Same as **LDEA**; may raise segment-overflow or protected-mode privilege faults.

Condition field: If the condition is false, no branch occurs and status flags are preserved.

Timing: Same as **LDEA**: 6 cycles, plus instruction-fetch wait states.

Privilege: Same as **LDEA**. Protected-mode branches must preserve the high word of *p*.

Example 15-1:

```
1 ; Unconditional branch forward
2 BRAN #20 ; Jump ahead 20 words
3
4 ; Backward branch (loop)
5 loop:
6     ; ... loop body ...
7     BRAN|!= @loop ; Jump back if not zero
```

BRSR

Branch to Subroutine Meta-Instruction

Applicability: SIRC-1, all revisions

Assembles to: **LDEL** with *p* as the source and destination

Instruction Fields:

- **Register field (bits 25–22, low 2 bits used):** fixed at 0x3 (*p*) by the meta-instruction.
- **AF (bits 5–4):** fixed at 0x3 (*p*) by the meta-instruction.
- **Immediate (bits 21–6):** signed 16-bit displacement.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 BRSR #disp ; l = next PC; p = p + disp
2 BRSR @label ; l = next PC; p = p + label offset
3 BRSR|cond #disp ; Conditional PC-relative call
```

Operands: #disp is a signed 16-bit word displacement (–32768 to +32767) added to the low word of *p* modulo 2^{16} ; a call cannot change the program segment, so the displacement wraps within the current 64K segment. @label is resolved by the linker to a word displacement from the address of the call instruction itself.

Operation:

```
; Assembles to LDEL p, (#disp, p)
if condition:
    l = address_of_next_instruction
    p = p + displacement
```

Description:

Calls a PC-relative subroutine by saving the return address in *l* and branching to the computed target. The displacement base is the branching instruction's own address, so the **BRSR** link value is that address plus 2 (the address of the next instruction). Return with **RETS**, which assembles to **LDEA** *p*, (#0, *l*).

Write-back: Inherits **LDEL** write-back to *l* and *p*.

Flags: Preserved.

Exceptions: Same as **LDEL**; may raise segment-overflow or protected-mode privilege faults.

Condition field: If the condition is false, no call occurs, *l* is not written, and status flags are preserved.

Timing: Same as **LDEL**: 6 cycles, plus instruction-fetch wait states.

Privilege: Same as **LDEL**. Protected-mode calls must preserve the high words written to *l* and *p*.

Example 15-2:

```
1 ; PC-relative call
2 BRSR @function
3
4 ; Return from the subroutine
5 RETS
```

RETS

Return from Subroutine Meta-Instruction

Applicability: SIRC-1, all revisions

Assembles to: `LDEA p, (#0, 1)`

Instruction Fields:

- **Register field (bits 25–22, low 2 bits used):** fixed at `0x3` (`p`) by the meta-instruction.
- **AF (bits 5–4):** fixed at `0x0` (`1`) by the meta-instruction.
- **Immediate (bits 21–6):** fixed at `0x0000` by the meta-instruction.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 RETS ; p = 1
2 RETS|cond ; Conditional return
```

Operands: None. The source address-register pair is implicitly `1`; the destination is implicitly `p`.

Operation:

```
; Assembles to LDEA p, (#0, 1)
if condition:
    p = 1
```

Description:

Returns from a subroutine by copying the link register to the program counter. Used after `LJSR` or `BRSR`.

Write-back: Inherits `LDEA` write-back to `p`.

Flags: Preserved.

Exceptions: Same as `LDEA`; may raise protected-mode privilege faults if the return would change `p`'s high word.

Condition field: If the condition is false, no return occurs and status flags are preserved.

Timing: Same as `LDEA`: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions.

Example 15-3:

```
1 my_function:
2     ; ... function body ...
3     RETS ; Return to caller
```

15.6 Load Effective Address

LDEA

Load Effective Address

Applicability: SIRC-1, all revisions

Opcodes: 0x18 (Immediate), 0x19 (Register), 0x1A (Pre-Dec Immediate), 0x1B (Pre-Dec Register)

Instruction Fields:

- **Register field (bits 25–22, low 2 bits used):** destination address-register pair `dest`; 0x0 = 1, 0x1 = a, 0x2 = s, 0x3 = p.
- **AF (bits 5–4):** source address-register pair `src`; 0x0 = 1, 0x1 = a, 0x2 = s, 0x3 = p.
- **Immediate (bits 21–6, immediate forms only):** 16-bit signed displacement.
- **R3 (bits 17–14, register forms only):** `r0`, a general-purpose register supplying the displacement.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```

1 LDEA dest, (#offset, src) ; dest = src + offset
2 LDEA dest, (src) ; equivalent to LDEA dest, (#0, src)
3 LDEA dest, (r0, src) ; dest = src + r0
4 LDEA dest, -(#offset, src) ; src -= 1; dest = src + offset
5 LDEA dest, -(src) ; equivalent to LDEA dest, -(#0, src)
6 LDEA dest, -(r0, src) ; src -= 1; dest = src + r0
7 LDEA|cond dest, (#offset, src)

```

Operands: `dest` and `src` are address-register pairs. Immediate forms use a 16-bit displacement. Register forms use `r0` as a displacement. Pre-decrement forms also update `src`.

Operation:

```

if condition:
    if pre_decrement:
        src.low = src.low - 1
    dest.high = src.high
    dest.low = src.low + displacement

```

Description:

Computes an effective address and loads it into the destination address-register pair. This is useful for pointer arithmetic and for control flow when the destination is `p`. Pre-decrement forms also update the source address register pair.

Write-back: If the condition is true, writes the computed address to `dest`. Pre-decrement forms also write the decremented source address back to `src`.

Flags: Preserved.

Exceptions: May raise a segment-overflow fault during effective-address calculation when `SR.A` is set. In protected mode, raises a privilege-violation fault if write-back would change a written address-register high word.

Condition field: If the condition is false, no destination or source address register is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states. No data-memory bus access is performed.

Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions.

Example 15-4:

```
1 ; Pointer arithmetic
2 LDEA a, (#16, a) ; a = a + 16
3
4 ; Computed jump
5 LDEA p, (r1, a) ; p = a + r1
6
7 ; Pre-decrement source pointer and copy nearby address
8 LDEA a, -(#0, s) ; s -= 1; a = s
```

LDEL

Load Effective Address and Link

Applicability: SIRC-1, all revisions

Opcodes: 0x1C (Immediate), 0x1D (Register), 0x1E (Post-Inc Immediate), 0x1F (Post-Inc Register)

Instruction Fields:

- **Register field (bits 25–22, low 2 bits used):** destination address-register pair `dest`; 0x1 = a, 0x2 = s, 0x3 = p (0x0 = 1 is not a legal **LDEL** destination).
- **AF (bits 5–4):** source address-register pair `src`; 0x0 = 1, 0x1 = a, 0x2 = s, 0x3 = p.
- **Immediate (bits 21–6, immediate forms only):** 16-bit signed displacement.
- **R3 (bits 17–14, register forms only):** `r0`, a general-purpose register supplying the displacement.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```

1 LDEL dest, (#offset, src) ; l = next PC; dest = src + offset
2 LDEL dest, (src) ; equivalent to LDEL dest, (#0, src)
3 LDEL dest, (r0, src) ; l = next PC; dest = src + r0
4 LDEL dest, (#offset, src)+ ; l = next PC; dest = src + offset; src += 1
5 LDEL dest, (src)+ ; equivalent to LDEL dest, (#0, src)+
6 LDEL dest, (r0, src)+ ; l = next PC; dest = src + r0; src += 1
7 LDEL|cond dest, (#offset, src)

```

Operands: `dest` and `src` are address-register pairs. Immediate forms use a 16-bit displacement. Register forms use `r0` as a displacement. Post-increment forms also update `src`.

Operation:

```

if condition:
    l = address_of_next_instruction
    dest.high = src.high
    dest.low = src.low + displacement
    if post_increment:
        src.low = src.low + 1

```

Description:

Computes an effective address, saves the return address to `l`, and writes the computed address to the destination address-register pair. It is the real link-writing primitive behind **BRSR** and **LJSR**.

Write-back: If the condition is true, writes the return address to `l` and the computed address to `dest`. Post-increment forms also write the incremented source address back to `src`.

Flags: Preserved.

Exceptions: May raise a segment-overflow fault during effective-address calculation when `SR.A` is set. In protected mode, raises a privilege-violation fault if write-back would change a written address-register high word.

Condition field: If the condition is false, no destination, link, or source address register is written and status flags are preserved.

Timing: 6 cycles, plus instruction-fetch wait states. No data-memory bus access is performed.

Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions.

Example 15-5:

```
1 ; Absolute call through a
2 LDEL p, (#0, a) ; l = next PC; p = a
3
4 ; Explicit destination form
5 LDEL a, (r1, s) ; l = next PC; a = s + r1
6
7 ; Post-increment source pointer after call target calculation
8 LDEL p, (#0, a)+ ; l = next PC; p = old a; a += 1
```

15.7 Long Jump Meta-Instructions

LJMP

Long Jump Meta-Instruction

Applicability: SIRC-1, all revisions

Assembles to: **LDEA** with *p* as the destination

Instruction Fields:

- **Register field (bits 25–22, low 2 bits used):** fixed at 0x3 (*p*) by the meta-instruction.
- **AF (bits 5–4):** source address-register pair *src*; 0x0 = 1, 0x1 = *a*, 0x2 = *s*, 0x3 = *p*.
- **Immediate (bits 21–6, #offset form only):** 16-bit signed displacement; fixed at 0x0000 for plain **LJMP** *src*.
- **R3 (bits 17–14, r0 form only):** *r0*, a general-purpose register supplying the displacement.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 LJMP src ; p = src
2 LJMP src, #offset ; p = src + offset
3 LJMP src, r0 ; p = src + r0
4 LJMP|cond src ; Conditional long jump
```

Operands: *src* is an address-register pair. Optional *#offset* or *r0* operands select immediate or register displacement forms.

Operation:

```
; Assembles to LDEA p, ...
if condition:
    p = src + displacement
```

Description:

Performs an absolute jump through an address register. **LJMP** is an assembler meta-instruction, not a separate CPU opcode.

Write-back: Inherits **LDEA** write-back to *p*.

Flags: Preserved.

Exceptions: Same as **LDEA**; may raise segment-overflow or protected-mode privilege faults.

Condition field: If the condition is false, no jump occurs and status flags are preserved.

Timing: Same as **LDEA**: 6 cycles, plus instruction-fetch wait states.

Privilege: Same as **LDEA**. Protected-mode jumps must preserve the high word of *p*.

Example 15-6:

```
1 ; Jump to address in a
2 LJMP a ; p = a
3
4 ; Jump with offset
5 LJMP a, #16 ; p = a + 16
6
7 ; Jump with register displacement
8 LOAD r1, #4
9 LJMP a, r1 ; p = a + r1
```

LJSR

Long Jump to Subroutine Meta-Instruction

Applicability: SIRC-1, all revisions

Assembles to: `LDEL` with `p` as the destination

Instruction Fields:

- **Register field (bits 25–22, low 2 bits used):** fixed at `0x3` (`p`) by the meta-instruction.
- **AF (bits 5–4):** source address-register pair `src`; `0x0 = l`, `0x1 = a`, `0x2 = s`, `0x3 = p` for non-post-increment forms. Post-increment forms accept only `0x1 = a` or `0x2 = s`; `l` and `p` are aliased address-register writes with architecturally undefined behavior and are rejected by the assembler.
- **Immediate (bits 21–6, #offset forms only):** 16-bit signed displacement; fixed at `0x0000` for plain `LJSR src`.
- **R3 (bits 17–14, r0 forms only):** `r0`, a general-purpose register supplying the displacement.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```

1 LJSR src ; l = next PC; p = src
2 LJSR src, #offset ; l = next PC; p = src + offset
3 LJSR src, r0 ; l = next PC; p = src + r0
4 LJSR (#offset, src)+ ; l = next PC; p = src + offset; src += 1
5 LJSR (src)+ ; equivalent to LJSR (#0, src)+
6 LJSR (r0, src)+ ; l = next PC; p = src + r0; src += 1
7 LJSR |cond src ; Conditional long call

```

Operands: `src` is an address-register pair. Optional `#offset` or `r0` operands select immediate or register displacement forms. Post-increment forms also update `src`.

Operation:

```

; Assembles to LDEL p, ...
if condition:
    l = address_of_next_instruction
    p = src + displacement
    if post_increment:
        src.low = src.low + 1

```

Description:

Calls a subroutine through an address register. The return address is saved in `l`; return with **RETS**. Post-increment forms advance `src` after computing the target. Because **LJSR** implicitly writes `l` and `p`, post-increment source registers must not be `l` or `p`; such forms are aliased address-register writes with architecturally undefined behavior and are rejected by the assembler (see Appendix C).

Write-back: Inherits **LDEL** write-back to `l`, `p`, and any post-increment source register.

Flags: Preserved.

Exceptions: Same as **LDEL**; may raise segment-overflow or protected-mode privilege faults.

Condition field: If the condition is false, no call occurs, `l` is not written, any post-increment source register is preserved, and status flags are preserved.

Timing: Same as **LDEL**: 6 cycles, plus instruction-fetch wait states.

Privilege: Same as **LDEL**. Protected-mode calls must preserve the high words written to `l` and `p`.

Example 15-7:

```

1 ; Call function at address in a
2 ; supervisor mode only -- writing ah is privileged
3 LOAD ah, #0x0040
4 LOAD al, #0x0000 ; a = 0x00400000
5 LJSR a ; Call 0x00400000
6
7 ; Call with offset
8 LJSR a, @function_offset ; p = a + function_offset
9
10 ; Call with register displacement
11 LOAD r1, #8
12 LJSR a, r1 ; p = a + r1
13
14 ; Call through a table entry and advance to the next entry
15 LJSR (#0, a)+ ; p = old a; a += 1
16 LJSR (r1, a)+ ; p = a + r1; a += 1

```

15.8 Control Flow Patterns

15.8.1 If-Then-Else

```

1 ; if (r1 > r2) { ... } else { ... }
2 CMPR r1, r2
3 BRAN |<= @else_part
4 ; Then part
5 ; ...
6 BRAN @end
7 else_part:
8 ; Else part
9 ; ...
10 end:

```

15.8.2 While Loop

```

1 ; while (r1 > 0) { ... }
2 while_start:
3   CMPI r1, #0
4   BRAN |<= @while_end
5   ; Loop body
6   ; ...
7   BRAN @while_start
8 while_end:

```

15.8.3 Switch/Jump Table

```

1 ; Switch on r1 (0-3)
2 LOAD r2, r1 ; r2 = r1
3 ; Scale the index to a 2-word BRAN entry and skip over this LDEA
4 ADDI r2, #2, LSL #1 ; r2 = (r2 << 1) + 2
5 LDEA p, (r2, p) ; Jump into the branch table below
6
7 jump_table:
8   BRAN @case0
9   BRAN @case1
10  BRAN @case2

```

11	BRAN @case3
----	--------------------

CHAPTER 16

Coprocessor Instructions

16.1 Overview

The SIRC-1 CPU includes a coprocessor interface that allows delegation of certain operations to specialized coprocessors. The primary coprocessor is the Exception Unit (Coprocessor 1), which handles exceptions, interrupts, and privileged operations.

Coprocessor calls are ordinary conditional instructions. If the instruction's condition is true, the processing unit computes a command operand and writes it to the 16-bit pending coprocessor-command latch during write-back. The selected coprocessor then interprets the command value. If the selected coprocessor is not present in the CPU model or does not implement the requested operation, the exception unit raises an invalid-opcode fault so the operation can be emulated or rejected by software.

16.2 Legal Forms

Instruction	Syntax	Encoding	Notes
COPI	COPI #imm16	0x0F	16-bit coprocessor command operand
COPR	COPR rS	0x3F	Command operand is read from rS

Table 16.1: Legal Coprocessor Instruction Forms

The CPU encoding reserves the register field of the immediate format (opcode 0x0F) and the R1, R2, and shift fields of the register format (opcode 0x3F); the immediate format has no shift fields. The programmer-facing **COPI** and **COPR** syntax hides those fields; assemblers emit canonical values and coprocessors must not interpret them as destination-register writes. Opcode 0x2F is Undocumented, in the same class as opcodes 0x08, 0x09, 0x0B, and 0x0D (see Appendix C); it has no public assembly syntax.

16.3 Common Semantics

Topic	Definition
Condition false	The instruction is skipped and has no side effects. The pending coprocessor command, registers, memory, address registers, and status flags are preserved. The program counter still advances to the following instruction.
Command operand	COP1 uses the 16-bit immediate value. COPR uses its source register value as the command operand.
Command format	For standard coprocessor commands, bits 15–12 select the coprocessor ID, bits 11–8 select the coprocessor operation, and bits 7–0 are coprocessor-defined operand bits.
Write-back	The processing unit writes the command operand to the pending coprocessor-command latch. It does not write general-purpose registers, address registers, or data memory as part of the call. Hidden encoding fields are ignored.
Status flags	Preserved by the processing-unit coprocessor-call write-back path. A coprocessor operation may define its own later CPU-state effects; exception-unit effects are documented in Chapter 6.
Privilege	In protected mode, coprocessor-operation nibbles 0x0–0x7 are user-callable and 0x8–0xF are supervisor-only. Privilege checks only apply when the instruction condition is true. For COPR the privilege decision is made on the run-time value of <code>rs</code> sampled during decode, not on any encoded field, so whether a given COPR instruction faults depends on register contents.
Timing	The coprocessor-call instruction itself uses the normal 6 execution phases and performs no data-memory bus access. Dispatch to coprocessor 1 then occupies one further complete 6-cycle slot beginning at the next phase 0, so a coprocessor-1 operation completes 12 cycles after the call's own fetch began.
Exceptions	Supervisor-only operations can raise privilege-violation faults; missing coprocessors and unimplemented operations can raise invalid-opcode faults. See the notes below.
Fault side effects	If a fault is raised while dispatching the command, software must not depend on the contents of the pending command latch after the fault. A pending coprocessor command is discarded if a fault or hardware exception is dispatched before the coprocessor's dispatch slot begins; the command is not re-issued after the handler returns.

Table 16.2: Common Coprocessor Instruction Semantics

Coprocessor-call instructions do not perform data-memory access and do not raise data bus, data bus-protection, data alignment, or effective-address segment-overflow faults. A program-counter wrap with `SR.A` set still raises a segment-overflow fault at the following instruction fetch, as it does for every instruction. Instruction fetch can still raise the normal fetch faults before the call executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the processing-unit call commits, unless an earlier fault prevents the instruction from completing. In protected mode, supervisor-only coprocessor operation nibbles are rejected before the command is dispatched. Missing coprocessors and unimplemented coprocessor operations are detected after the command has been latched for dispatch and raise invalid-opcode faults through the exception unit.

COPI/COPR

Coprocessor Call

Applicability: SIRC-1, all revisions

Opcodes: 0x0F (Immediate), 0x3F (Register). Opcode 0x2F is Undocumented.

Instruction Fields:

- **Register (bits 25–22, Immediate format only):** reserved; the assembler emits 0x0 and coprocessors must not interpret it as a destination register.
- **Register 1, Register 2 (Register format, bits 25–18):** reserved; the assembler emits 0x0 for both.
- **Register 3 (Register format, bits 17–14):** rS, any of the 16 registers, 0x0–0xF; its value becomes the 16-bit command operand.
- **Immediate (bits 21–6, Immediate format only):** 16-bit command operand for **COPR**.
- **Shift fields (Register format, bits 13–6):** reserved; the assembler emits 0 and coprocessors must not interpret them.
- **AF (bits 5–4):** reserved; the assembler emits 00 and coprocessors must not interpret it as a status-update selector.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 COPI #opcode ; Call coprocessor with 16-bit command
2 COPR rS ; Call coprocessor with command in rS
```

Operands: **COPI** takes a 16-bit immediate command operand. **COPR** takes any register name as its source; its 16-bit value is used as the command operand. A protected-mode **COPR sr** reads only the low byte of **sr** and therefore always builds a coprocessor-0 command. Hidden encoded register and shift fields are canonicalized by the assembler and are not programmer-visible operands.

Operation:

```
1 if condition_true then
2     pending_coprocessor_command = command_operand
3 endif
```

Description:

Transfers a command to a coprocessor. For the standard command layout, the high nibble selects the coprocessor and the next nibble selects that coprocessor's operation. The remaining low byte is defined by the selected operation.

Write-back: Writes the command operand to the pending coprocessor-command latch.

Flags: Preserved by the coprocessor-call instruction.

Exceptions: May raise privilege-violation faults for supervisor-only operations in protected mode. May raise invalid-opcode faults for missing coprocessors and unimplemented coprocessor operations.

Condition field: If the condition is false, no command is dispatched, the pending command latch is preserved, no coprocessor privilege or opcode checks are performed, and status flags are preserved.

Timing: 6 cycles for the processing-unit call, plus a further 6-cycle coprocessor dispatch slot (12 cycles in total), plus any bus wait states.

Privilege: Available in protected mode and supervisor mode. In protected mode, coprocessor-operation nibbles 0x8–0xF are supervisor-only and raise privilege-violation faults when executed. For **COPR** the privilege decision is made on the run-time value of **rs** sampled during decode, not on any encoded field, so whether a given **COPR** instruction faults depends on register contents.

Example 16-1:

```

1 ; Wait for interrupt (Exception Unit)
2 COPR #0x1900 ; Coprocessor 1, operation 0x9
3 ; Prefer the WAIT meta-instruction for normal assembly code.
4
5 ; Return from exception
6 COPR #0x1A00 ; Coprocessor 1, operation 0xA
7 ; Prefer the RETE meta-instruction for normal assembly code.
8
9 ; User exception (system call)
10 COPR #0x1180 ; Coprocessor 1, operation 0x1, vector 0x80
11 ; Prefer EXCP #0x80 for normal assembly code.
```

16.4 Coprocessor Compatibility

Standard coprocessor IDs are stable across SIRC-1 models. A CPU model may omit an optional coprocessor, but it must not assign that coprocessor ID to an incompatible unit. If software calls an absent coprocessor or an operation that the selected coprocessor does not implement, the CPU raises an invalid-opcode fault.

ID	Name	Required	Operations	Compatibility rule
0x0	Processing Unit	Yes	Internal	Executes SIRCIS processing-unit instructions
0x1	Exception Unit	Yes	0x1, 0x9–0xD (software); 0x0, 0xE–0xF (internal)	Handles reset, faults, interrupts, traps, and links
0x2	DMA Unit	Yes	0x8–0xA	Required standard DMA transfer unit
0x3	Integer Maths Unit	No	0x0–0x3	Optional standard integer multiply/divide unit
0x4–0xF	Reserved/model space	No	Model-defined	Reserved unless a CPU model documents the assignment

Table 16.3: Standard Coprocessor IDs

Exception-unit operation nibble 0x0 is the internal no-operation encoding and operation nibbles 0x2–0x8 are architecturally undefined. Operations 0xE and 0xF are internal encodings the exception unit uses to dispatch a fault or a hardware exception; see Chapter 6 for the internal-instruction mechanism.

16.4.1 Model and Revision Policy

- Required coprocessors and documented operations must remain backward compatible within the SIRC-1 family.
- Optional standard coprocessors keep their assigned IDs even on models that omit them.
- Reserved coprocessor IDs must raise invalid-opcode faults when they are not implemented by the selected CPU model. Reserved operation nibbles and reserved operand values within an implemented coprocessor are architecturally undefined.

- Software may probe optional coprocessors by executing the documented operation and handling an invalid-opcode fault, or by using a model-specific capability table if one is provided by system firmware. The invalid-opcode fault records the offending 16-bit coprocessor command word in the `return_address` field of the fault-metadata link register (link register #7), which an emulation handler reads with **ETFR**. The pending command is cleared before the handler runs.
- Processing-unit undocumented opcodes are a separate case: they are accepted by the processing unit and have architecturally undefined effects unless a future CPU revision documents them.

16.5 Exception Unit (Coprocessor 1)

The Exception Unit handles:

- Faults (highest priority)
- Hardware exceptions (interrupts, multiple priority levels)
- Software exceptions (traps)
- Privileged operations

16.5.1 Common Operations

Opcode	Mnemonic	Operation
0x1100-0x11FF	EXCP	User exception (trap) at vector 0x60-0xFF
0x1900	WAIT	Wait for interrupt
0x1A00	RETE	Return from exception
0x1B00	RSET	System reset
0x1C00-0x1C37	ETFR	Transfer from exception link register
0x1D00-0x1D37	ETTR	Transfer to exception link register

Table 16.4: Exception Unit Operations

For **ETFR** and **ETTR**, command bits 7–6 are not decoded, bits 5–4 select the register (00 none, 01 saved address, 10 saved status, 11 both), and bits 3–0 hold the link-register index, 0–7; values with bit 6 or bit 7 set are not distinct operations. Coprocessor-1 operations 0xE and 0xF are the exception unit’s internal encodings for dispatching a fault or a hardware exception; a software-issued **COPI** or **COPR** naming operation 0xE or 0xF is architecturally undefined.

16.6 Exception Unit Meta-Instructions

EXCP

User Exception Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Assembles to: `COP1 #0x1100 | vector`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** `0x1100 | #vector`; bits 15–12 select coprocessor 1, bits 11–8 select operation `0x1`, and bits 7–0 hold `#vector, 0x00–0xFF`.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax: `EXCP #vector`

Operands: `#vector` is the low 8-bit user exception vector selector.

Operation:

```

1 if condition_true then
2     if protected_mode and vector < 0x60 then
3         raise privilege_violation_fault
4     else if current_exception_level != 0 then
5         discard pending_coprocessor_command
6     else
7         link[0] = { pc, sr, current_exception_level }
8         current_exception_level = 1
9         pc = vector_table[vector]
10    endif
11 endif

```

Description: Triggers a software exception at the specified user vector, normally `0x60–0xFF`. Used by programs running in protected mode to invoke system calls or request supervisor-mode services.

Write-back: Inherits `COP1` command dispatch to the exception unit.

Flags: Preserved by the coprocessor-call instruction; exception entry effects are defined by the exception unit.

Exceptions: Dispatches a software exception through the exception unit. Raises a privilege-violation fault in protected mode if the selected vector is below `0x60`. Software exceptions are accepted only at exception level 0; an **EXCP** executed inside any exception or fault handler is discarded – the pending coprocessor command is cleared, no vector is fetched, and no fault is raised. Nested system calls from a handler are therefore not possible.

Condition field: If the condition is false, no trap is dispatched and status flags are preserved.

Timing: 6 cycles for the call plus a 6-cycle exception-unit dispatch slot.

Privilege: User-callable in protected mode and supervisor mode.

Example 16-2:

```

1 LOAD r1, #'A' ; Character to print
2 EXCP #0x80 ; Trap to OS print routine

```

WAIT

Wait for Interrupt Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Assembles to: `COP1 #0x1900`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x1900` (coprocessor 1, operation `0x9`, operand byte `0x00`).
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax: `WAIT`

Operands: None.

Operation:

```

1 if condition_true then
2     if protected_mode then
3         raise_privilege_violation_fault
4     else
5         stop_instruction_fetch
6         wait until an enabled hardware interrupt line is asserted or the CPU
           is reset
7     endif
8 endif

```

Description: Stops instruction fetch and places the CPU in a low-power wait state. The CPU issues no bus cycles while waiting. It leaves the wait state only when an enabled hardware interrupt line is asserted (NMI is always enabled) or when the CPU is reset. Interrupts masked by the hardware interrupt enable bits (`SR.E1–SR.E4`) do not wake the CPU, and faults and software exceptions cannot occur while waiting.

Write-back: Inherits `COP1` command dispatch to the exception unit.

Flags: Preserved by the coprocessor-call instruction.

Exceptions: Raises a privilege-violation fault if executed in protected mode.

Condition field: If the condition is false, the CPU does not enter wait state and status flags are preserved.

Timing: 6 cycles for the call plus a 6-cycle exception-unit dispatch slot.

Privilege: Supervisor mode only.

Example 16-4:

```

1 idle_loop:
2     WAIT ; Halt until an interrupt or reset occurs
3     BRAN @idle_loop ; Resume waiting if woken spuriously

```

RETE

Return from Exception Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Assembles to: `COP1 #0x1A00`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x1A00` (coprocessor 1, operation `0xA`, operand byte `0x00`).
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax: `RETE`

Operands: None.

Operation:

```

1 if condition_true then
2     if protected_mode then
3         raise_privilege_violation_fault
4     else
5         n = current_exception_level - 1
6         pc = link[n].return_address
7         sr = link[n].return_status
8         current_exception_level = link[n].saved_level
9     endif
10 endif

```

Description: Returns from an exception handler by restoring the saved program counter and status register. Executing **RETE** at exception level 0, that is, outside any exception or fault handler, is architecturally undefined.

Write-back: Inherits **COP1** command dispatch to the exception unit.

Flags: Restored by the exception unit as part of exception return.

Exceptions: Raises a privilege-violation fault if executed in protected mode.

Condition field: If the condition is false, no exception return occurs and current status flags are preserved.

Timing: 6 cycles for the call plus a 6-cycle exception-unit dispatch slot.

Privilege: Supervisor mode only.

Example 16-5:

```

1 timer_handler:
2     ; ... service the timer interrupt ...
3     RETE ; Return to the interrupted program

```

RSET

System Reset Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Assembles to: `COP1 #0x1B00`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x1B00` (coprocessor 1, operation `0xB`, operand byte `0x00`).
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax: `RSET`

Operands: None.

Operation:

```

1 if condition_true then
2     if protected_mode then
3         raise_privilege_violation_fault
4     else
5         assert_reset_output_for_6_cycles
6         sr = 0x0000
7         pc = vector_table[reset_vector]
8     endif
9 endif

```

Description: Requests a full processor reset. The reset output is asserted and held for 6 cycles, after which the exception unit fetches the reset vector, sets `sr` to `0x0000`, and loads the program counter from the vector. General-purpose registers, the `a`, `l`, and `s` pairs, and the exception link registers are not affected by reset and hold undefined values after power-on.

Write-back: Inherits `COP1` command dispatch to the exception unit.

Flags: Reset by the exception unit as part of reset processing.

Exceptions: Raises a privilege-violation fault if executed in protected mode.

Condition field: If the condition is false, no reset occurs and current status flags are preserved.

Timing: 6 cycles for the call, plus a 6-cycle `RSTO` hold, plus 6 cycles for the exception unit to fetch the reset vector – 18 cycles from the **RSET** fetch to the first instruction of the reset handler.

Privilege: Supervisor mode only.

Example 16-6:

```

1 ; Request a full processor reset from supervisor code
2 RSET ; Reboots through the reset vector

```

ETFR/ETTR

Exception Link Transfer Meta-Instructions

Applicability: SIRC-1, all revisions; requires Coprocessor 1 (Exception Unit), a required coprocessor present in all SIRC-1 models.

Assembles to: `COPI #0x1Cxy` or `COPI #0x1Dxy`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** `0x1Cxy` (**ETFR**) or `0x1Dxy` (**ETTR**); bits 15–12 select coprocessor 1, bits 11–8 select operation `0xC` or `0xD`. Within the low operand byte `xy`: bits 7–6 are not decoded, bits 5–4 (`x`) select the register (`00` none, `01` saved address only, `10` saved status only, `11` both), and bits 3–0 (`y`) hold `#n`, the link-register index 0–7.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```

1 ETFR #n ; Transfer saved address and status to a and r7
2 ETFR a, #n ; Transfer saved address to a only
3 ETFR r7, #n ; Transfer saved status to r7 only
4 ETTR #n ; Transfer a and r7 to saved address and status
5 ETTR #n, a ; Transfer a to saved address only
6 ETTR #n, r7 ; Transfer r7 to saved status only

```

Operands: `#n` selects one of the eight exception link registers and must be in the range `#0–#7`. Link register `#7` holds fault metadata; link register `L – 1` holds the state saved on entry to an exception at level `L`. Values above `#7` are architecturally undefined; the assembler currently masks `#n` to four bits without diagnosing the error. Optional operands select whether to transfer the saved return address, saved status register, or both.

Operation:

```

1 if condition_true then
2     if protected_mode then
3         raise_privilege_violation_fault
4     else if operation == ETFR then
5         if x includes saved address then a = link[n].return_address
6         if x includes saved status then r7 = link[n].return_status
7     else
8         if x includes saved address then link[n].return_address = a
9         if x includes saved status then link[n].return_status = r7
10    endif
11 endif

```

Description: **ETFR** copies saved exception state from an exception link register to CPU registers. **ETTR** copies CPU register state back to the selected exception link register before returning.

Write-back: **ETFR** writes selected saved state to `a`, `r7`, or both. **ETTR** writes `a`, `r7`, or both to selected saved exception state.

Flags: Preserved by the coprocessor-call instruction.

Exceptions: Raises a privilege-violation fault if executed in protected mode.

Condition field: If the condition is false, no transfer occurs and status flags are preserved.

Timing: 6 cycles for the call plus a 6-cycle exception-unit dispatch slot.

Privilege: Supervisor mode only.

Example 16-3:

```

1 alignment_fault_handler:
2     ETFR a, #6 ; Get faulting return address
3     LOAD al, #0x0100 ; Correct the address
4     ETTR #6, a ; Write corrected address back
5     RETE

```

See Chapter 6 for detailed documentation of exception handling and all exception coprocessor instructions.

16.7 DMA Unit (Coprocessor 2)

The required DMA unit provides bulk word transfers using the existing address register pairs and the general-purpose register file. It is intentionally simple: it has no descriptor table, hidden parameter registers, or autonomous background execution. When a DMA command is accepted, the processing unit stops fetching further instructions until the DMA command completes or faults. As of this printing, the reference SIRC-1 emulator has not yet implemented the DMA unit; software must not assume Coprocessor 2 is present until an emulator release documents it.

DMA operations are supervisor-only because they perform memory access using supervisor-visible address state. In protected mode, they raise a privilege-violation fault before dispatch.

16.7.1 DMA Operations

Mnemonic	Command	Count	Registers	Operation
DMAR	0x2800 operand	#-7-#7	a/l/s, r1-r7	Read memory into registers
DMAW	0x2900 operand	#-7-#7	a/l/s, r1-r7	Write registers to memory
DMAT	0x2A00 n	0-255	a, l, r1-r7	Copy memory at a to memory at l

Table 16.5: DMA Unit Operations

For **DMAR** and **DMAW**, the command operand byte is encoded as follows: bits 7-6 select the address register (00 = a, 01 = l, 10 = s, 11 reserved), bit 5 selects direction (0 = forward, 1 = backward), bits 2-0 hold the count magnitude, and bits 4-3 are reserved and written as zero. Negative counts are encoded as the backward direction bit plus the positive magnitude, not as a two's-complement count field. A zero count is encoded with the direction bit clear (forward). Note: this DMA operand field is not the AF address-register field; its 00/01 assignment differs from Table 8.6 in Chapter 8. For all DMA operations, a count of zero is a no-op: no memory access occurs, registers are preserved, and address registers are not advanced. CPU status flags are preserved. If the condition is false, no DMA command is dispatched, no coprocessor privilege or opcode checks are performed, and the pending coprocessor command, registers, memory, address registers, and status flags are preserved; the program counter still advances to the following instruction.

DMA timing assumes the command condition is true and every bus access is acknowledged without wait states. The **COPI** dispatch takes the normal 6 cycles; dispatch to the DMA unit then occupies one further complete 6-cycle slot beginning at the next phase 0, matching the coprocessor-1 dispatch model (see Table 16.2). For a zero count, that slot accepts the command, decodes the count, and resumes instruction fetch, and no further cycles are needed. For a non-zero count, the same slot performs the first no-wait DMA bus access. Each additional DMA read or write bus access then takes one further cycle in the no-wait case. Therefore **DMAR** and **DMAW** take $12 + \max(|n| - 1, 0)$ cycles, and **DMAT** takes $12 + \max(2n - 1, 0)$ cycles. A false condition skips the DMA command entirely and takes only the normal 6-cycle **COPI** instruction time.

DMA memory reads and writes can raise data bus and bus-protection faults using DMA read/write bus access types. If an address-register low word overflows or underflows and **SR.A** is set, the DMA operation raises a segment-overflow fault; otherwise the low word wraps modulo 2^{16} . Partial side

effects are possible if a fault occurs mid-transfer, and software must not assume that re-executing the same DMA instruction restarts the original operation.

DMAR

DMA Read To Registers Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 2 (DMA Unit), a required coprocessor present in all SIRC-1 models.

Assembles to: `COP1 #0x2800 | operand`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** `0x2800 | operand`; bits 15–12 select coprocessor 2 and bits 11–8 select operation `0x8`. Within the low operand byte: bits 7–6 select the address register (`00 = a`, `01 = l`, `10 = s`, `11 reserved` – this is not the AF address-register encoding), bit 5 selects direction (`0 forward`, `1 backward`), bits 4–3 are reserved and written as zero, and bits 2–0 hold the count magnitude, `0–7`.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax: `DMAR addr, #n`

Operands: `addr` must be `a`, `l`, or `s`. `#n` is a literal count operand in the range `#-7–#7`. Counts outside that range are invalid DMA operations.

Operation:

```
1 count = abs(n)
2 if n < 0:
3     addr.low = addr.low - count
4 transfer = addr
5 for i in 1..count:
6     r[i] = memory[transfer]
7     transfer.low = transfer.low + 1
8 if n > 0:
9     addr.low = addr.low + count
```

Description: Reads sequential words from the selected address register into `r1` through `rn`. Positive counts read from the current address and advance it. Negative counts first move the address register backward by the count magnitude, then read the ascending memory block from that new address.

Write-back: A successful positive-count **DMAR** advances the selected address register by the count magnitude. A successful negative-count **DMAR** leaves the selected address register at its decremented start address. Address-register high words are not changed by the transfer increments or decrements. Writes `r1` through `rn`.

Flags: Preserved.

Exceptions: Unsupported DMA operations, reserved operand encodings, and invalid counts raise invalid-opcode faults. In protected mode, raises a privilege-violation fault. DMA reads can raise data bus, bus-protection, and segment-overflow faults.

Condition field: If the condition is false, no DMA command is dispatched and registers, memory, address registers, and status flags are preserved; the program counter still advances to the following instruction.

Timing: $12 + \max(|n| - 1, 0)$ cycles when the command condition is true and the DMA read cycles are acknowledged without wait states. The $n = 0$ no-op form and one-word transfer forms take 12 cycles. Wait states add to the relevant DMA read bus cycle. If the condition is false, the DMA command is not dispatched and the instruction takes 6 cycles.

Privilege: Supervisor mode only.

Example 16-7:

```
1 LOAD ah, #0x0040  
2 LOAD al, #0x0000 ; a = 0x00400000, start of source block  
3 DMAR a, #3 ; r1, r2, r3 = memory[a], memory[a+1], memory[a+2]; a += 3
```

DMAW

DMA Write From Registers Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 2 (DMA Unit), a required coprocessor present in all SIRC-1 models.

Assembles to: COPI #0x2900 | operand

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits 0x0.
- **Immediate (bits 21–6):** 0x2900 | operand; bits 15–12 select coprocessor 2 and bits 11–8 select operation 0x9. Within the low operand byte: bits 7–6 select the address register (00 = a, 01 = l, 10 = s, 11 reserved – this is not the AF address-register encoding), bit 5 selects direction (0 forward, 1 backward), bits 4–3 are reserved and written as zero, and bits 2–0 hold the count magnitude, 0–7.
- **AF (bits 5–4):** fixed at 00.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax: DMAW addr, #n

Operands: addr must be a, l, or s. #n is a literal count operand in the range #-7–#7. Counts outside that range are invalid DMA operations.

Operation:

```

1 count = abs(n)
2 if n < 0:
3     addr.low = addr.low - count
4 transfer = addr
5 for i in 1..count:
6     memory[transfer] = r[i]
7     transfer.low = transfer.low + 1
8 if n > 0:
9     addr.low = addr.low + count

```

Description: Writes r1 through rn to sequential words at the selected address register. Positive counts write from the current address and advance it. Negative counts first move the address register backward by the count magnitude, then write the ascending memory block from that new address. Register order always corresponds to ascending memory addresses, so DMAW s, #-n followed by DMAR s, #n restores r1 through rn.

Write-back: A successful positive-count DMAW advances the selected address register by the count magnitude. A successful negative-count DMAW leaves the selected address register at its decremented start address. Address-register high words are not changed by the transfer increments or decrements. General-purpose registers are preserved; DMAW only reads them.

Flags: Preserved.

Exceptions: Unsupported DMA operations, reserved operand encodings, and invalid counts raise invalid-opcode faults. In protected mode, raises a privilege-violation fault. DMA writes can raise data bus, bus-protection, and segment-overflow faults.

Condition field: If the condition is false, no DMA command is dispatched and registers, memory, address registers, and status flags are preserved; the program counter still advances to the following instruction.

Timing: $12 + \max(|n| - 1, 0)$ cycles when the command condition is true and the DMA write cycles are acknowledged without wait states. The $n = 0$ no-op form and one-word transfer forms take 12 cycles. Wait states add to the relevant DMA write bus cycle. If the condition is false, the DMA command is not dispatched and the instruction takes 6 cycles.

Privilege: Supervisor mode only.

Example 16-8:

```
1 LOAD ah, #0x0040
2 LOAD al, #0x0000 ; a = 0x00400000, start of destination block
3 DMAW a, #3 ; memory[a], memory[a+1], memory[a+2] = r1, r2, r3; a += 3
```

DMAT

DMA Memory Transfer Meta-Instruction

Applicability: SIRC-1, all revisions; requires Coprocessor 2 (DMA Unit), a required coprocessor present in all SIRC-1 models.

Assembles to: `COPI #0x2A00 | n`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** `0x2A00 | #n`; bits 15–12 select coprocessor 2, bits 11–8 select operation `0xA`, and bits 7–0 hold `#n`, 0–255.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax: `DMAT a, l, #n`

Operands: The source and destination address registers must be written explicitly as `a` and `l`. `#n` is an 8-bit literal count operand in the range 0–255. A count of zero is a no-op.

Operation:

```
1 while remaining > 0:
2     chunk = min(remaining, 7)
3     read chunk words from memory[a] into r1..r[chunk]
4     write chunk words from r1..r[chunk] to memory[l]
5     a.low = a.low + chunk
6     l.low = l.low + chunk
7     remaining = remaining - chunk
```

Description: Copies sequential words from the address in `a` to the address in `l`, using `r1–r7` as an internal transfer window in chunks of up to seven words.

Write-back: A successful non-zero **DMAT** advances `a` and `l` by the number of words transferred. Address-register high words are not changed by the transfer increments. `r1–r7` are clobbered; after completion they contain the final transfer window.

Flags: Preserved.

Exceptions: Unsupported DMA operations and reserved operand encodings raise invalid-opcode faults. In protected mode, raises a privilege-violation fault. DMA reads and writes can raise data bus, bus-protection, and segment-overflow faults.

Fault side effects: Registers, memory, and address registers reflect the last completed word transfer if a fault occurs mid-transfer.

Condition field: If the condition is false, no DMA command is dispatched and registers, memory, address registers, and status flags are preserved; the program counter still advances to the following instruction.

Timing: $12 + \max(2n - 1, 0)$ cycles when the command condition is true and all DMA read and write cycles are acknowledged without wait states. The $n = 0$ no-op form takes 12 cycles, and a one-word memory transfer takes 13 cycles. Wait states add to the relevant DMA read or write bus cycle. Instruction fetch resumes only after the DMA operation completes or faults. If the condition is false, the DMA command is not dispatched and the instruction takes 6 cycles.

Privilege: Supervisor mode only.

Overlap: Overlapping source and destination ranges are architecturally undefined unless the source and destination addresses are identical.

Example 16-9:

```

1 LOAD ah, #0x0040
2 LOAD al, #0x0000 ; a = 0x00400000, source block
3 LOAD lh, #0x0050
4 LOAD ll, #0x0000 ; l = 0x00500000, destination block
5 DMAT a, l, #16 ; Copy 16 words from memory[a] to memory[l]

```

16.8 Integer Maths Unit (Coprocessor 3)

The optional integer maths unit provides a stable multiply/divide convention. CPU models without the hardware raise an invalid-opcode fault for these commands, allowing supervisor software to emulate the operation and return as if the coprocessor call had completed.

Reference source listings for software emulation routines are supplied with the SIRC-1 distribution media. Those listings are examples of a supervisor compatibility handler, not additional instruction encodings.

16.8.1 Maths Operations

Mnemonic	Command	Inputs	Outputs	Operation
MULU	0x3000	r1, r2	r1:r2, r3	Unsigned 16 x 16 -> 32 multiply
MULS	0x3100	r1, r2	r1:r2, r3	Signed 16 x 16 -> 32 multiply
DIVU	0x3200	r1:r2, r3	r1, r2, r3	Unsigned divide
DIVS	0x3300	r1:r2, r3	r1, r2, r3	Signed divide

Table 16.6: Integer Maths Unit Operations

Bit	Name	Meaning
0	E	Any maths error
1	DZ	Divide by zero
2	OV	Quotient overflow
3–15	Reserved	Written as zero, ignore read

Table 16.7: Maths Status Word in r3

Integer maths operations preserve CPU status flags. Missing maths hardware and unsupported maths operations raise invalid-opcode faults. Divide by zero and quotient overflow are reported in r3, not as arithmetic exceptions. If the condition is false, no maths command is dispatched, no coprocessor opcode check is performed, and registers and status flags are preserved.

MULU/MULS

Integer Multiply Meta-Instructions

Applicability: SIRC-1, all revisions; requires Coprocessor 3 (Integer Maths Unit), an optional coprocessor absent on models that do not implement it.

Assembles to: `COP1 #0x3000` or `COP1 #0x3100`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x3000` (**MULU**) or `0x3100` (**MULS**); bits 15–12 select coprocessor 3, bits 11–8 select the operation, and bits 7–0 are unused and written as zero.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 MULU ; Unsigned multiply
2 MULS ; Signed multiply
```

Operands: None in assembly syntax. The operations multiply `r1` by `r2`.

Operation:

```
1 MULU:
2     product = unsigned(r1) * unsigned(r2)
3     r1 = product[31:16]
4     r2 = product[15:0]
5     r3 = 0
6
7 MULS:
8     product = signed(r1) * signed(r2)
9     r1 = product[31:16]
10    r2 = product[15:0]
11    r3 = 0
```

Description: Multiplies two 16-bit operands and writes the 32-bit product to `r1:r2`, high word first. **MULU** treats both inputs as unsigned. **MULS** treats both inputs as two's-complement signed values.

Write-back: Writes `r1`, `r2`, and `r3`.

Flags: CPU status flags are preserved. `r3` is written as zero.

Exceptions: Missing maths hardware and unsupported multiply operations raise invalid-opcode faults. These operations perform no data-memory bus access.

Condition field: If the condition is false, no maths command is dispatched and registers and status flags are preserved.

Timing: The `COP1` command dispatch uses the normal 6 execution phases. Hardware maths operations may take additional implementation-defined cycles before instruction fetch resumes.

Privilege: User-callable in protected mode and supervisor mode.

Example 16-10:

```
1 LOAD r1, #1000 ; Multiplicand
2 LOAD r2, #2000 ; Multiplier
3 MULU ; r1:r2 = 1000 * 2000, r3 = 0
```

DIVU/DIVS

Integer Divide Meta-Instructions

Applicability: SIRC-1, all revisions; requires Coprocessor 3 (Integer Maths Unit), an optional coprocessor absent on models that do not implement it.

Assembles to: `COP1 #0x3200` or `COP1 #0x3300`

Instruction Fields:

- **Register (bits 25–22):** reserved; the assembler emits `0x0`.
- **Immediate (bits 21–6):** fixed at `0x3200` (**DIVU**) or `0x3300` (**DIVS**); bits 15–12 select coprocessor 3, bits 11–8 select the operation, and bits 7–0 are unused and written as zero.
- **AF (bits 5–4):** fixed at `00`.
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax:

```
1 DIVU ; Unsigned divide
2 DIVS ; Signed divide
```

Operands: None in assembly syntax. The dividend is `r1:r2`, high word first. The divisor is `r3`.

Operation:

```
1 if divisor is zero:
2     r3 = E | DZ
3 else if quotient does not fit in 16 bits:
4     r3 = E | OV
5 else:
6     r1 = remainder
7     r2 = quotient
8     r3 = 0
```

Description: Divides a 32-bit dividend by a 16-bit divisor. Successful division writes the remainder to `r1`, the quotient to `r2`, and zero to `r3`.

Write-back: Successful division writes `r1`, `r2`, and `r3`. Divide-by-zero and quotient-overflow cases leave `r1:r2` unchanged and write the maths status word to `r3`.

Flags: CPU status flags are preserved. Maths errors are reported through `r3`, not through `sr`.

Exceptions: Missing maths hardware and unsupported divide operations raise invalid-opcode faults. Divide by zero and quotient overflow do not raise arithmetic exceptions. These operations perform no data-memory bus access.

Condition field: If the condition is false, no maths command is dispatched and registers and status flags are preserved.

Timing: The `COP1` command dispatch uses the normal 6 execution phases. Hardware maths operations may take additional implementation-defined cycles before instruction fetch resumes. Software emulation timing is defined by the invalid-opcode handler.

Privilege: User-callable in protected mode and supervisor mode.

Signed division: **DIVS** treats `r1:r2` as a 32-bit two's-complement dividend and `r3` as a 16-bit two's-complement divisor. The quotient is truncated toward zero and the remainder has the sign of the dividend. The case `-2147483648 / -1` is quotient overflow.

Example 16-11:

```
1 LOAD r1, #0 ; High word of dividend
2 LOAD r2, #100 ; Low word of dividend
3 LOAD r3, #7 ; Divisor
4 DIVU ; r1 = remainder, r2 = quotient, r3 = 0 on success
```


CHAPTER 17

Meta-Instructions

17.1 Overview

Meta-instructions are assembly mnemonics that lower to ordinary SIRCIS instruction encodings. They exist to make common operations easier to read and write; they do not introduce additional CPU opcodes.

Meta-Instruction	Primary Documentation	Lowers To
NOOP	Chapter 17	ADDI [N] r1, #0
SHFT	Chapter 13	ORRI [S]
BRAN	Chapter 15	LDEA p, ...
BRSR	Chapter 15	LDEL p, ...
LJMP	Chapter 15	LDEA p, ...
LJSR	Chapter 15	LDEL p, ...
RETS	Chapter 15	LDEA p, (#0, 1)
EXCP	Chapter 16	COPI
WAIT	Chapter 16	COPI #0x1900
RETE	Chapter 16	COPI #0x1A00
RSET	Chapter 16	COPI #0x1B00
ETFR	Chapter 16	COPI #0x1Cxy
ETTR	Chapter 16	COPI #0x1Dxy
DMAR	Chapter 16	COPI #0x28xx
DMAW	Chapter 16	COPI #0x29xx
DMAT	Chapter 16	COPI #0x2Ann
MULU	Chapter 16	COPI #0x3000
MULS	Chapter 16	COPI #0x3100
DIVU	Chapter 16	COPI #0x3200
DIVS	Chapter 16	COPI #0x3300

Table 17.1: Meta-Instruction Cross-Reference

NOOP

No Operation

Applicability: SIRC-1, all revisions

Assembles to: `ADDI [N] r1, #0`

Instruction Fields:

- **Register (bits 25–22):** fixed at `0x0` (`r1`).
- **Immediate (bits 21–6):** fixed at `#0`.
- **AF (bits 5–4):** fixed at `00` (status register not updated).
- **Condition field (bits 3–0):** any of the 16 condition-code encodings in Table 7.5.

Syntax: `NOOP`

Operands: None.

Operation: Executes an add-immediate-zero instruction whose destination is `r1`: `r1` is rewritten with its own value plus zero, so no architectural state changes occur. Because the status-register update source is `None`, no condition flags are written. Register field `0x0` is deliberately assigned to `r1`, an ordinary unprivileged register (see Chapter 3), so this instruction is unprivileged and an all-zero instruction word (as read from erased or uninitialized memory) always executes safely as a no-op, in both protected and supervisor mode.

Description: Does nothing for 6 cycles. Useful for timing delays, instruction padding, or placeholders during development.

Write-back: No architectural state changes.

Flags: Preserved.

Exceptions: Inherits `ADDI [N]` exception behavior. It does not access data memory and raises no instruction-specific faults; instruction-fetch faults can still occur before it executes, and trace faults can still occur after it commits.

Condition field: If the condition is false, no state changes occur. Conditional `NOOP` is allowed but normally not useful.

Timing: Same as `ADDI`: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode.

Example 17-1:

```
1 NOOP ; Wait 6 cycles
2 NOOP
3 NOOP ; Total 18 cycles delay
```

CHAPTER A

Opcode Map

A.1 Complete Opcode Reference

This appendix provides a complete map of all 64 possible opcodes in the SIRCIS instruction set.

Table A.1: Complete SIRCIS Opcode Map

Hex	Binary	Mnemonic	Description
ALU Immediate (0x00–0x0F)			
0x00	000000	ADDI	Add Immediate
0x01	000001	ADCI	Add with Carry Immediate
0x02	000010	SUBI	Subtract Immediate
0x03	000011	SBCI	Subtract with Carry Immediate
0x04	000100	ANDI	AND Immediate
0x05	000101	ORRI	OR Immediate
0x06	000110	XORI	XOR Immediate
0x07	000111	LOAD	Load Immediate
0x08	001000	–	Undocumented
0x09	001001	–	Undocumented
0x0A	001010	CMPI	Compare Immediate
0x0B	001011	–	Undocumented
0x0C	001100	TSAI	Test AND Immediate
0x0D	001101	–	Undocumented
0x0E	001110	TSXI	Test XOR Immediate
0x0F	001111	COPI	Coprocessor Immediate
Memory and Control (0x10–0x1F)			
0x10	010000	STOR	Store Indirect Immediate
0x11	010001	STOR	Store Indirect Register
0x12	010010	STOR	Store Pre-Decrement Immediate
0x13	010011	STOR	Store Pre-Decrement Register
0x14	010100	LOAD	Load Indirect Immediate
0x15	010101	LOAD	Load Indirect Register
0x16	010110	LOAD	Load Post-Increment Immediate
0x17	010111	LOAD	Load Post-Increment Register
0x18	011000	LDEA	Load Effective Address Immediate
0x19	011001	LDEA	Load Effective Address Register
0x1A	011010	LDEA	Load Effective Address Pre-Decrement Immediate
0x1B	011011	LDEA	Load Effective Address Pre-Decrement Register
0x1C	011100	LDEL	Load Effective Address and Link Immediate
0x1D	011101	LDEL	Load Effective Address and Link Register
0x1E	011110	LDEL	Load Effective Address and Link Post-Increment Immediate
0x1F	011111	LDEL	Load Effective Address and Link Post-Increment Register
ALU Short Immediate (0x20–0x2F)			
0x20	100000	ADDI	Add Short Immediate

continued on next page

Table A.1 (continued)

Hex	Binary	Mnemonic	Description
0x21	100001	ADCI	Add with Carry Short Immediate
0x22	100010	SUBI	Subtract Short Immediate
0x23	100011	SBCI	Subtract with Carry Short Immediate
0x24	100100	ANDI	AND Short Immediate
0x25	100101	ORRI	OR Short Immediate
0x26	100110	XORI	XOR Short Immediate
0x27	100111	–	Undocumented (LOAD-like)
0x28	101000	–	Undocumented
0x29	101001	–	Undocumented
0x2A	101010	CMPI	Compare Short Immediate
0x2B	101011	–	Undocumented
0x2C	101100	TSAI	Test AND Short Immediate
0x2D	101101	–	Undocumented
0x2E	101110	TSXI	Test XOR Short Immediate
0x2F	101111	–	Undocumented (COP-like)
ALU Register (0x30–0x3F)			
0x30	110000	ADDR	Add Register
0x31	110001	ADCR	Add with Carry Register
0x32	110010	SUBR	Subtract Register
0x33	110011	SBCR	Subtract with Carry Register
0x34	110100	ANDR	AND Register
0x35	110101	ORRR	OR Register
0x36	110110	XORR	XOR Register
0x37	110111	LOAD	Load Register
0x38	111000	–	Undocumented
0x39	111001	–	Undocumented
0x3A	111010	CMPR	Compare Register
0x3B	111011	–	Undocumented
0x3C	111100	TSAR	Test AND Register
0x3D	111101	–	Undocumented
0x3E	111110	TSXR	Test XOR Register
0x3F	111111	COPR	Coprocessor Register

A.2 Opcode Patterns

A.2.1 Format Identification

Bit numbers in this section refer to the 6-bit opcode value (instruction bits 31–26), not to the instruction word.

- Opcode bits 5–4 = 00: Immediate format (0x00–0x0F)
- Opcode bits 5–4 = 01: Memory and control operations (0x10–0x1F). Format is selected by opcode bit 0: even opcodes use Immediate format, odd opcodes use Register format.
- Opcode bits 5–4 = 10: Short Immediate with Shift (0x20–0x2F)
- Opcode bits 5–4 = 11: Register format (0x30–0x3F)

A.2.2 Save vs. Test

- Bit 3 = 0 and low nibble 0–6: save result (0x00–0x06, 0x20–0x26, 0x30–0x36)
- Bit 3 = 1 and low nibble 8–E: test only, discard result (0x08–0x0E, 0x28–0x2E, 0x38–0x3E)
- Low nibble F is the coprocessor call (0x0F, 0x3F), not a test form
- Low nibble 7 is **LOAD** (0x07, 0x37)
- 0x27 and 0x2F are undocumented

A.2.3 Memory Operations Decoding

For opcodes `0x10–0x1F`, bits encode memory operation type:

- Bit 4 = 1: Always set for memory/control section
- Bits 3–2: Operation type
 - 00 = Store
 - 01 = Load
 - 10 = Load Effective Address
 - 11 = Load EA with Link (jump to subroutine)
- Bit 1: Auto-update of the source address register (0 = no update, 1 = pre-decrement for STOR/LDEA, post-increment for LOAD/LDEL). Only the low word of the pair is adjusted, and always by one word regardless of the displacement.
- Bit 0: Immediate (0) or Register (1) offset

A.3 Undocumented Instructions

The following opcodes are undocumented and should not be used in production code:

- `0x08`, `0x09`, `0x0B`, `0x0D` (Immediate format, test variants)
- `0x28`, `0x29`, `0x2B`, `0x2D` (Short Immediate format, test variants)
- `0x38`, `0x39`, `0x3B`, `0x3D` (Register format, test variants)
- `0x27`, `0x2F` (no public assembly syntax)

The twelve test-variant opcodes above execute in hardware, but their behavior is architecturally undefined and may change between CPU revisions. See Appendix C for details on all fourteen undocumented opcodes, including `0x27` and `0x2F`.

A.4 Quick Lookup Table

Operation	Immediate	Short+Shift	Register
ADD	<code>0x00</code>	<code>0x20</code>	<code>0x30</code>
ADC	<code>0x01</code>	<code>0x21</code>	<code>0x31</code>
SUB	<code>0x02</code>	<code>0x22</code>	<code>0x32</code>
SBC	<code>0x03</code>	<code>0x23</code>	<code>0x33</code>
AND	<code>0x04</code>	<code>0x24</code>	<code>0x34</code>
ORR	<code>0x05</code>	<code>0x25</code>	<code>0x35</code>
XOR	<code>0x06</code>	<code>0x26</code>	<code>0x36</code>
LOAD	<code>0x07</code>	<code>0x27</code> *	<code>0x37</code>
CMP	<code>0x0A</code>	<code>0x2A</code>	<code>0x3A</code>
TSA	<code>0x0C</code>	<code>0x2C</code>	<code>0x3C</code>
TSX	<code>0x0E</code>	<code>0x2E</code>	<code>0x3E</code>
COP	<code>0x0F</code>	<code>0x2F</code> *	<code>0x3F</code>

Table A.2: Instruction Variant Quick Reference

* These opcodes have no public assembly syntax and are undocumented.

CHAPTER B

Instruction Timing

B.1 Overview

Most SIRCIS instructions follow the six-stage execution phase model and complete in 6 clock cycles when no bus wait states are required. A bus operation may hold the current phase until it is acknowledged, adding wait cycles without adding execution phases. Coprocessor calls are the exception: the dispatched coprocessor occupies at least one further slot after the six-phase call. This fixed execution sequence greatly simplifies software development and hardware design. See Appendix F for the bus signal timing, wait-state, and response-priority rules.

B.2 Six-Stage Execution Phases

For a detailed description of each phase, see Chapter 2.

1. **Instruction Fetch (High Word)** – 1 cycle
2. **Instruction Fetch (Low Word)** – 1 cycle
3. **Decode and Register Fetch** – 1 cycle
4. **Execute and Address Calculation** – 1 cycle
5. **Memory Access** – 1 cycle
6. **Write Back** – 1 cycle

B.3 Timing Characteristics

Instruction Type	Cycles
ALU (Register)	6
ALU (Immediate)	6
Load from Memory	6
Store to Memory	6
Branch (taken)	6
Branch (not taken)	6
Conditional (executed)	6
Conditional (not executed)	6
NOOP	6
DMAR/DMAW	$12 + \max(n - 1, 0)$
DMAT	$12 + \max(2n - 1, 0)$
COPI/COPR	12 (6 + 6 coprocessor dispatch)
Exception-unit meta-instructions (EXCP , RETE , ETFR , ETTR)	12 (6 + 6 dispatch)
RSET	18 (6 call + 6 RSTO hold + 6 reset-vector fetch)
WAIT	12 + idle until an enabled interrupt or reset

Table B.1: Instruction Timing

The cycle counts in this appendix assume that every bus operation is acknowledged without a wait state. For **DMAR**, **DMAW**, and **DMAT**, the DMA unit gets at least one post-dispatch cycle after the 6-cycle **COP1** dispatch. For non-zero transfers, that first post-dispatch cycle may also be the first no-wait DMA bus access. A conditional DMA meta-instruction whose condition is false does not dispatch to the DMA unit and takes only 6 cycles.

B.3.1 Exception and Fault Timing

A hardware-exception or fault dispatch occupies one 6-cycle slot: the vector's high word is fetched at phase 0, the low word at phase 1, and entry to the handler is committed at phase 3. A latched interrupt is dispatched at the next phase 0, so the worst-case latency from assertion to the handler's first instruction fetch is 6 cycles to finish the current instruction plus 6 cycles to dispatch. Reset holds **RSTO** for 6 cycles before the reset-vector fetch.

A fault aborts the six-phase sequence of the faulting instruction at the phase that detects it; the remaining phases of that instruction do not run. Alignment is detected before phase 0; Privilege Violation is detected at phase 2; Segment Overflow is detected at phase 2 or phase 3; Bus and Bus Protection faults are detected at the phase that acknowledges the failing bus access. In every case the exception-unit dispatch slot begins in the cycle immediately following detection. Alignment, Bus, Bus Protection, Invalid Opcode, Privilege Violation, and Segment Overflow all save the address of the faulting instruction and are retryable.

B.4 Conditional Execution

When a conditional instruction's condition is not met:

- All six phases still occur and consume one cycle each
- The execute, memory-access, and write-back phases perform no operation: no effective address is computed, no **SR.A** overflow check is made, and no data bus cycle is driven
- No register, memory, or address-register update occurs
- Status flags are not updated

This means conditional instructions do not branch and avoid control-flow stalls, but failed conditions still consume execution time.

B.5 Program Timing Examples

B.5.1 Simple Loop

```
1 ; Process 10 elements (assuming no bus wait states)
2 LOAD r7, #10 ; 6 cycles
3 :loop
4     LOAD r1, (#0, a)+ ; 6 cycles
5     ADDI r1, #1 ; 6 cycles
6     STOR (#-1, a), r1 ; 6 cycles
7     SUBI r7, #1 ; 6 cycles
8     BRAN != @loop ; 6 cycles
9 ; Total per iteration: 30 cycles
10 ; Total for 10 iterations: 6 + (10 * 30) = 306 cycles
```

B.5.2 Function Call Overhead

```
1 ; Call function
2 LJSR a ; 6 cycles
3
4 ; Function body executes...
5
6 ; Return
7 RETS ; 6 cycles
8 ; Total overhead: 12 cycles
```

B.5.3 Conditional Block

```

1 CMPR r1, r2 ; 6 cycles
2 ADDR|HI r3, r4, r5 ; 6 cycles (exec if r1 > r2)
3 ADDR|LO r3, r6, r7 ; 6 cycles (exec if r1 <= r2)
4 ; Total: 18 cycles (one conditional executes, one doesn't)

```

B.6 Performance Considerations

B.6.1 No Pipeline Stalls

The SIRC-1 does not implement pipelining in the traditional sense where multiple instructions are in flight simultaneously. Instead:

- Each instruction completes fully before the next begins
- No data hazards or phase bubbles
- No branch prediction penalties
- Base execution time is predictable before bus wait states are considered

B.6.2 Memory Access

- Instruction fetch, data memory access, and exception vector fetch each occupy one execution phase per 16-bit word transferred.
- If BACK is not asserted and neither BERR nor BPER aborts the request, the CPU holds that phase until the bus operation is acknowledged.
- Each wait state adds one clock cycle to the instruction
- Every instruction performs two instruction-fetch bus accesses; a load or store adds one data access; an exception or fault dispatch adds two exception-vector-fetch accesses; a coprocessor dispatch performs no bus access of its own. A system that inserts w wait states per access therefore adds $2w$ cycles to an ALU instruction and $3w$ cycles to a load or store.

B.6.3 Optimization Guidelines

1. Use conditional execution for short sequences

```

1 ; Instead of:
2 CMPR r1, r2
3 BRAN|LO @skip
4 ADDI r3, #1
5 :skip
6
7 ; Use:
8 CMPR r1, r2
9 ADDI|HI r3, #1 ; Saves branch overhead

```

2. Combine operations with shifts

```

1 ; Instead of:
2 ADDR r1, r2, r2 ; r1 = 2 * r2
3 ADDR r1, r1, r2 ; r1 = 3 * r2
4
5 ; Use:
6 ADDR r1, r2, r2, LSL #1 ; Multiply by 3 in one instruction

```

3. Use post-increment/pre-decrement for iteration

```

1 ; Instead of:
2 LOAD r1, (#0, a)
3 ADDI a1, #1
4
5 ; Use:
6 LOAD r1, (#0, a)+ ; Same result, fewer cycles

```

4. Loop unrolling reduces overhead

```
1 ; Instead of processing one item per iteration,  
2 ; process multiple items to reduce branch overhead  
3 :loop  
4     LOAD r1, (#0, a)+  
5     LOAD r2, (#0, a)+  
6     LOAD r3, (#0, a)+  
7     LOAD r4, (#0, a)+  
8     ; Process 4 items...  
9     SUBI r7, #4  
10    BRAN|HI @loop
```

B.7 Theoretical Performance

At a given clock frequency:

- **Instructions per second:** $f_{clock}/6$
- **At 10 MHz:** ≈ 1.67 million instructions/second
- **At 24 MHz (maximum rated clock):** ≈ 4.0 million instructions/second

These are theoretical maximums assuming perfect code with no inefficiencies.

The SIRC-1's fixed 6-cycle execution trades peak throughput for predictability: every ordinary instruction takes the same number of cycles regardless of the operation or operands, which is simpler to implement in hardware and easier to reason about than a variable-cycle-count design.

CHAPTER C

Undocumented Instructions

C.1 Overview

The SIRCIS instruction set includes 14 undocumented instruction opcodes. Most of these opcodes are valid encodings with architecturally undefined behaviour: software must not depend on their results, side effects, timing, flag behaviour, or fault behaviour. Two of the fourteen, `0x27` and `0x2F`, are the exception – each has fully defined, tested behaviour (`0x27`: `rD` is loaded with the zero-extended 8-bit immediate; `0x2F`: the coprocessor command is the 8-bit immediate and the coprocessor ID is always 0, so the instruction is never privileged) but no public assembly syntax, and they are listed here for that reason rather than because their behaviour is undefined. Undocumented opcodes are reserved for future CPU revisions and may be repurposed without preserving their current simulator or hardware behaviour.

Warning: Use of undocumented instructions is not recommended for production code.

C.2 Undefined Operand Combinations

Undefined operand combinations are different from undocumented opcodes: the opcode is documented, but a particular choice of operands creates multiple architectural writes to the same address-register pair during the same instruction.

C.2.1 Aliased Address-Register Writes

The auto-update effective-address forms can write both a destination address-register pair and an updated source address-register pair. `LDEL` also writes the link-register pair. If two of those write-back targets are the same address-register pair, the final value is architecturally undefined.

```
1 ; Undefined: a is both destination and auto-updated source
2 LDEA a, -(#0, a)
3
4 ; Undefined: l is both link register and auto-updated source
5 LDEL p, (#0, l)+
6
7 ; Undefined: l is both link register and destination
8 LDEL l, (#0, a)
```

Programs should avoid aliased register writes. The assembler rejects the known hazardous forms so that programs do not accidentally depend on undefined write-back ordering.

C.3 Undocumented Opcode List

Opcode	Format	Pattern	Notes
0x08	Immediate	ADD-like + 0x8	Test variant (no save)
0x09	Immediate	ADC-like + 0x8	Test variant (no save)
0x0B	Immediate	SBC-like + 0x8	Test variant (no save)
0x0D	Immediate	OR-like + 0x8	Test variant (no save)
0x27	Short Imm	LOAD-like	No public assembly syntax
0x28	Short Imm	ADD-like + 0x8	Test variant (no save)
0x29	Short Imm	ADC-like + 0x8	Test variant (no save)
0x2B	Short Imm	SBC-like + 0x8	Test variant (no save)
0x2D	Short Imm	OR-like + 0x8	Test variant (no save)
0x2F	Short Imm	COP-like	No public assembly syntax
0x38	Register	ADD-like + 0x8	Test variant (no save)
0x39	Register	ADC-like + 0x8	Test variant (no save)
0x3B	Register	SBC-like + 0x8	Test variant (no save)
0x3D	Register	OR-like + 0x8	Test variant (no save)

Table C.1: Undocumented Instruction Opcodes

C.4 Observed Behavior

The undocumented opcodes with bit 3 set in the ALU operation field compute `alu_code & 0x7` as the operation, discard the result, and update all four flags exactly as the corresponding documented operation does. This is the observed behavior of the current reference implementation; it is architecturally undefined, so software must not rely on it.

C.4.1 0x08, 0x28, 0x38 (ADD Test)

Computes the sum of the two operands but discards the result, updating flags as an ADD would. This is comparable to **CMP**, but using ADD semantics instead of SUB.

```

1 result = operand1 + operand2 ; Calculate
2 ; result is discarded (not written to register)
3 ; Flags updated based on result

```

Utility: Limited. **CMP** already provides comparison functionality.

C.4.2 0x09, 0x29, 0x39 (ADC Test)

Computes the sum of the two operands plus the incoming carry, but discards the result, updating flags.

```

1 result = operand1 + operand2 + C
2 ; result is discarded
3 ; Flags updated

```

Utility: Possibly useful for testing carry propagation without modifying registers.

C.4.3 0x0B, 0x2B, 0x3B (SBC Test)

Computes the difference of the two operands minus the incoming borrow, but discards the result, updating flags.

```

1 result = operand1 - operand2 - C ; C is the borrow input
2 ; result is discarded
3 ; Flags updated

```

Utility: Possibly useful for testing borrow propagation in multi-precision arithmetic.

C.4.4 0x0D, 0x2D, 0x3D (OR Test)

Computes the bitwise OR of the two operands but discards the result. N and Z are updated from the OR result; C and V are cleared.

```
1 result = operand1 | operand2
2 ; result is discarded
3 ; N and Z updated from result; C and V cleared
```

Utility: Limited. **ORRI** already provides the same flag effects with a result write-back.

C.5 Implementation Notes

C.5.1 Current Simulator

In the current SIRC-VM simulator:

- Undocumented instructions are recognized and decoded
- The undocumented ALU test-variant opcodes execute following the "test variant" pattern (no write-back)
- Flag updates match the corresponding documented operation exactly, as verified by the reference test suite
- Behavior may change in future versions

C.5.2 Hardware Implementation

In a hardware implementation:

- The decoder treats bit 3 = 1 as "don't write back"
- ALU operations still execute normally
- Flags are still updated
- The write-back stage is simply disabled

This means undocumented encodings may execute, but their exact behavior depends on the hardware implementation details and is not part of the architectural contract.

C.6 Why Undocumented?

These instructions were left undocumented for several reasons:

1. **Limited utility:** Most have no practical use beyond documented instructions
2. **Incomplete semantics:** The designer hasn't fully defined their behavior
3. **Opcod space reservation:** May be used for future features
4. **Implementation flexibility:** Allows hardware optimizations

C.7 Recommendations

C.7.1 For Application Developers

Do not use undocumented instructions in production code.

- Behavior may change between CPU revisions
- Assemblers may reject them or emit warnings
- Code may not be portable
- Debugging tools may not recognize them

C.7.2 For Compiler Writers

- Do not generate undocumented instructions
- Use only documented instructions for all operations
- If optimization requires new operations, request formal documentation

C.7.3 For Hardware Implementers

- These opcodes may be implemented following the logical pattern
- Document any implementation-specific behavior
- Consider reserving them for future standardization

C.7.4 For Researchers/Hackers

Experimenting with undocumented instructions requires the following precautions:

- Test thoroughly on the specific hardware or simulator version in use
- Document the findings
- Isolate usage in test code, not production
- Be prepared for behavior changes

C.8 Future Standardization

Some undocumented instructions may be formally specified in future SIRCIS revisions. Candidates for standardization include:

- **ADC Test (0x09/0x29/0x39)**: For carry flag testing in multi-precision arithmetic
- **SBC Test (0x0B/0x2B/0x3B)**: For borrow flag testing in multi-precision arithmetic

Others (the ADD Test and OR Test variants) will likely remain undocumented or be repurposed.

C.9 Opcode Space for Expansion

The SIRCIS instruction set has limited expansion space:

- 14 undocumented opcodes (could be formally specified)
- Reserved shift type (111) could enable additional shift modes
- The reserved status-update source encoding (AF = 11) could select a new flag-update source
- Standard optional coprocessors and model-specific coprocessors can provide extended functionality through the documented coprocessor compatibility rules

Any future expansion must maintain backward compatibility with existing code.

CHAPTER D

Example Programs

This appendix contains sample programs that demonstrate several features of the CPU.

D.1 Byte Sieve

This program computes primes below 1000 using the Sieve of Eratosthenes, storing one byte per candidate number in a memory-mapped array rather than in a register or the stack. It is a compact illustration of addressing-register setup (ah/al and sh/sl) and of using **STOR** and **LOAD** with an indexed effective address to walk an array. The two nested loops starting at labels :2 and :3 (lines 35–50) are the core of the sieve: the outer loop advances the candidate, and the inner loop marks its multiples as composite.

Listing D.1: Example D-1: Byte sieve

```
1 ; https://rosettacode.org/wiki/Sieve\_of\_Eratosthenes
2
3 .EQU $MAX_PRIME #1000
4
5 ; Reserved space for 256x32 bit exception vectors
6 .ORG 0x0000
7 .DQ @main
8
9 .ORG 0x0200
10 :main
11
12 ; File I/O segment
13 LOAD ah, #0x00F0
14 ; Array Start = 0x00F0_0000
15 LOAD al, #0x0000
16
17 ; Stack segment (other end of mapped file)
18 LOAD sh, #0x00F0
19 ; Stack top = 0x00F0_FFFF
20 LOAD sl, #0xFFFF
21
22 LOAD r7, #2
23 LOAD r1, #2
24 LOAD r2, #1
25
26 :1
27
28 STOR (r1, a), r2
29 ADDI r1, #2
30 CMPI r1, $MAX_PRIME
31 BRAN |<= @1
32 LOAD r1, #3
33 LOAD r3, #1
34
35 :2
36
37 LOAD r2, (r1, a)
38 CMPI r2, #1
39 BRAN |== @4
40 ; THIS NUMBER IS A PRIME!
```

```

41 ; TODO: Log this somehow
42 STOR -(s), r1
43 LOAD r7, r1
44 LOAD r2, r1
45
46 :3
47 STOR (r2, a), r3
48 ADDR r2, r1
49 CMPI r2, $MAX_PRIME
50 BRAN|<= @3
51
52 :4
53 ADDI r1, #2
54 CMPI r1, $MAX_PRIME
55 BRAN|<= @2
56
57 :DONE
58
59 ; Halt CPU
60 COPI #0x14FF

```

D.2 Faults

This pair of programs exercises every fault type the CPU can raise and the vector table that routes them to handlers. `faults.sasm` deliberately triggers, in sequence, a bus fault, an alignment fault, two segment-overflow faults (one from an effective-address calculation, one from program-counter overflow), an instruction-trace fault, a bus-protection fault, an invalid-opcode fault (which itself triggers a nested double fault), a privilege violation, and a level-five interrupt. Each handler, defined from line 144 onward, increments a distinct register so a reader can confirm which faults fired and in what order. The segment-overflow handler (lines 167–204) is worth particular attention: it reads exception metadata with **ETFR** to distinguish an instruction-fetch overflow from an effective-address overflow before deciding how to fix up the return address.

Listing D.2: Example D-2: Fault handling

```

1 ;; Segments
2 .EQU $PROGRAM_SEGMENT #0x0000
3 .EQU $SCRATCH_SEGMENT #0x0001
4 .EQU $PROGRAM_SEGMENT_HIGH #0x0002
5
6 ;; Devices
7 ; Debug
8 .EQU $DEBUG_DEVICE_SEGMENT #0x000B
9 .EQU $DEBUG_DEVICE_BUS_ERROR #0x0000
10 .EQU $DEBUG_DEVICE_EXCEPTION_L5 #0x0005
11 .EQU $DEBUG_DEVICE_BUS_PROTECTION_ERROR #0x0006
12
13
14 ;; Bus access types (fault metadata register, bits 0-2)
15 .EQU $BAT_NONE #0x0000
16 .EQU $BAT_DATA_WRITE #0x0003
17
18
19 ; Reserved space for 256x32 bit exception vectors
20 .ORG 0x0000
21 .DQ @init
22 .ORG 0x0002
23 .DQ @bus_fault_handler
24 .ORG 0x0004
25 .DQ @alignment_fault_handler
26 .ORG 0x0006

```

```

27 .DQ @segment_overflow_fault_handler
28 .ORG 0x0008
29 .DQ @invalid_opcode_fault_handler
30 .ORG 0x000A
31 .DQ @privilege_violation_fault_handler
32 .ORG 0x000C
33 .DQ @instruction_trace_fault_handler
34 .ORG 0x000E
35 .DQ @level_five_interrupt_conflict_handler
36 .ORG 0x0010
37 .DQ @double_fault_handler
38 .ORG 0x0012
39 .DQ @bus_protection_fault_handler
40
41 .ORG 0x0020
42 .DQ @level_five_interrupt_handler
43
44 .ORG 0x0200
45 :init
46 LOAD r1, #1
47 LOAD r2, #1
48 LOAD r3, #1
49 LOAD r4, #1
50 LOAD r5, #1
51 LOAD r6, #1
52 LOAD r7, #1
53
54 ; Get the debug device to raise a fault on the bus
55 LOAD ah, $DEBUG_DEVICE_SEGMENT
56 LOAD al, $DEBUG_DEVICE_BUS_ERROR
57 STOR (a), r1
58
59 ; Jumping to an odd address triggers an alignment fault
60 LOAD pl, #0x0201
61
62 :after_odd_address
63
64 ; Loading/storing from an odd address should not trigger an alignment fault
65 LOAD ah, $SCRATCH_SEGMENT
66 LOAD al, #0x0
67 LOAD r7, #0xCAFE
68 STOR (#1, a), r7
69 LOAD r7, (#1, a)
70
71
72 ; Calculating an effective address that overflows triggers a segment
   overflow fault
73 ; (but only if the SR is set up the right way)
74 LOAD ah, $PROGRAM_SEGMENT
75 LOAD al, #0xFFFE
76
77 ; This should not trigger fault
78 LOAD r7, (#2, a)
79 ; Switch on trap on overflow bit
80 ORRI sr, #0b0100_0000_0000_0000
81 ; This _should_ trigger a fault
82 LOAD r7, (#2, a)
83 ; Switch off trap on overflow bit
84 ANDI sr, #0b1011_1111_1111_1111

```

```
85
86 ; Segment overflow again, but this time the PC overflow causes it
87
88 LOAD ah, $PROGRAM_SEGMENT_HIGH
89 LOAD al, #0xFFFE
90
91 ; This should not trigger fault
92 LJMP a
93
94 :return_from_testing_pc_overflow_no_fault
95
96 ; Switch on trap on overflow bit
97 ORRI sr, #0b0100_0000_0000_0000
98
99 LOAD ah, $PROGRAM_SEGMENT_HIGH
100 LOAD al, #0xFFFE
101
102 ; This _should_ trigger a fault
103 LJMP a
104
105 :return_from_testing_pc_overflow_with_fault
106
107 ; Switch off trap on overflow bit
108 ANDI sr, #0b1011_1111_1111_1111
109
110 ; Instruction trace fault test
111 ; Set T bit - the next instruction will be traced and the handler will fire
112 ORRI sr, #0b1000_0000_0000_0000
113 ; This instruction is traced
114 NOOP
115 ; T bit was cleared by the trace handler, so no further instructions are
    traced
116
117 ; Bus protection fault test
118 ; Get the debug device to raise a protection error on the bus
119 LOAD ah, $DEBUG_DEVICE_SEGMENT
120 LOAD al, $DEBUG_DEVICE_BUS_PROTECTION_ERROR
121 LOAD r7, #0x1
122 STOR (a), r7
123
124 ; There is no coprocessor at ID 4, so should trigger an invalid opcode
    fault
125 COPI #0x4000
126
127 ; Switch to non privileged mode
128 ORRI sr, #0b0000_0001_0000_0000
129 ; Try to escape the current segment
130 LOAD ph, #0xFEFE
131
132
133 ; Get the debug device to raise an interrupt (this one should be fine)
134 LOAD r7, #0x1
135 LOAD ah, $DEBUG_DEVICE_SEGMENT
136 LOAD al, $DEBUG_DEVICE_EXCEPTION_L5
137 STOR (a), r7
138
139 ; Halt CPU
140 COPI #0b0001_0100_1111_1111
141
```

```

142 .ORG 0x0300
143
144 :bus_fault_handler
145 ADDI r1, #1
146 RETE
147
148 :bus_protection_fault_handler
149 ADDI r5, #1
150 RETE
151
152 :alignment_fault_handler
153 ADDI r2, #1
154
155 ; Fix up the return address because the original PC causes an alignment
    fault
156 ; Returning without fixing up the return address would cause an endless
    loop
157
158 ; Transfer current ELR into 'a' register
159 ETFR a, #6
160 ; Correct the lower word
161 LOAD al, @after_odd_address
162 ; Transfer corrected address back to ELR
163 ETTR #6, a
164
165 RETE
166
167 :segment_overflow_fault_handler
168 ; Check which kind of address computation caused the overflow, using the
169 ; bus access type captured in the fault metadata register
170
171 ; Transfer fault metadata to r7
172 ETFR r7, #7
173 ; Mask off the bus access type (bits 0-2) from the fault metadata register
174 ANDI r7, #0x7
175
176 CMPI r7, $BAT_NONE
177 BRAN |== @segment_overflow_fault_handler_pc
178 ; Mask off the bus access type (bits 0-2) from the fault metadata register
179 ANDI r7, #0x7
180
181 CMPI r7, $BAT_DATA_WRITE
182 BRAN |== @segment_overflow_fault_handler_address
183
184 BRAN @segment_overflow_fault_handler_end
185
186 :segment_overflow_fault_handler_pc
187
188 ADDI r3, #0x0F00
189
190 ; Fix up the return address because it isn't valid
191 ; Load address after fault
192 LOAD ah, $PROGRAM_SEGMENT
193 LOAD al, @return_from_testing_pc_overflow_with_fault
194 ; Transfer corrected address back to ELR
195 ETTR #6, a
196
197 BRAN @segment_overflow_fault_handler_end
198

```

```
199 :segment_overflow_fault_handler_address
200
201 ADDI r3, #0x000F
202
203 :segment_overflow_fault_handler_end
204
205 RETE
206
207 :invalid_opcode_fault_handler
208 ; A double fault will blat the EU link registers so we need to save them
   ; before we trigger it
209 ETFR #6
210
211 ADDI r4, #1
212 ; Trigger another invalid opcode fault to test double fault handling
213 ; This will cause a fault while already handling a fault
214 COPI #0xF000
215
216 ; Restore link register
217 ETTR #6
218
219 RETE
220
221 :privilege_violation_fault_handler
222 ADDI r5, #1
223
224 ; Make sure that the CPU is back in privileged mode again after handling
   ; this fault
225
226 ; Transfer current ELR into 'r7' register
227 ETFR r7, #6
228 ; Correct the lower word
229 ANDI r7, #0xFEFF
230 ; Transfer corrected SR back to ELR
231 ETTR #6, r7
232
233 RETE
234
235 :double_fault_handler
236 ; A double fault occurred - this is typically a last-chance handler
237 ; In this example, we'll increment r4 to show we were here
238 ; and then return (though in production you'd probably reset)
239 ADDI r4, #1
240 RETE
241
242 :level_five_interrupt_conflict_handler
243 ADDI r6, #0x1
244 RETE
245
246 :level_five_interrupt_handler
247 ADDI r6, #0x1000
248
249 ; Trigger another L5 interrupt while it is already being serviced
250 ; which will trigger a fault
251 LOAD r7, #0x1
252 LOAD ah, $DEBUG_DEVICE_SEGMENT
253 LOAD al, $DEBUG_DEVICE_EXCEPTION_L5
254 STOR (a), r7
255
```

```

256 RETE
257
258 :instruction_trace_fault_handler
259 ADDI r6, #0x100
260 ; Clear T bit from the saved SR so RETE does not re-enable tracing
261 ETFR r7, #6
262 ANDI r7, #0x7FFF
263 ETTR #6, r7
264 RETE
265
266 ; Trampoline for faults-high to come back
267 .ORG 0xFFFF0
268 LOAD pl, @return_from_testing_pc_overflow_no_fault

```

faults-high.sasm is a second, separately assembled segment that the program-counter overflow test in Example D-2 jumps into; it exists only to let that fault trigger without the CPU executing the vector table sitting at address 0x0000 in the main segment. It jumps back into faults.sasm through the trampoline at 0xFFFF0.

Listing D.3: Example D-3: High-address fault handling

```

1 ;; Segments
2 .EQU $PROGRAM_SEGMENT #0x0000
3 .EQU $SCRATCH_SEGMENT #0x0001
4 .EQU $PROGRAM_SEGMENT_HIGH #0x0002
5
6 ; Section to test program counter overflow
7 ; without trying to execute the vector table
8 ; that lives at 0x0000 in the main segment
9
10 ; Note: This is not linked together with the main program
11 ; This is compiled as a separate program and mapped as a separate segment
12
13 .ORG 0x0000
14
15 LOAD ah, $PROGRAM_SEGMENT
16 LOAD al, #0xFFFF0
17 LJMP a
18
19
20 ; TODO: Work out how to avoid padding this out
21 ; I had to pad it out because even though I said this segment has a length
   of 0xFFFF
22 ; I think the way files are mapped means that length is ignored
23 .ORG 0xFFFFE
24 LOAD r1, r1

```

D.3 Hardware Exception

This example shows a device-driven interrupt handled through the serial device: the program prints a help message, waits on **WAIT** for a hardware exception, and exits once the handler reports that the expected character has arrived in **r7**. The exception handler itself is installed at vector 0x0080 and defined from line 137 onward; it saves the link register, calls `read_pending_byte` and `write_pending_byte` in turn, and restores the link register before returning with **RETE**. Those two subroutines show the poll-and-acknowledge pattern used to drain and refill the serial device's receive and send buffers one byte at a time.

Listing D.4: Example D-4: Hardware exception

```

1 ; Reserved space for 256x32 bit exception vectors
2
3

```

```
4 .EQU $FALSE #0x0
5 .EQU $TRUE #0x1
6
7 ; When this character is typed, the program should exit
8 .EQU $EXPECTED_CHAR #0x61
9
10 ;; Segments
11 .EQU $PROGRAM_SEGMENT #0x0000
12 .EQU $SCRATCH_SEGMENT #0x0001
13
14 ;; Devices
15 ; Serial
16 .EQU $SERIAL_DEVICE_SEGMENT #0x000A
17 .EQU $SERIAL_DEVICE_BAUD #0x0000
18 .EQU $SERIAL_DEVICE_RECV_ENABLED #0x0001
19 .EQU $SERIAL_DEVICE_RECV_PENDING #0x0002
20 .EQU $SERIAL_DEVICE_RECV_DATA #0x0003
21 .EQU $SERIAL_DEVICE_SEND_ENABLED #0x0004
22 .EQU $SERIAL_DEVICE_SEND_PENDING #0x0005
23 .EQU $SERIAL_DEVICE_SEND_DATA #0x0006
24
25 ;; Scratch Variables
26 .EQU $MESSAGE_SEND_BASE #0x0000
27 .EQU $MESSAGE_SEND_POINTER #0x0001
28
29
30 ;; Exception Vectors
31 ; First vector is the reset vector
32 .ORG 0x0000
33 .DQ @start
34
35 .ORG 0x0080
36 .DQ @exception_handler_p3
37
38 ;;;;;; MAIN CODE SECTION
39
40 .ORG 0x0200
41 :start
42
43 ; Enable all four maskable hardware interrupt lines (bits 9-12 of SR)
44 ORRI sr, #0b0001_1110_0000_0000
45
46 ; Setup routines
47 BRSR @setup_serial
48 BRSR @print_help
49 ; Make sure this is after the initial help message print (see the
   ; subroutine for more info)
50 BRSR @enable_serial_recv
51
52 :wait_for_char
53 ; Wait for exception (will spin until interrupted)
54 WAIT
55
56 ; Pending byte should be in r7 after exeception handler runs
57 CMPI r7, $EXPECTED_CHAR
58 BRAN |== @finish
59
60 BRAN @wait_for_char
61
```

```

62 :setup_serial
63
64 LOAD r1, #9600
65 LOAD ah, $SERIAL_DEVICE_SEGMENT
66 LOAD al, $SERIAL_DEVICE_BAUD
67 STOR (a), r1
68
69 RETS
70
71 :enable_serial_recv
72
73 ; When piping data to stdin, an EOF character gets sent which closes stdin
74 ; This means that we can't trigger any more interrupts manually after the
   data is piped
75 ; (stdin is currently the only way to externally trigger an interrupt)
76 ; If the data is coming in at the same type we are sending data out,
77 ; it ends up with the program gets stuck waiting for an interrupt and never
   finishes
78 ; TODO: Actually that shouldn't be a problem - better investigate this
   further (
79 ; maybe the exception handler routines are conflicting with each other)
80
81 LOAD r1, #0x1
82 LOAD al, $SERIAL_DEVICE_RECV_ENABLED
83 STOR (a), r1
84
85 RETS
86
87 :print_help
88
89 LOAD r1, @help_message
90 BRAN @print
91
92 :print_exit
93
94 LOAD r1, @exit_message
95 BRAN @print
96
97 :print
98
99 LOAD ah, $SCRATCH_SEGMENT
100 LOAD al, $MESSAGE_SEND_BASE
101 STOR (a), r1
102
103 ; Align pointer
104 LOAD r1, #1
105
106 LOAD ah, $SCRATCH_SEGMENT
107 LOAD al, $MESSAGE_SEND_POINTER
108 STOR (a), r1
109
110 LOAD r1, $TRUE
111 LOAD ah, $SERIAL_DEVICE_SEGMENT
112 LOAD al, $SERIAL_DEVICE_SEND_ENABLED
113 STOR (a), r1
114
115 :wait_for_print_finish
116
117 LOAD ah, $SERIAL_DEVICE_SEGMENT

```

```
118 LOAD a1, $SERIAL_DEVICE_SEND_ENABLED
119 LOAD r1, (a)
120
121 CMPI r1, $TRUE
122 BRAN |= @wait_for_print_finish
123
124 RETS
125
126 :finish
127
128 BRSR @print_exit
129
130 ; Halt CPU
131 COPI #0b0001_0100_1111_1111
132
133 ;;;;;; EXCEPTION HANDLERS
134
135 .ORG 0x0400
136 :exception_handler_p3
137
138 ; Save the link register
139 LDEA s, (l)
140
141 BRSR @read_pending_byte
142 BRSR @write_pending_byte
143
144 ; Restore the link register
145 LDEA l, (s)
146
147 RETE
148
149 :read_pending_byte
150 ; Check if there is something we need to read
151 LOAD ah, $SERIAL_DEVICE_SEGMENT
152 LOAD al, $SERIAL_DEVICE_RECV_PENDING
153 LOAD r1, (a)
154
155 ; Return early if not
156 CMPI r1, $FALSE
157 RETS |=
158
159 ; Read pending byte
160 LOAD al, $SERIAL_DEVICE_RECV_DATA
161 LOAD r7, (a)
162
163 LOAD al, $SERIAL_DEVICE_RECV_PENDING
164 LOAD r1, $FALSE
165 STOR (a), r1
166 RETS
167
168 :write_pending_byte
169 ; Check if the device is ready to write
170 LOAD ah, $SERIAL_DEVICE_SEGMENT
171 LOAD al, $SERIAL_DEVICE_SEND_PENDING
172 LOAD r1, (a)
173
174 ; Return early if not (there is already a send pending)
175 CMPI r1, $TRUE
176 RETS |=
```

```

177
178 LOAD ah, $SCRATCH_SEGMENT
179 LOAD al, $MESSAGE_SEND_POINTER
180 LOAD r2, (a)
181 LOAD al, $MESSAGE_SEND_BASE
182 LOAD al, (a)
183
184 LOAD ah, $PROGRAM_SEGMENT
185 LOAD r3, (r2, a)
186
187 ; If the message buffer (zero terminated) has been sent. Stop sending
188 CMPI r3, #0
189 BRAN |== @stop_send
190
191 LOAD ah, $SERIAL_DEVICE_SEGMENT
192 LOAD al, $SERIAL_DEVICE_SEND_DATA
193 STOR (a), r3
194
195 ; Increment by two because of lack of packing (a word actually is padded to
    a DW)
196 ADDI r2, #2
197 LOAD ah, $SCRATCH_SEGMENT
198 LOAD al, $MESSAGE_SEND_POINTER
199 STOR (a), r2
200
201 LOAD ah, $SERIAL_DEVICE_SEGMENT
202 LOAD al, $SERIAL_DEVICE_SEND_PENDING
203 LOAD r1, $TRUE
204 STOR (a), r1
205
206 RETS
207
208 :stop_send
209 LOAD ah, $SERIAL_DEVICE_SEGMENT
210 LOAD al, $SERIAL_DEVICE_SEND_ENABLED
211 LOAD r1, $FALSE
212 STOR (a), r1
213
214 RETS

```

The data segment below supplies the two zero-terminated, word-per-character messages (`help_message` and `exit_message`) that Example D-4 prints at startup and on exit.

Listing D.5: Example D-5: Hardware exception data

```

1  ;;;;;;;;; DATA
2
3  ; TODO: A string data type that does this for us
4  :help_message
5  .DW #84
6  .DW #121
7  .DW #112
8  .DW #101
9  .DW #32
10 .DW #39
11 .DW #97
12 .DW #39
13 .DW #32
14 .DW #116
15 .DW #111

```

```
16 .DW #32
17 .DW #101
18 .DW #120
19 .DW #105
20 .DW #116
21 .DW #10
22 .DW #0
23
24 :exit_message
25 .DW #71
26 .DW #111
27 .DW #111
28 .DW #100
29 .DW #98
30 .DW #121
31 .DW #101
32 .DW #33
33 .DW #10
34 .DW #0
```

D.4 Software Exception

This short program demonstrates a software-raised exception with **EXCP** and confirms that nesting is blocked while a handler is running. It sets `r1` to 1, raises exception 0x40, and expects `r1` to hold 0xFF once the handler at line 26 has run. That handler itself raises a second exception, 0x41, from inside the handler; because the handler at vector 0x0500 (line 37) sets `r1` to 0xBB only if it runs, the final value of `r1` shows whether the nested exception was correctly ignored.

Listing D.6: Example D-6: Software exception

```
1 ; Reserved space for 256x32 bit exception vectors
2
3 ; First vector is the reset vector
4 .ORG 0x0000
5 .DQ @start
6
7 ; 64th vector (*2 words per vector)
8 .ORG 0x0080
9 .DQ @exception_handler
10 .DQ @exception_handler_that_should_be_ignored
11
12 .ORG 0x0200
13 :start
14
15 ; Reset register
16 LOAD r1, #1
17 ; Trigger Software Exception
18 EXCP #0x40
19
20 ; r1 should be 0xFF here after exception handler runs
21
22 ; Halt CPU
23 COP1 #0x14FF
24
25 .ORG 0x0300
26 :exception_handler
27
28 ; Load a value to detect if this code runs
29 ; Final value of r1 should be this value
30 LOAD r1, #0xFF
```

```

31 ; Try and trigger nested software exception (should be ignored)
32 EXCP #0x41
33 ; Return from exception
34 RETE
35
36 .ORG 0x0500
37 :exception_handler_that_should_be_ignored
38
39 ; Load a value to detect if this code runs
40 ; Final value of r1 should not be BB because this vector should never be
   jumped to
41 LOAD r1, #0xBB
42 ; Return from exception
43 RETE

```

D.5 Store Load

This program is a self-checking test of indexed and immediate-offset memory access, followed by an exhaustive exercise of every **LJMP**/**LJSR** addressing form. The store and load block (lines 8–19) round-trips values through memory using an immediate offset and a register offset from an address-register pair. The remainder of the listing chains together register-offset, immediate-offset, and plain register forms of **LJMP** and **LJSR**; every path not on the intended route lands on an **LJMP** s/**LJSR** s instruction commented **FAIL!**, so a working implementation reaches the `:pass` label at the end while a faulty one jumps into the "forbidden zone" and fails at `:fail`.

Listing D.7: Example D-7: Store/load

```

1 ; Reserved space for 256x32 bit exception vectors
2
3 .ORG 0x0000
4 .DQ @init
5
6 .ORG 0x0200
7
8 :init
9 LOAD r1, #0xCAFE
10 LOAD r3, #0xBEEF
11 LOAD r5, #0x000A
12
13 LOAD ah, #0x0008
14 LOAD al, #0xAAF0
15
16 STOR (#0x000F, a), r1
17 LOAD r2, (#0x000F, a)
18 STOR (r5, a), r3
19 LOAD r4, (r5, a)
20
21
22 ; Lets test jumping!
23
24 ; TODO: Add some tests that:
25 ; LJMP to another segment
26 ; LJMP with offset
27 ; LJMP with shifted offset
28 ; Same three things but with LJSR
29 ; Make a maze of jumps so if the jump goes to the wrong place it will land
   in the forbidden zone where every instruction jumps to the fail label
30
31 ; r7 will store the test result
32 LOAD r7, #0
33 ; Use the stack address register pair to store fail address

```

```
34 LOAD sh, #0x0
35 LOAD sl, @fail
36 ; Address register pair used as part of test
37 LOAD ah, #0x0
38 LOAD al, #0x0
39
40 BRAN #12 ; Step 1
41 LJMP s ; FAIL!
42 LJMP s ; FAIL!
43 BRAN #10 ; Step 3
44 LJMP s ; FAIL!
45 LJMP s ; FAIL!
46 BRAN #-6 ; Step 2
47 LJMP s ; FAIL!
48 BRAN @ljmp_register_offset
49
50
51 :ljmp_register_offset
52 LOAD al, @ljmp_register_offset_anchor
53 LOAD r1, #-6
54 LOAD r2, #4
55 BRAN @ljmp_register_offset_anchor
56 LJMP s ; FAIL!
57 LJMP s ; FAIL!
58 LJMP s ; FAIL!
59 LJMP a, r2
60 LJMP s ; FAIL!
61 LJMP s ; FAIL!
62 :ljmp_register_offset_anchor
63 LJMP a, r1
64 LJMP s ; FAIL!
65 BRAN @ljmp_immediate_offset
66 LJMP s ; FAIL!
67 LJMP s ; FAIL!
68 LJMP s ; FAIL!
69 LJMP s ; FAIL!
70
71
72 :ljmp_immediate_offset
73 LOAD al, @ljmp_immediate_offset_anchor
74 BRAN @ljmp_immediate_offset_anchor
75 LJMP s ; FAIL!
76 LJMP s ; FAIL!
77 LJMP s ; FAIL!
78 LJMP a, #4
79 LJMP s ; FAIL!
80 LJMP s ; FAIL!
81 :ljmp_immediate_offset_anchor
82 LJMP a, #-6
83 LJMP s ; FAIL!
84 BRAN @ljsr_register_offset
85 LJMP s ; FAIL!
86 LJMP s ; FAIL!
87 LJMP s ; FAIL!
88 LJMP s ; FAIL!
89
90
91 :ljsr_register_offset
92 LOAD al, @ljsr_register_offset_anchor
```

```

93 LOAD r1, #-6
94 LOAD r2, #4
95 BRAN @ljsr_register_offset_anchor
96 LJSR s ; FAIL!
97 LJSR s ; FAIL!
98 LJSR s ; FAIL!
99 LJSR a, r2
100 LJSR s ; FAIL!
101 LJSR s ; FAIL!
102 :ljsr_register_offset_anchor
103 LJSR a, r1
104 LJSR s ; FAIL!
105 BRAN @ljsr_immediate_offset
106 LJSR s ; FAIL!
107 LJSR s ; FAIL!
108 LJSR s ; FAIL!
109 LJSR s ; FAIL!
110
111
112 :ljsr_immediate_offset
113 LOAD a1, @ljsr_immediate_offset_anchor
114 BRAN @ljsr_immediate_offset_anchor
115 LJSR s ; FAIL!
116 LJSR s ; FAIL!
117 LJSR s ; FAIL!
118 LJSR a, #4
119 LJSR s ; FAIL!
120 LJSR s ; FAIL!
121 :ljsr_immediate_offset_anchor
122 LJSR a, #-6
123 LJSR s ; FAIL!
124 BRAN @finish
125 LJSR s ; FAIL!
126 LJSR s ; FAIL!
127 LJSR s ; FAIL!
128 LJSR s ; FAIL!
129
130
131 :finish
132 LOAD r7, #0x0FAB
133
134 LOAD sh, #0x0
135 LOAD sl, @pass
136 LJMP s
137
138 .ORG 0x0400
139
140 :fail
141 LOAD r7, #0xFA11
142
143 :pass
144
145 ; Halt CPU
146 COPI #0x14FF

```


CHAPTER E

Quick Reference

This appendix collects the most frequently consulted tables in a single location for use during development. Section E.4 lists where to find additional reference tables defined in the main chapters.

E.1 Instruction Mnemonic Index

All instruction mnemonics in alphabetical order. *Type*: ALU, Memory (Mem), Control-flow (CF), Coprocessor (COP), or Meta. *N, Z, C, V*: uses the symbols defined in Table 11.2 (Chapter 11): * (updated), 0, 1, – (preserved), and S (from the shifter). Rows marked [†] are preserved by the call instruction itself, but the selected coprocessor may define further effects once dispatched. Rows marked [‡] are updated from a named source rather than from the instruction's own result. Meta-instructions inherit the flag behaviour of the underlying CPU instruction. For full encoding details, see the chapter cited in the header of each group.

Table E.1: Alphabetical Mnemonic Index

Mnemonic	Type	Opcode(s)	N	Z	C	V	Description
<i>A – Ch.13</i>							
ADCI	ALU	0x01/0x21	*	*	*	*	Add with carry, immediate or short immediate
ADCR	ALU	0x31	*	*	*	*	Add with carry, register
ADDI	ALU	0x00/0x20	*	*	*	*	Add immediate or short immediate
ADDR	ALU	0x30	*	*	*	*	Add register
ANDI	ALU	0x04/0x24	*	*	0	0	Bitwise AND immediate or short immediate
ANDR	ALU	0x34	*	*	0	0	Bitwise AND register
<i>B – Ch.15 / Ch.17</i>							
BRAN	Meta	LDEA p,...	-	-	-	-	PC-relative conditional branch
BRSR	Meta	LDEL p,...	-	-	-	-	PC-relative call (saves return address to l)
<i>C – Ch.13 / Ch.16</i>							
CMPI	ALU	0x0A/0x2A	*	*	*	*	Compare immediate or short immediate (no write-back)
CMPR	ALU	0x3A	*	*	*	*	Compare register (no write-back)
COPI	COP	0x0F	-	-	-	- [†]	Coprocessor call immediate
COPR	COP	0x3F	-	-	-	- [†]	Coprocessor call register
<i>D – Ch.17</i>							
DIVS	Meta	COPI #0x3300	-	-	-	- [†]	Signed divide (Integer Maths Unit, coprocessor 0x3)
DIVU	Meta	COPI #0x3200	-	-	-	- [†]	Unsigned divide (Integer Maths Unit, coprocessor 0x3)
DMAR	Meta	COPI #0x28xx	-	-	-	-	DMA read into registers (DMA unit, coprocessor 0x2)
DMAT	Meta	COPI #0x2Ann	-	-	-	-	DMA memory-to-memory transfer
DMAW	Meta	COPI #0x29xx	-	-	-	-	DMA write from registers

continued on next page

Table E.1 (continued)

Mnemonic	Type	Opcode(s)	N	Z	C	V	Description
<i>E – Ch. 17</i>							
ETFR	Meta	COPI #0x1Cxy	-	-	-	-	Read exception-unit state
ETTR	Meta	COPI #0x1Dxy	-	-	-	-	Write exception-unit state
EXCP	Meta	COPI #0x11vv	-	-	-	-	Raise user exception (trap) with vector vv
<i>L – Ch. 15 / Ch. 17</i>							
LDEA	CF	0x18–0x1B	-	-	-	-	Load effective address
LDEL	CF	0x1C–0x1F	-	-	-	-	Load effective address and link (saves PC to l)
LJMP	Meta	LDEA p,...	-	-	-	-	Long (absolute) jump
LJSR	Meta	LDEL p,...	-	-	-	-	Long (absolute) call
LOAD	Mem/ALU	0x07/0x14– 0x17/0x37	-	-	-	-	Load from memory (0x14–0x17) or move an immediate or register value (0x07, 0x37)
<i>M – Ch. 17</i>							
MULS	Meta	COPI #0x3100	-	-	-	- [†]	Signed multiply (Integer Maths Unit, coprocessor 0x3)
MULU	Meta	COPI #0x3000	-	-	-	- [†]	Unsigned multiply (Integer Maths Unit, coprocessor 0x3)
<i>N – Ch. 17</i>							
NOOP	Meta	ADDI[N] r1,#0	-	-	-	-	No operation
<i>O – Ch. 13</i>							
ORRI	ALU	0x05/0x25	*	*	0	0	Bitwise OR immediate or short immed- iate
ORRR	ALU	0x35	*	*	0	0	Bitwise OR register
<i>R – Ch. 15 / Ch. 17</i>							
RETE	Meta	COPI #0x1A00	*	*	*	* [‡]	Return from exception handler (SR re- stored from the link register)
RETS	Meta	LDEA p,(#0,l)	-	-	-	-	Return from subroutine
RSET	Meta	COPI #0x1B00	*	*	*	* [‡]	System reset (SR cleared to 0)
<i>S – Ch. 13 / Ch. 14 / Ch. 17</i>							
SBCI	ALU	0x03/0x23	*	*	*	*	Subtract with carry, immediate or short immediate
SBCR	ALU	0x33	*	*	*	*	Subtract with carry, register
SHFT	Meta	ORRI[S] rD,#0	S	S	S	S	Shift with status-register update
STOR	Mem	0x10–0x13	-	-	-	-	Store to memory
SUBI	ALU	0x02/0x22	*	*	*	*	Subtract immediate or short immedi- ate
SUBR	ALU	0x32	*	*	*	*	Subtract register
<i>T – Ch. 13</i>							
TSAI	ALU	0x0C/0x2C	*	*	0	0	Test AND immediate or short (no write- back)
TSAR	ALU	0x3C	*	*	0	0	Test AND register (no write-back)
TSXI	ALU	0x0E/0x2E	*	*	0	0	Test XOR immediate or short (no write- back)

continued on next page

Table E.1 (continued)

Mnemonic	Type	Opcode(s)	N	Z	C	V	Description
TSXR	ALU	0x3E	*	*	0	0	Test XOR register (no write-back)
<i>W – Ch. 17</i>							
WAIT	Meta	COPI #0x1900	-	-	-	-	Idle until an enabled hardware interrupt or a reset occurs
<i>X – Ch. 13</i>							
XORI	ALU	0x06/0x26	*	*	0	0	Bitwise XOR immediate or short immediate
XORR	ALU	0x36	*	*	0	0	Bitwise XOR register

[†] Preserved by the coprocessor-call instruction itself; the selected coprocessor may define further effects once the call is dispatched. [‡] **RETE** restores all four flags from the link register; **RSET** clears all four flags to zero as part of clearing `sr`.

E.2 Status Register Bit Reference

The status register (`sr`) is 16 bits wide. The high byte (bits 15–8) is privileged: reads in protected mode return zero for those bits. In protected mode any instruction that names `sr` as its destination raises a privilege violation fault, regardless of which bits the write would change.

Bit	Name	Access	Reset	Function
15	T	Privileged	0	Trace mode: fault after every instruction (used by debuggers)
14	A	Privileged	0	Address overflow trap: fault when an address calculation overflows
13	EA	Privileged	0	Exception Active: set automatically while inside an exception handler (read-only)
12	E4	Privileged	0	Enable hardware interrupt line 4 (level 4, highest maskable)
11	E3	Privileged	0	Enable hardware interrupt line 3
10	E2	Privileged	0	Enable hardware interrupt line 2
9	E1	Privileged	0	Enable hardware interrupt line 1 (level 1, lowest maskable)
8	P	Privileged	0	Protected mode: 0 = supervisor, 1 = protected
7–4	–	Either	0	Reserved for future use; software must not rely on the value read
3	V	Either	0	Overflow: signed arithmetic overflow occurred
2	C	Either	0	Carry: unsigned carry out or borrow
1	N	Either	0	Negative: result bit 15 = 1 (two's complement sign bit)
0	Z	Either	0	Zero: result was zero

Table E.2: Status Register Bit Summary

Hardware interrupt line 5 (level 5) is a Non-Maskable Interrupt (NMI) and is always enabled; it has no enable bit in `sr`. See Chapter 5 for full details and examples.

E.3 Bus Access Type (BAT) Reference

The 3-bit Bus Access Type field is driven onto the BAT pins during every bus cycle. External devices (memory controllers, coprocessors) can inspect these pins to determine the nature of the access.

BAT[2:0]	Value	Meaning
000	0	No I/O (internal cycle, no bus transaction)
001	1	Instruction Fetch
010	2	Data Read
011	3	Data Write
100	4	Exception Vector Fetch
101	5	DMA Coprocessor Read Burst
110	6	DMA Coprocessor Write Burst
111	7	Reserved

Table E.3: Bus Access Type (BAT) Encoding

See Chapter 2, Section "Pin Descriptions", for usage examples including the PROT pin interaction.

E.4 Where to Find Other Tables

Table	Chapter / Appendix	Label
Complete opcode map (all 64 opcodes)	Appendix A	A.1
Register encoding (4-bit IDs)	Chapter 3	3.1
Condition code encoding	Chapter 7	7.5
Shift type encoding	Chapter 7	7.6
Addressing mode summary	Chapter 8	8.1
Exception vector table	Chapter 6	—
Instruction timing (cycle counts)	Appendix B	—
Complete instruction list by opcode	Chapter 12	12.1
Meta-instructions cross-reference	Chapter 17	17.1
Flag effect symbols	Chapter 11	11.2

Table E.4: Reference Table Locations

CHAPTER F

External Interface

F.0.1 Chip Layout

The SIRC CPU is most commonly provided as a DIP-64 package.

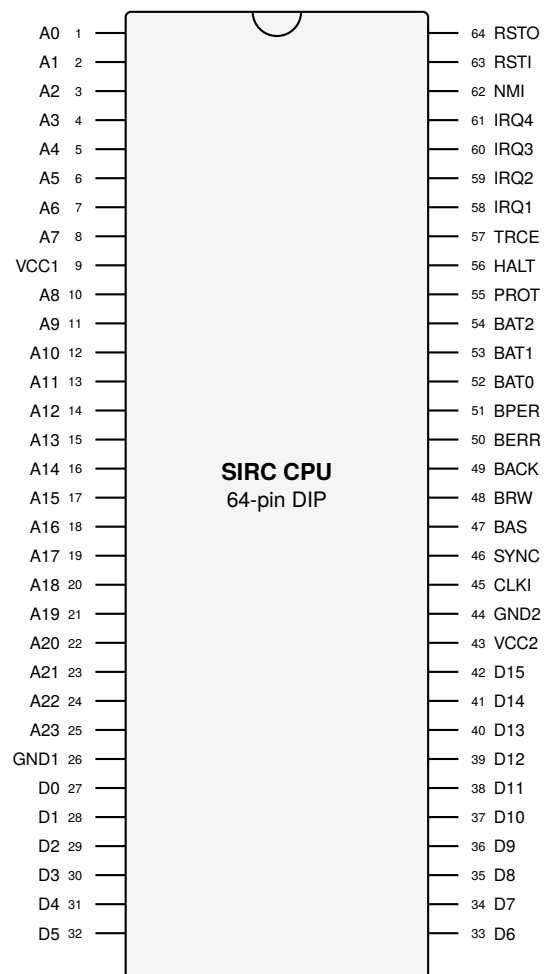


Figure F.1: SIRC CPU 64-pin DIP pinout, top view

Pin	Name	Description
1-8	A0-A7	Address bus bits 0-7
9	VCC1	Power supply
10-25	A8-A23	Address bus bits 8-23
26	GND1	Ground
27-42	D0-D15	Data bus bit 0-15
43	VCC2	Power supply
44	GND2	Ground
45	CLKI	Clock input
46	SYNC	Instruction-sync output
47	BAS	Bus access strobe output
48	BRW	Bus read/write output
49	BACK	Bus acknowledge input
50	BERR	Bus error input
51	BPER	Bus protection error
52-54	BAT0-BAT2	Bus access type bits 0-2
55	PROT	Protected-mode output
56	HALT	External CPU halt input
57	TRCE	Force trace mode input
58-61	IRQ1-IRQ4	Maskable interrupt inputs 1-4
62	NMI	Non-maskable interrupt input
63	RSTI	Reset input
64	RSTO	Reset output

Table F.1: SIRC CPU 64-pin DIP pinout

Each signal lists its own assertion convention. Address, data, and bus-type value pins use positive logic, while bus control, handshake, interrupt, halt, trace, and reset pins generally use active-low conventions. The descriptions below use "asserted" and "deasserted" to refer to the logical state of a signal according to its signal type.

F.0.2 Pin Descriptions

Address Bus Bits (A0-23)

- **Direction:** Output
- **Signal Type:** Active High

Connected to the address bus.

These 24 pins declare the address on the bus that the CPU reads from or writes to when BAS (bus access strobe) is asserted.

Power Pins (VCC1-2, GND1-2)

These pins provide the 5V power that the CPU requires to function.

The VCC pins must be connected to 5V +/- 5% and GND pins must be connected to a 0V or ground rail.

Data Bus Bits (D0-D15)

- **Direction:** Bi-directional

-
- **Signal Type:** Active High

Connected to the data bus.

These 16 pins are driven to a 16-bit value when the CPU is outputting to the bus, and are an input for a 16-bit value when the CPU is reading from the bus.

Clock Input (CLKI)

- **Direction:** Input
- **Signal Type:** Rising Edge
- **Max Rate:** 24 MHz

Progresses the CPU to the next cycle when the input transitions from low to high (rising edge).

Each transition from low to high advances the CPU's internal six-stage sequence by one stage, so every six clock transitions complete one full instruction execution when no bus wait states occur.

Instruction Sync Output (SYNC)

- **Direction:** Output
- **Signal Type:** Active High

Asserted while the CPU is in the first execution phase of an instruction or exception-unit dispatch. It remains asserted if a wait state holds that phase, and is deasserted during every other phase.

It can be useful for debugging purposes.

Bus Access Strobe Output (BAS)

- **Direction:** Output
- **Signal Type:** Active Low

When the CPU is ready to perform a bus operation, BAS is asserted after the address pins, BRW, BAT, PROT, and any CPU-driven data pins are valid.

BAS remains asserted until the bus responds with BACK, BERR, or BPER. If no response is asserted, the CPU holds the current execution phase and repeats the same bus request on the following clock.

Bus Read/Write Output (BRW)

- **Direction:** Output
- **Signal Type:** Read High / Write Low

Only relevant when the BAS (Bus access strobe output) is asserted.

When high, the CPU is requesting a read from the bus. When low, the CPU is requesting a write to the bus.

Bus Acknowledge Input (BACK)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write completed successfully. On reads, the CPU samples D0-D15 on the clock edge that accepts BACK.

If all bus devices are fast and can complete reads/writes in a single cycle, this pin can be permanently held asserted.

Bus Error Input (BERR)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write was unsuccessful.

This causes the CPU to abort the I/O operation and raise a bus fault exception.

An example of a situation where a bus error might be raised would be I/O to an address that is not connected to any device.

Bus Protection Error Input (BPER)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write was to a valid address/device, but that the I/O is disallowed given the current state of the CPU or bus.

This causes the CPU to abort the I/O operation and raise a bus protection fault exception.

It behaves much like the BERR pin but raises a different fault so that programs can handle it differently.

An example of a situation where a bus protection error might be raised would be a write to ROM, or a write to a privileged section of memory when the CPU is not in supervisor mode.

Bus Access Type Bits (BAT0-BAT2)

- **Direction:** Output
- **Signal Type:** Active High

Output by the CPU to describe what I/O operation it is performing.

Can be used to provide useful hints to external devices during I/O.

For example, if a memory controller knows that the DMA coprocessor will be reading more than one sequential address, it can prefetch the next address.

Another example might be a memory controller providing memory protection if these pins are used in conjunction with the PROT (Protected-mode output) pin. A memory controller might trigger a bus error if BAT is 010 (data read), PROT is 1 (protected mode) and the address is in a protected memory region.

BAT2	BAT1	BAT0	Description
0	0	0	No I/O
0	0	1	Instruction Fetch
0	1	0	Data Read
0	1	1	Data Write
1	0	0	Exception Vector Fetch
1	0	1	DMA Coprocessor Read Burst
1	1	0	DMA Coprocessor Write Burst
1	1	1	Reserved

Table F.2: Bus Access Type Bits Meaning

Protected-Mode Output (PROT)

- **Direction:** Output
- **Signal Type:** Active High

Indicates whether the CPU is in protected mode (high) or in supervisor mode (low).

Connected directly to the status register Protected Mode (P) bit.

When the CPU is in protected mode, programs have access to fewer types of instructions, for example they cannot write to the high word of the address registers.

Exposing this as an external pin allows other devices such as memory controllers to block access to protected regions of memory by raising bus protection errors.

External CPU Halt Input (HALT)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the CPU stops executing instructions.

HALT is sampled at instruction boundaries. If asserted, the CPU completes the current instruction and halts before the next instruction fetch. It resumes as normal when the input is deasserted.

This pin is useful for hardware-driven debugging, as the CPU can be halted and the state of the system examined.

Force Trace Mode Input (TRCE)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted at an instruction boundary, the instruction being started is traced regardless of whether software has cleared the Trace Mode (T) flag in the status register.

When in trace mode, the CPU raises an exception on every instruction to allow software debugging.

Maskable Interrupt Inputs (IRQ1-IRQ4)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the corresponding hardware interrupt line is sampled by the CPU.

They can be "masked" by the Hardware Interrupt Enable Bits (E1-E4) of the status register, which disables them.

Interrupt inputs are level-sensitive and are sampled at instruction boundaries. Enabled interrupt lines are latched in the pending hardware-exceptions register; disabled maskable lines are ignored and not queued. A line that is asserted while its enable bit is set is latched at the moment of assertion, even if it is released before the next instruction boundary; only an assertion that begins and ends while the enable bit is 0 is lost. If an interrupt pin remains asserted after its handler returns, it may be sampled again through the normal interrupt rules.

See Chapter 6 for more details.

Non-Maskable Interrupt Inputs (NMI)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the CPU samples the Level 5 hardware exception line. Unlike IRQ1-IRQ4, which are level-sensitive, NMI is edge-triggered: it fires once on assertion regardless of how long the pin is held, and the pin must be released and reasserted to fire again.

This interrupt cannot be masked, and if this pin is asserted while another level 5 exception is being handled, a "level five hardware exception conflict" fault is raised.

Because NMI is edge-triggered, holding the pin asserted after its handler returns does not by itself cause it to fire again – a fresh falling-then-rising transition is required. Therefore, it is important that the level 5 exception handlers are executed quickly and that NMI is only asserted for important non-interruptible events.

Reset Input (RSTI)

- **Direction:** Input
- **Signal Type:** Active Low

Can be used by external devices to force a reset of the CPU.

When asserted, the CPU immediately aborts the current instruction or bus wait, stops normal processing, resets to phase 0, and begins the reset sequence described in Chapter 6.

The RSTO pin is also asserted.

When this pin is deasserted, the CPU completes the reset-output hold, stops asserting the RSTO pin, and the exception unit fetches the reset vector.

If a system needs external devices to be reset for longer than the CPU reset-output hold, reset timing must be managed externally to the CPU, and RSTI must remain asserted for as long as the other devices need to reset.

Reset Output (RSTO)

- **Direction:** Output
- **Signal Type:** Active Low

Asserted while the RSTI pin is asserted and during the reset-output hold before reset-vector fetch.

It is also asserted for the reset-output hold after a software reset.

A reset can be triggered by asserting the Reset input (RSTI) pin, or in software by a special exception coprocessor meta-instruction (RSET).

This pin is useful to reset other devices on the bus whenever the CPU is reset or to trigger more complex reset/boot controllers.

F.0.3 Timing Notes

Clocked Bus Model

All bus timing is described relative to the rising edge of CLKI. The CPU presents a bus request during the execution phase that needs external access. If no response is asserted, the CPU repeats the same request and does not advance to the next execution phase.

A no-wait-state bus access assumes that the selected external device can assert `BACK` and, for reads, drive valid data during the same CLKI interval in which `BAS` is asserted, so the CPU accepts the transfer on the next rising edge. A device that samples the request on one clock edge and cannot return `BACK` or read data until a later edge adds wait states; this is the normal way to attach slower synchronous memory or peripherals.

Signal	Timing rule
A0–A23	Valid whenever BAS is asserted. Stable until the request completes or aborts.
D0–D15	Driven by the CPU only for writes. On reads, the selected device drives the bus and the CPU samples D0–D15 when BACK is accepted.
BRW	Valid whenever BAS is asserted. High requests a read; low requests a write. Stable until the request completes or aborts.
BAT0–BAT2	Valid whenever BAS is asserted. Encodes the access type for the active request and is stable until completion or abort.
PROT	Reflects the status register P bit and may be used with BAS, BAT, and A0–A23 by external protection logic.
BAS	Asserted while a bus request is active. Remains asserted across wait states.
SYNC	Asserted while phase 0 (Instruction Fetch, high word) of an instruction or exception-unit dispatch is active.

Table F.3: CPU bus output timing rules

When BAS is deasserted there is no active bus transaction. External devices must ignore A0–A23, D0–D15, BRW, and BAT0–BAT2 unless BAS is asserted.

Bus Transaction Completion

Response	Effect when sampled while BAS is asserted
BACK	Completes the request successfully. For reads, D0–D15 is sampled on the accepting clock edge. For writes, the write is considered externally committed.
BERR	Aborts the request and raises a bus fault. Read data is ignored and writes are not considered successfully committed by the CPU.
BPER	Aborts the request and raises a bus protection fault. Read data is ignored and writes are not considered successfully committed by the CPU.
No response	Inserts a wait state. The CPU remains in the current execution phase and keeps A0–A23, D0–D15 for writes, BRW, BAT0–BAT2, PROT, and BAS stable.

Table F.4: Bus response timing rules

A bus transaction terminated by Bus Error or Bus Protection Error is architecturally unsuccessful. An external device must not commit any externally visible effect from a transaction for which it asserts the corresponding error signal: for a write, the device must not act on the data it was driven; for a read, any side effect the access would otherwise have (for example, a status register that clears on read) must not occur either. This is what makes Bus Fault and Bus Protection Fault safe to retry (Chapter 6) – the CPU can assume that a transaction it aborted with one of these signals never happened.

BACK, BERR and BPER are logically mutually exclusive. If more than one is asserted for the same request, BERR has priority, then BPER, then BACK.

Bus Cycle Types

Cycle	CPU drives	External device drives	Completion
Read	A0–A23, BRW=1, BAT, BAS, PROT	D0–D15 for the selected address	BACK samples data; BERR/BPER aborts.
Write	A0–A23, D0–D15, BRW=0, BAT, BAS, PROT	No data bus drive	BACK accepts write; BERR/BPER aborts.
Wait state	Same values as the previous request cycle	Same ownership as the active request	Continues until BACK, BERR, or BPER is asserted.
No bus cycle	No active transaction	No device drives D0–D15	CPU advances internally.

Table F.5: External bus cycle ownership

Bus Timing Diagrams

The timing diagrams below show cycle-level logical timing. Each labelled interval represents one CLKI cycle as observed between rising edges. Waveform rows show physical logic levels; for active-low signals, **L** means asserted and **H** means deasserted. Address, data, BAT, and phase rows are value bands rather than electrical setup/hold specifications. The diagrams use **IF** for instruction fetch, **DR** for data read, **DW** for data write, **EV** for exception vector fetch, **EA** for effective address, and **RV** for reset vector.

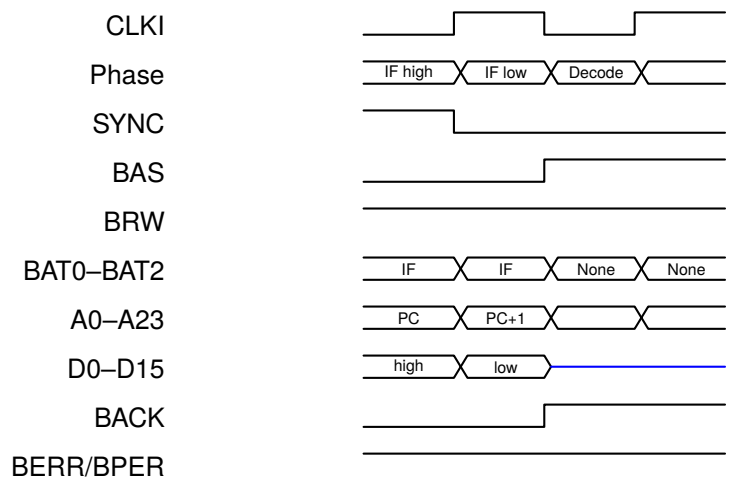


Figure F.2: Instruction-fetch high-word and low-word reads with no wait states

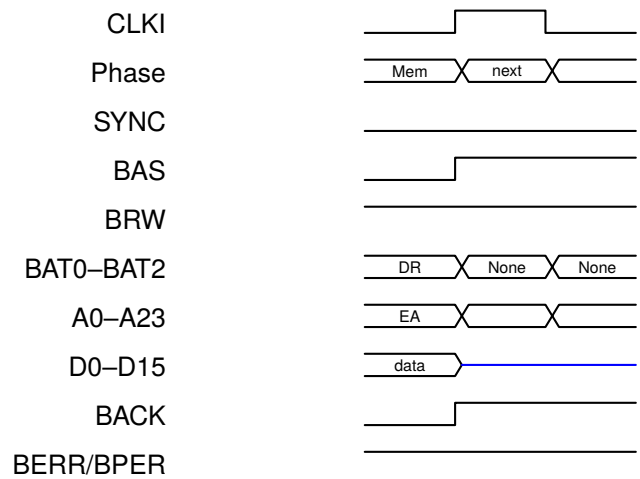


Figure F.3: Data read cycle with no wait state

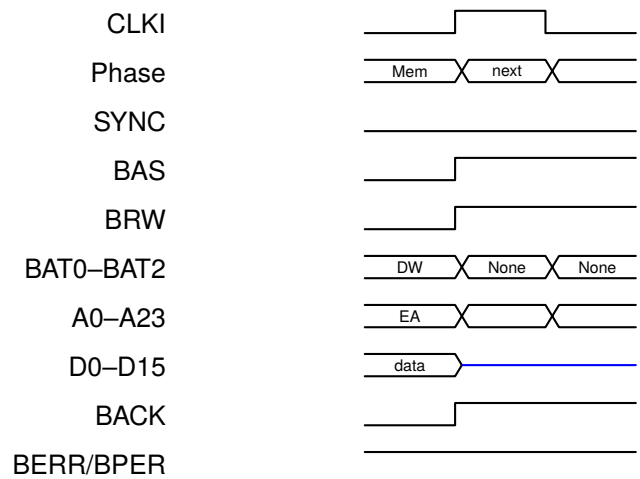


Figure F.4: Data write cycle with no wait state

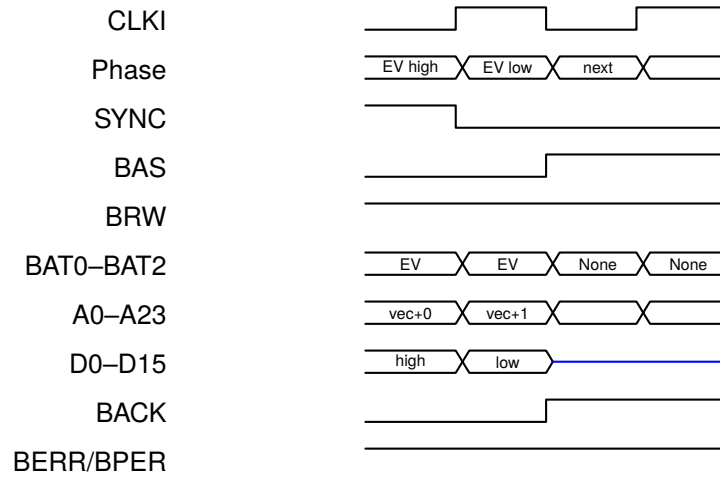


Figure F.5: Exception-vector fetch with high-word then low-word reads

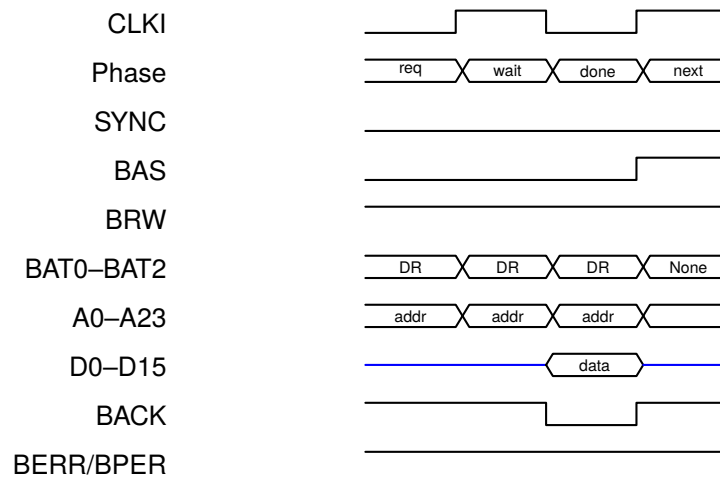


Figure F.6: Wait-state insertion when no bus response is asserted

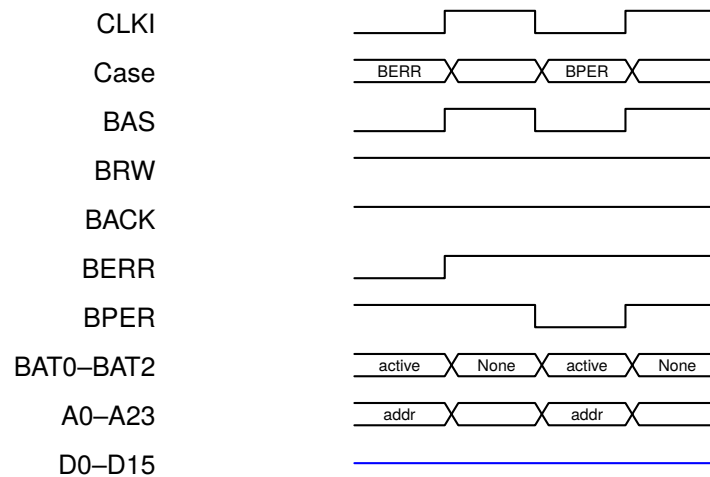


Figure F.7: Bus-error and bus-protection abort timing

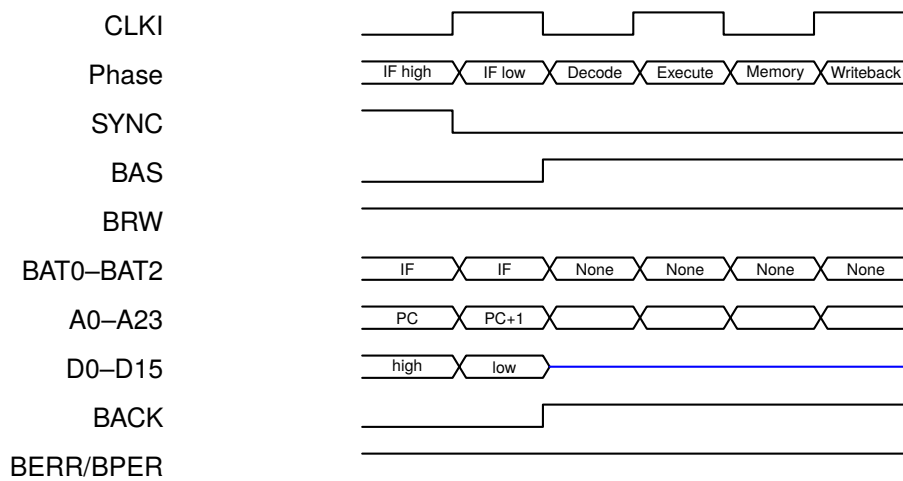


Figure F.8: Complete register-only ALU instruction with no wait states

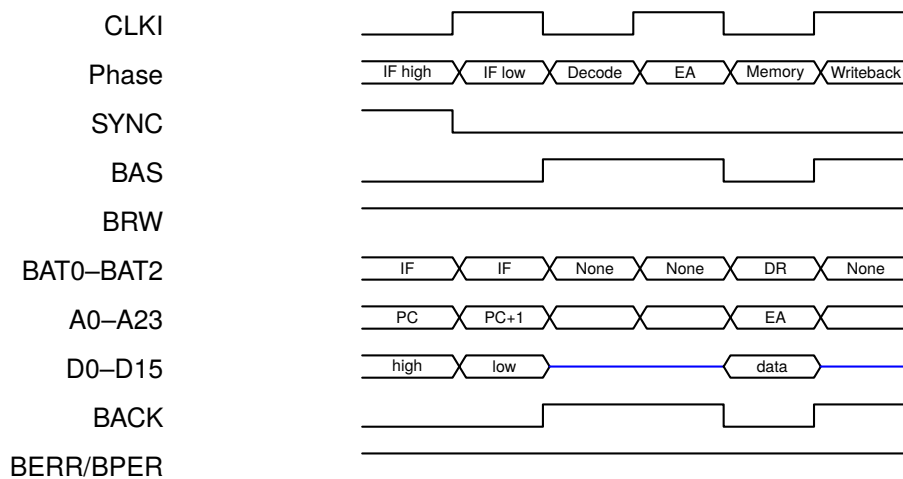


Figure F.9: Complete LOAD instruction with no wait states

Reset and Interrupt Timing Examples

Figure F.10 shows a hardware reset where `RSTI` is asserted for one clock. Holding `RSTI` asserted for longer extends the reset-output interval; the reset-vector fetch does not begin until `RSTI` is deasserted and the reset-output hold has completed.

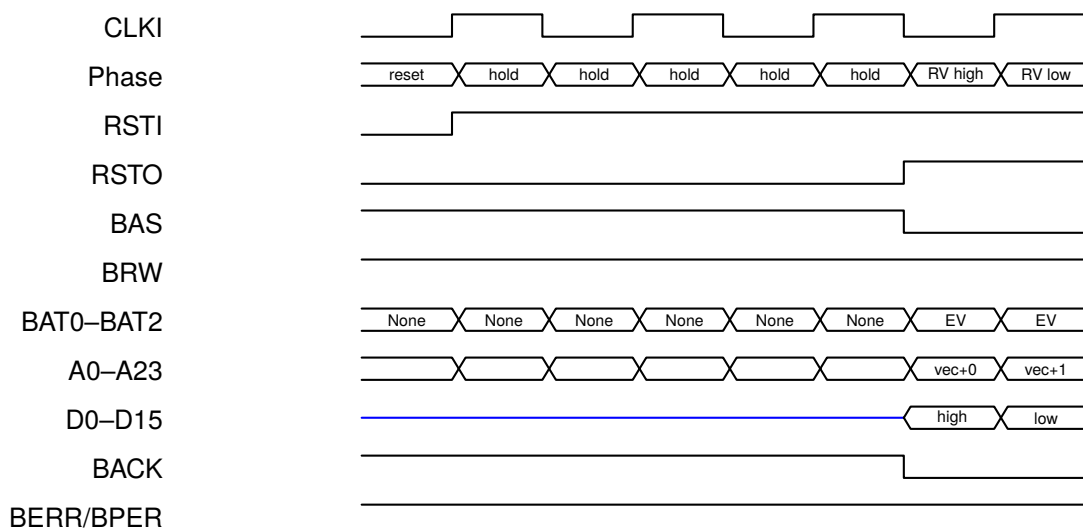


Figure F.10: Hardware reset with one-cycle RSTI assertion and reset-vector fetch

Figure F.11 shows an enabled maskable interrupt asserted before an instruction boundary. The CPU samples and latches the interrupt at the boundary. The following exception-unit dispatch fetches the selected hardware-interrupt vector instead of fetching the next processing-unit instruction. If an interrupt line is asserted after an instruction boundary, it is not sampled until the next boundary.

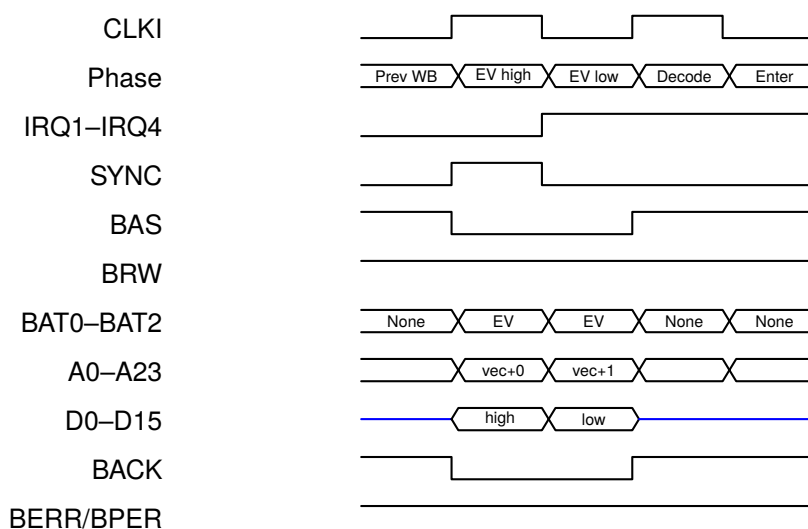


Figure F.11: Enabled IRQ sampled at an instruction boundary and dispatched with no wait states

Asynchronous and Boundary-Sampled Inputs

Input	Sampling rule
BACK, BERR, BPER IRQ1–IRQ4	Sampled while BAS is asserted. They complete or abort the active bus request. Level-sensitive. A line asserted while enabled is latched at that moment, even if released before the next instruction boundary; the exception unit dispatches the latched request at that boundary.
NMI	Edge-triggered. Fires once on assertion regardless of how long the pin is held; the pin must be released and reasserted to fire again.
HALT	Sampled at instruction boundaries. If asserted, the CPU halts before the next instruction fetch and resumes when deasserted.
TRCE	Sampled at instruction boundaries. If asserted, the instruction being started is traced.
RSTI	May be asserted at any time. It aborts the current instruction, exception dispatch, or bus wait and starts the reset sequence.

Table F.6: External input sampling rules

This manual defines cycle-level logical timing only. Electrical setup, hold, propagation delay, voltage, fanout, and maximum-board-loading requirements are outside the architectural specification and should be defined by a future datasheet or hardware implementation note.